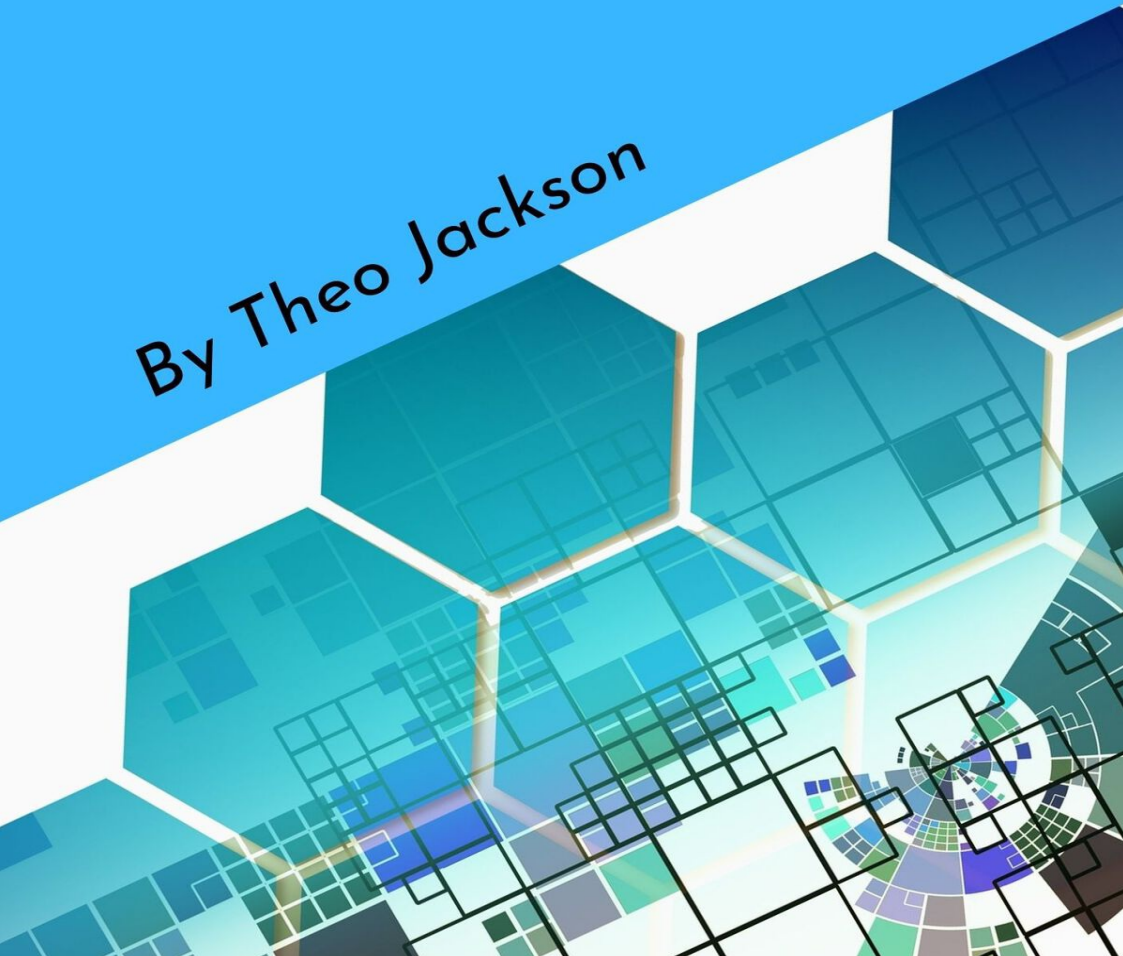


Data Structure and algorithms' in C language

*A step by step guide in C
language*

By Theo Jackson



Data Structure and algorithms' in C language

**A step by step guide in C
language**

Theo Jackson

Contents

Introduction

1 DATA TYPES

AND OPERATORS

Variables and base data types

Constants

Several words about the main ()

function

INPUT AND WITHDRAWAL IN C

Standard I / O

Formatted input Output

Conditional operator

Switch operator

Loop Operators

Break and continue statements

Examples of

Tasks

SEQUENCE PROCESSING

Examples of

Tasks

ONE-DIMENSIONAL ARRAYS

Getting Started with Arrays

Examples of working with arrays

Tasks

MULTI-DIMENSIONAL ARRAYS

Defining and initializing two-dimensional arrays

Tasks

POINTS AND ARRAYS

Pointers and Addresses

Function pointers and arguments

Pointers and Arrays

Const modifier

Variable size arrays

Variable size 2D arrays

SYMBOLS AND STRINGS

Representation of symbolic information
in a computer

Symbol handling library

C strings

String processing functions

String conversion functions

Examples of working with strings

Breaking a string into tokens

Tasks

STRUCTURES

Understanding Structures

Associations

DIRECTIVES PREPROCESSOR

Directive #include

#Define directive

Directive #undef

FUNCTIONS

Function Basics

Function prototypes

Memory classes

Function pointers

Recursion

Examples using recursion

Recursion problems

WORK WITH MEMORY BITS

Bitwise operations

Tasks

WORKING WITH FILES

Files and streams

Text files

Binary files

Tasks for text files

Binary Tasks

DATA STRUCTURES

Singly linked lists

Examples of working with singly

[linked lists](#)

[Singly linked list tasks](#)

[Stacks, queues](#)

[Tasks per stacks and queues](#)

[Doubly linked lists](#)

[Doubly linked list problems](#)

[Binary trees](#)

[Examples using binary trees](#)

[Binary tree problems](#)

Introduction

The C language is one of the most popular and powerful programming languages. This language has a rich set of operators and allows you to write down expressions in a compact way. It is widely used in the development of system and application programs.

This language is structured, modular, and also portable, that is, programs written in it can be easily installed on any platform.

The C language was created by Denis Ritchie to develop the UNIX operating system and is implemented within this system. C is a general-purpose programming language well known for its efficiency and economy.

This language supports pointers to variables and functions. If you use pointers wisely, you can write efficient programs because pointers

allow you to refer to objects in the same way a computer does. The C language supports pointer arithmetic, which allows direct access and manipulation of data.

The C language is a compiling type language. The source code for the object module is compiled in two stages. First, the preprocessor takes effect, which processes directive lines starting with the # sign, after which the program text is translated and the object (machine) code is created.

1 DATA TYPES AND OPERATORS

Variables and base data types

Computer memory can be viewed as a sequence of numbered cells. The cell numbers are called memory addresses. The address allows you to refer to a particular memory cell. Each cell is one byte in size. You can work with these cells individually or in whole blocks.

Variables in the C language are used to store information. A variable can occupy one or several memory cells, in which some values can be stored. To define a variable, you must specify its type and name:

```
variable_type variable_name;
```

Variable type - a characteristic that determines the format of data representation in the computer memory, the set of admissible values of these data and the totality of operations on them. Knowing the type of the variable, the compiler will allocate the required number of memory cells for it and will know what kind of data will be stored in this variable.

Note that reserving memory for a variable does not mean that the allocated memory cells will be "cleared" of the values that were previously stored in them. Therefore, if you do not initialize the variable, then its value can be arbitrary.

There are several basic types in the C language: char - a character;

int is an integer;

float - single precision floating point number;
double is a double precision floating point number.

There are also several qualifiers that are used

in conjunction with base types. The short and long qualifiers apply to int. The short qualifier is used when it is known in advance that a variable will accept “sufficiently” small integer values. If a variable can take large integer values, then a long modifier. For example, below are two variables named i and j, where i is a short int and j is a long int:

```
short int i; long int j;
```

In the definitions of short int and long int, the word int can be omitted. Therefore, you can write

```
short i; long j;
```

The signed and unsigned qualifiers are applied to the char, int, short int, long int types. Variables with the unsigned qualifier are always non-negative. If a variable is declared without the unsigned qualifier, then, by default, it is considered signed.

Below is a table of the basic data types showing the required amount of memory and range of values.

Note that integers of type `int` are available in both 16-bit and 32-bit sizes. The size of this type depends on the version of the compiler and the brand of the computer. When using an integer variable

Signed type of the most significant bit of a number is interpreted as a sign (0 - positive number, 1 - negative). Therefore, the range of values, for example, for 16-bit integers of type `int` is obtained as follows: from -215 to 215-1.

The value of a variable of type `char` is a code (of size in

1 byte of memory) corresponding to the represented character.

The long double type is intended for calculations with high precision.

To find out the size of an object, you can use the `sizeof ()` operator, which returns the size of an object in bytes. For example, `sizeof`

(int), sizeof (double), etc. You can also use the name of the variable, for example, after declaring double x, you can find out the size of the variable x by writing the instruction sizeof (x). The sizeof () operator is executed at the compilation stage of the program, the value of which will be a constant equal to the number of bytes of the object.

The header files <limits.h> and <float.h> define symbolic constants that represent limit values for various data types. For example, the constant INT_MIN (INT_MAX) represents the minimum (maximum) possible value that can be of type int; DBL_MIN, DBL_MAX - respectively the minimum and maximum values for positive variables of the double type, etc. Below is a table of symbolic constants from the <limits.h> file.

The C language has a strict requirement: all declarations of variables within a block must be placed at the beginning of this block,

before the first statement.

Overflow of unsigned integer types. If the value of a variable with an unsigned integer type goes beyond the boundaries of its range (from 0 to M , where M is the maximum allowable maximum value), then this variable will store the number modulo the number $M + 1$.

For example, suppose the variable `a` is of type unsigned short. Let's write the value of the symbolic constant `USHRT_MAX` (`a = USHRT_MAX`) into the variable `a`. That is, the variable `a` contains the number 65535. If now the value of the variable `a` is increased by one (`a ++`), then the variable `a` will take the value 0. If now we subtract one from `a` (`a --`), then the variable `a` will again take the value 65535.

Overflow of signed integer types. For example, suppose the variable `a` is now of type short and `a = SHRT_MAX` (that is, `a = 32767`). If we add one to `a` (`a ++`), then the variable `a` will take the value `SHRT_MIN`,

that is, `a` will have the minimum allowed negative value. Conversely, if the variable `a` has the value `SHRT_MIN` and subtract one from `a` (`a--`), then the variable `a` will take the value `SHRT_MAX`.

Type conversions. If operands of different base types appear in an expression, they are converted to some general type according to the following general rules.

- If any of the operands is of type long double, then the rest are cast to long double.
- Otherwise, if any of the operands is of type double, then the others are also cast to double.
- Otherwise, if any of the operands is of type float, then the rest are converted to float.
- Otherwise, if any of the operands is of type unsigned

long, then the rest are cast to type unsigned long.

- Otherwise, if any of the operands is of type long, then the rest are converted to type long.

- Otherwise, if any of the operands is of type unsigned int, then the rest are cast to type unsigned int.

- Otherwise, if the operands are of type unsigned short and / or unsigned char, then they are converted to unsigned int.

- Otherwise, all operands are cast to type int.

Operators and Expressions

Arithmetic operators. These include

addition (+),

subtraction (-), multiplication (*), division (/), modulo (%). Division of integers is accompanied by the discarding of the fractional part. The expression `a % b` gives the remainder when `a` is divided by `b`. For example, after executing the next part of the program

```
int a = 5, b = 3, q, r; q = a / b;
```

```
r = a % b;
```

the variable `q` will be 1 and `r` will be 2.

Note that the `%` operator only applies to integers. Relational and logical operators.

These include the following operators: greater than or equal ($\geq$), greater than ($>$), less than or equal ($\leq$), less than ($<$), equal ($==$), not equal ($!=$), Log

logical AND ($\&\&$), logical OR ($\|\|$), negation NOT (!).

Any logical expression can be either true or false. Boolean value "true" in C language is represented by any integer non-zero value, and the value

"False" is zero.

For an arbitrary expression A, the following equalities will hold:

$A \&\& \text{false} = \text{false}$, $A \&\& \text{true} = A$, $A \|\| \text{false} = A$,

$A \|\| \text{true} = \text{true}$.

Expressions that contain the $\&\&$ and $\|\|$ operators are evaluated from left to right, and

evaluation of the expression value stops as soon as it becomes clear whether the result will be true or false. Note that the precedence of the `&&` operator is higher than the precedence of the `||` operator.

For example, let the integer variables `x` and `y` be defined with initialization of the initial values:

```
int x = 1, y = 2;
```

In logical terms

```
... (x > y && y > 0) ...
```

first, the value of the expression `x > y` is evaluated, which in this case is false (zero). Since for a logical expression of the form `A && B` to be true, where `A` and `B` are some expressions, it is necessary and sufficient that both `A` and `B` be true expressions, the expression `(x > y && y > 0)` will be false meaning. Therefore, the compiler will not evaluate the value of the expression `y > 0`.

Similarly, let `x = 1, y = 2`. Then the value of

the expression

... (y > x && x > 0 || x == y && x > 0) ...

will be true, since the expression y > x && x > 0 is already true, so there is no need to calculate the value of the expression x == y && x > 0 (recall that an expression of the form A || B is true if and only if at least one of the expressions A or B).

It follows from the above that it is more efficient to put the first operand of the && operator with the one with the highest probability of being false. And for the operator ||, on the contrary, as the first operand it is better to put the one with the greatest probability of being true.

Assignment operator. The general view of the assignment operator is as follows:

variable_name = expression For example:

```
double x, y, z; /* define three valid variables
* / x =0.2; /* to the
variable x we assign the value 0.2 * / y = 0.5;
```

/ * to the variable y we assign the value 0.5 *
/ z = x + y; / * to variable z we assign the sum
of values x and y * /

Since the result of an expression with an assignment operator is the value of the expression to the right of the assignment operation, assignment expressions can be nested, for example $x = y = 0$. Here, the assignment $y = 0$ will be performed first, after which the variable y will be zero, and then this value will be assigned to the variable x.

In the C language, the following rule is accepted: any expression with an assignment operator, enclosed in parentheses, has a value equal to the assigned expression. For example, the expression $(x = 1 + 2)$ has the value 3. Therefore, you can write this expression:

$$y = (x = 1 + 2) + 4,$$

where the variable y is assigned the value 7, and the variable x will have the value 3. Given this circumstance, you can also write logical expressions with an assignment

operator, for example

$((x = 1 + 2) < 4),$

which in this case is true.

In the C language, the following special arithmetic operators are available for compact notation of expressions involving assignment operators:

Increment and decrement operators. Very often in programs, one is added to a variable, or one is subtracted from a variable. Increasing the value by 1 is called an increment, and decreasing by 1 is called a decrement. For these actions, special operators are provided: the increment operator (++), which increases the value of a variable by 1, and the decrement operator (-), which, in turn, decreases the value of the variable.

value by 1. For example, suppose the variable x is of type double. The following three

expressions will produce the same result:

$$\begin{array}{lcl} x ++; x & & = x + 1; x + \\ = & 1; & \end{array}$$

The increment and decrement operators work in two flavors: prefix and postfix. The prefix variant is written before the variable name (`++ x`), and the postfix variant is written after it (`x ++`). In simple expressions like

`x++`; and `++ x`;

both options are equivalent. In a complex expression (if the expression contains an assignment operator and an increment or decrement operator), the prefix and postfix variants will lead to different results. The prefix operator is executed before the assignment operator, and the postfix operator is executed after it. For example, after the next part of the program, `y` will have the value 2:

`x = 1;`

`y = ++ x;`

and after such lines of the program

```
x = 1;
```

```
y = x ++;
```

the variable y will have the value 1. The variable x in both cases will have the value 2.

Empty operator. This operator consists of a semicolon (;) and is used to denote the empty body of a control operator.

Comments in the text of programs. Comment is a set of characters that are ignored by the compiler. A comment can be placed anywhere in the program and occupy one or more lines. Within the character set representing a comment, there must be no / * and * / characters representing the beginning and end of a comment, respectively. Example of a comment:

```
/ * Comment text * /
```

Constants

Constants in the C language can be string, character, integer and real. To define a constant, you need to tell the compiler its name and value. To define it, the `#define` directive is used, which has the following syntax:

```
#define name value
```

There is no semicolon after the directive. Before starting to generate object (machine) codes, the compiler substitutes its value for every constant name it encounters.

To define a numeric constant, you must specify its name and value. For example, let there be a directive

```
#define PI 3.14
```

Then the preprocessor, before translating the program, will replace all PI names in it with

the numerical value 3.14:

```
/* source program */ #include <stdio.h>

#define PI 3.14 int main ()

{

double x = 2 * PI; printf ("% f \ n", PI * PI);
return 0;

}/* after preprocessing */

int main ()

{

double x = 2 * 3.14; printf ("% f \ n", 3.14 *
3.14); return 0;

}
```

When defining a constant, its type is not specified. The compiler itself will determine

what type the declared constant belongs to, by its value. In the above example, the constant PI will be defined as a real floating point number, since 3.14 is a real number.

If you declare

```
#define N 100
```

then the constant N will be assigned to the integer type, since the number 100

is whole.

For the declared constant to be classified as a float, the number must have at least one decimal place, for example

```
#define X 2.0
```

If a constant of a character type is declared, then it must be enclosed in single quotes:

```
#define LETTER 'a'
```

If you need to declare a string as a constant, then it must be enclosed in double quotes:

```
#define SENTENCE "This is a string  
constant"
```

To memorize a string constant, one byte is used for each character and the end-of-line character is automatically added to it - the character '\ 0', which has a zero code in the ASCII character table. This character is a line terminator.

Note that the sizeof () operator can also be used for constants, for example:

```
#include <stdio.h> #define EXP 2.711828  
#define GREETING "Hi" int main ()  
  
{  
  
printf ("% d \ n", sizeof (EXP)); printf ("% d \n", sizeof (GREETING)); return 0;  
  
}
```

In this case, the size of the string constant GREETING will have a value of 3, since the '\0' character is taken into account. The same result will be obtained after the instruction.

```
printf ("%d \n", sizeof ("Hi"));
```

Several words about the main () function

The C language is a procedural language. Any program written in this language consists of one or more functions, among which the main () function must be present, since the program always starts with this function when it starts.

Note that the main () function returns int. This is written in the C99 standard (October 1999): “The function that causes the program to run is called main. The implementation does not declare a prototype for this function. It must be defined by returning an integer type int ... If the return type is not compatible with type int, the completion state returned to

the host environment is unspecified. " That is, if the return type of the main () function is different from the int type, then the compiler is free to return any state to the host environment (basically, this is the operating system).

Of course, for small programs, returning a value to the operating system is not so important. But still, some operating systems provide for the ability to check the value returned by the program. Therefore, it is convenient to return a zero value as the code for the normal termination of the function.

INPUT AND WITHDRAWAL IN C

Standard I / O

There are a number of C library functions that provide a standard I / O system for C programs. All input and output is provided through so-called streams — sequences of characters that are organized into lines and terminated with newlines.

Standard output... The putchar () function is the simplest console I / O function. It prints a single character to the screen:

```
int putchar (int c)
```

As a result, this function returns the character c passed as a parameter, in case of an error - EOF. The parameter of this function must refer to one of the following types of data: / * character constant * / #include <stdio.h> #define LETTER 'A'

```
int main ()
```

```
{
```

```
    putchar (LETTER); return 0;
```

```
}
```

```
/* character literal */
```

```
#include <stdio.h> int main ()
```

```
{
```

```
    putchar ('A');
```

```
    putchar ('\n'); return 0;
```

```
}
```

```
/* character variable */
```

```
#include <stdio.h> int main ()  
  
{  
  
char letter = 'A'; putchar (letter); return 0;  
  
}
```

The following are control characters, some of which allow you to move the cursor around the screen:

The puts () function outputs the string to the screen: int puts (const char * s)

As a parameter, this function must receive

one of the following types of data: / * string
constant * / #include <stdio.h>

```
#define MESSAGE "a message"
```

```
int main ()
```

```
{
```

```
puts (MESSAGE);
```

```
return 0;
```

```
}
```

```
/ * string literal * / #include <stdio.h>
```

```
int main ()
```

```
{
```

```
puts ("a message"); return 0;
```

```
}
```

```
/ * string variable * / #include <stdio.h>
```

```
int main ()  
  
{  
  
    char message [] = "a message";  
  
    puts (message); return 0;  
  
}
```

Note that a string literal is a sequence of characters enclosed in double quotes.

When the puts () function is called, it displays the value of the passed parameter on the screen, and then places the cursor at the beginning of the next line. This function returns a non-negative number as a value if

no error occurred while outputting string s, and if an error occurs, it returns EOF.

Standard input... The gets () function enters a string into s until it encounters a newline or end-of-file indicator. After that, the delimiter character '\0' is appended to the character array s:

```
char * gets (char * s)
```

The parameter of this function is the name of the variable. For example:

```
#include <stdio.h> int main ()  
  
{  
  
char s [10];  
  
gets (s); / * input string from keyboard * /  
puts (s); / * line output to the screen * / return  
0;  
  
}
```

The `gets ()` function will treat the first 9 characters entered from the keyboard as the value of the string variable `s`, and one character is reserved for the null character `'\0'`. Therefore, you can actually enter one character less.

The `gets (char * s)` function returns the string `s`. For example, in the following example, a request is made to enter a line, after which the entered line is displayed along with the value of its length, and so on until instead of entering the next line, the ENTER key is simply pressed.

```
#include <stdio.h> #include <string.h> int  
main ()  
  
{  
  
char s [10];  
  
while (strlen (gets (s))> 0) printf ("% s% d \n", s, strlen (s));  
  
return 0;
```



```
}
```

Function **getchar ()** enters a single character from the keyboard: `int getchar (void).`

The `getchar ()` function returns the input character or, if end-of-file is found, EOF as the result of each call it calls.

```
#include <stdio.h> int main ()  
  
{  
  
char c;  
  
c = getchar (); putchar (c); putchar ('\ n');  
return 0;  
  
}
```

As an example, let's give a program that asks

for a character and outputs the ASCII code of the entered character:

```
#include <stdio.h> int main ()  
  
{  
  
char c;  
  
c = getchar ();  
  
printf ("The ASCII code of% c is% d \ n", c,  
c); return 0;  
  
}
```

When using the gets () function, it is necessary to control the input of characters so as not to enter more characters than stated in the line (including the null character) where these characters are entered. The following program reads characters from the standard input device into a character array s,

controlling the number of characters entered:

```
#include <stdio.h> int main ()  
  
{  
  
char c, s [10]; int i = 0;  
  
while ((c = getchar ()) != '\n' && i < 9) s [i  
++] = c;  
  
s [i] = '\0'; puts (s); return 0;  
  
}
```

But it is better to use the `fgets ()` function instead of the `gets ()` function, which allows you to specify the size of the character array that you want to fill with keyboard input. Specifying the size of the symbolic array of this function makes it possible to avoid the error of going out of its limits, which is not provided by the `gets ()` function. A small drawback of `fgets ()` is that it does not remove the trailing `'\n'` character, as `gets ()` does.

Below is an example using the `fgets ()` function. For more information on this function, see Chapter 13, "Working with Files".

```
#include <stdio.h> int main ()  
  
{  
  
    char s [10]; fgets (s, 10, stdin); printf ("% s",  
    s); return 0;  
  
}
```

Returning to the `getchar ()` function, note that it can also be used to suspend a program:

```
int main ()  
  
{
```

```
... / * Program * / getchar ();  
  
return 0;  
  
}
```

Formatted input Output

Formatted output... Despite the fact that the functions `putchar ()` and `puts ()` are used quite often, their capabilities are somewhat limited. These functions cannot provide numeric output.

The `printf ()` function allows you to print all types of data to the screen and take multiple arguments as parameters. This function converts the internal representation of variables (in computer memory) into text format:

```
int printf (char * format, argument_1,  
argument_2,..., argument_n)
```

The `printf ()` function converts, formats, and prints its arguments to standard output under the control of the format. As a result, it returns the number of characters printed. The format control string contains conversion specifiers, flags, field widths, precision, and literal characters.

Basic transformations of the `printf ()` function:

To set the field width when displaying the value of a variable, you need to insert an integer number in the conversion specification between the percent sign (%) and the conversion specifier. For example, `printf ("% 10d", 15)` prints an integer 15 with a field width of 10.

If you need to specify the precision of representation, then you should place a decimal point (.) in the conversion specification between the percent sign (%) and the conversion specifier, followed by an integer specifying the precision of the representation.

data transfer. For example, after the printf statement (`“%. 2f”, 0.14788`)

the number 0.15 will be displayed on the screen.

If precision is used when printing a string variable, then this will mean the maximum number of characters in the string that will be printed. For example, after the instruction `printf (“%. 3s”, “abcde”)` the word abc will appear on the screen.

Field width and precision can be combined, for example `printf (“% 10.2f”, 0.14788)`.

Below are examples of using the `printf ()` statement.

The `sprintf ()` function does the same as the `printf ()` function, except that the output is stored in the string array (the first argument to the function), rather than being displayed on the screen:

```
int sprintf (char * string, char * format,  
argument_1, argument_2, ..., argument_n)
```

For example:

```
int main ()  
  
{  
  
char s [10];  
  
sprintf (s, "a =% d", 50); puts (s);  
  
return 0;  
  
}
```

Then the string s will have the value “a = 50”.

Formatted input. The scanf () function reads characters from the standard input stream, interprets them according to the format, and sends the results to its arguments.


```
int scanf (char * format, & argument_1, &
argument_2,..., & argument_n)
```

As a result, the scanf () function returns the number of successfully entered data items. For example:

```
int main ()

{

double x;

while (scanf ("% lf", & x) == 1) printf ("%
10.2f \n", x);

return 0;

}
```

Let's see how the scanf () function works. Let the standard input stream contain a sequence of characters. First, the format control string

is scanned from left to right. If the next character of this line is a space character " ", or a tabulation character '\ t', or a line break character '\ n' (we will call these characters whitespace), then all consecutive whitespace characters are skipped in the input stream, until another character is encountered. Further, if the format control line contains a format that begins with a % sign, then a sequence of contiguous characters up to the first whitespace is read from the input stream. The read sequence of characters is converted into an internal value in the computer memory in accordance with the format type, which is passed to the address of the next argument (& argument_i). If an illegal character for this format is encountered in the sequence of read characters, then in accordance with this format, all characters from the read sequence to this illegal character are converted.

If the format control string contains a character other than a whitespace character that is not included in any format, then when reading the next character from the input

stream, a check for equality of these characters is performed. If the data characters will turn out to be different, then the `scanf ()` function terminates its work ahead of schedule (returning the number of successfully converted and passed to arguments values), and the mismatching character remains in the input stream. If these characters match, then this character is skipped and the `scanf ()` function will continue its work.

For example, let the `scanf ()` function has the following arguments: `int a;`

```
scanf ("a =% d", & a);
```

To correctly enter a number into the variable `a`, you must enter from the keyboard a line like

```
a = 10
```

One of the important differences between the `scanf ()` function and the `gets ()` and `fgets ()` functions when entering a stream of characters into the string variable `s` is as

follows. The `scanf ()` function reads characters from standard input and writes them to `s` until it encounters a whitespace character (`"`, `\ t`, `\ n`). The functions `gets ()` and `fgets ()` write characters from the input stream to string `s` until a `\ n` character is encountered. Thus, you cannot use the `scanf ()` function to enter some sentence containing whitespace into the `s` variable.

For example, after defining the string `char s [100]` and the `scanf` instruction (`"% s", s`), an attempt was made to enter the sentence "Hello world! \ n " will cause string `s` to contain the value "Hello, ". If, instead of the `scanf ()` function, we use the `gets ()` function, then the string `s` will contain "Hello, world!". If, in this case, the `fgets ()` function is used, then the following value will be written to the string `s`: "Hello, world! \ n ".

The `sscanf ()` function reads characters from string:

```
int scanf (char * string, char * format, &
argument_1,..., & argument_n)
```

This function scans the string according to the format

format and transfers the received values to argument_1, argument_2,

..., argument_n.

The datatypes and conversion specifications for the printf and scanf statements are listed below.

CYCLES AND COMPARISON OPERATORS

Conditional operator

The conditional operator allows you to check

a certain condition and, depending on the results of the check, perform one or another action. The structure of the conditional operator is as follows:

```
if (expression)  
  
instruction_1; else  
  
instruction_2;
```

moreover, the else-part may be absent. If the expression in parentheses is true (nonzero), then instruction_1 is executed, and if false (equal to zero) and the else-part is present, instruction_2 is executed. Instead of one, you can use a set of instructions, for this they must be enclosed in curly braces.

Since the if statement tests the numeric value of the expression in parentheses, in some cases, when an expression is tested for equality or non-zero, the condition can be

written in an abbreviated form. In this case, an entry of the form

```
if (expression! = 0)
```

can be rewritten as follows: `if (expression)`

According to the C convention, the word `else` always refers to the closest `if` condition. For the compiler to correctly interpret the nested `if-else` constructs, you need to properly place the curly braces. For example, in the two cases below, the word `else` refers to different `if` conditions: in the first case, `else` refers to the second condition, and in the second, to the first.

```
if (a > 10) if (b < 10)
```

```
    a = 0;
```

```
else
```

```
    b = 0;
```

```
if (a > 10)
```

```
{
```

```
if (b <10) a = 0;
```

```
}
```

```
else
```

```
b = 0;
```

The C language has an operator that works with three operands at once. This is the conditional operator “? :”. The syntax for using this operator is as follows:

```
expression_1? expression_2: expression_3
```


This line is read as follows: if expression_1 is true, then expression_2 is executed and the value of expression_2 becomes the value of the entire conditional expression. Otherwise, expression_3 is executed, and its value becomes the value of the entire conditional expression. For example, to write to the variable max the maximum value of two numbers a and b, you can write it like this:

```
max = (a > b)? a: b; / * max = max {a, b} * /
```

that is, the variable max is assigned the value of the expression

```
(a > b)? a: b.
```

Switch operator

The switch statement allows you to choose one of several alternatives. This operator has the following general form:

```
switch (expression)

{

case constant-expression_1: instructions_1;
break;

...

case constant-expression_n: instructions_n;
break; default: instructions_ (n + 1);

}
```

First, the value of the whole expression in parentheses is calculated, and then it is compared with all expression constants. All values of constant expressions must be different. When the value of the expression in parentheses matches the value of one of the expression constants, the corresponding instructions are executed. If none of the constants match, then the instructions with the default keyword are executed. If default is absent and none of the constants match, then

nothing happens.

For example, the following program checks if an odd or even digit has been entered:

```
#include <stdio.h> int main ()  
  
{  
  
    int a; printf ("a =");  
  
    scanf ("%d", & a); switch (a)  
  
    {  
  
        case 1: case 3: case 5: case 7: case 9: printf  
("odd digit entered"); break;  
  
        case 0: case 2: case 4: case 6: case 8: printf  
("an even digit is entered"); break;  
  
        default:
```

```
printf ("not a number entered");  
  
}  
  
return 0;  
  
}
```

Loop Operators

C language constructs allow us to consider three loop operators: for, while and do-while.

For statement can be formally written as follows: for (expression_1; expression_2; expression_3) body of the loop The body of the loop is either one instruction or a set of instructions enclosed in curly braces. Expression_1 is usually used to initialize the initial values of the variables that control the loop (control variables). Expression_2 defines the conditions under which the body of the loop will be executed. Expression_3 determines the change in the values of control variables. The for loop runs as follows:

1. Expression_1 is evaluated.
2. Expression_2 is evaluated.
3. If the value of expression_2 is true (non-zero), then the body of the loop is executed, and then expression_3 is evaluated and the transition to expression_2 is performed. If the value of expression_2 is false (zero), then the execution of the for statement ends. In the absence of expression_2, it is always assumed to be true.

For example, after executing the loop

```
for (i = 1; i <= 10; i++) printf ("%d \n", i);
```

integers 1, 2, ..., 10 will be displayed on the screen, and after the cycle

```
for (i = 10; i >= 1; i--)
```

```
printf ("%d \n", i);
```

numbers 10, 9, ..., 1 will appear on the screen.

Let's also give an example of a for loop with two control variables:

```
for (i = 1, j = 10; i < j; i ++, j --)  
    printf ("%d %d \n", i, j);
```

The control variable can be symbolic. For example, below is the output to the screen of all lowercase Latin characters:

```
char c;  
  
for (c = 'a'; c <= 'z'; c ++)  
    printf ("%c", c);
```

Any of the three expressions in the loop can be missing. For example, a cycle

```
for (;;) {loop body}
```

is endless.

While statement is formally written as

while (expression) body of the loop

The parenthesized expression can be either null (false) or non-null (true). The body of the loop will only execute if the expression is true. If the expression is false, then the loop ends. For example, displaying numbers 1, 2, ..., 10 can be done as follows:

```
i = 1;

while (i <= 10)

{

printf ("%d \n", i); i ++;

}
```

The for loop operator of the form

for (expression_1; expression_2;
expression_3) the body of the loop can be
replaced with the while loop operator as

follows:

```
expression_1;  
while (expression_2)  
{  
loop body expression_3;  
}
```

Do-while statement. If the check of the truth of the expression of the while loop occurs before the execution of the loop body, then in the do-while loop this check is performed after passing the loop body. The do-while statement is formally written as

```
do body of the while loop (expression);
```

The body of the loop will execute as long as the expression in parentheses is true. If the expression is false upon entering the loop,

then the body of the loop will be executed exactly once.

Break and continue statements

The break statement allows you to exit the for, while, do-while loop statements and switch. This statement allows you to immediately exit the innermost loop or switch. For example, you can run through all points of the segment $[0, 1]$ as follows with a step of 0.1:

```
#include <stdio.h> int main ()  
  
{  
  
double x;  
  
for (x = 0;; x += 0.1) / * endless cycle */
```

```
{  
    if (x > 1) break;  
    else  
        printf("%.1f \n", x);  
}  
  
return 0;  
}
```

The continue statement causes the execution of the loop body to terminate prematurely with a transition to the next iteration step. For while and do-while loops, this means immediately jump to the test of the condition, and for the for loop, jump to expression_3. The continue statement acts on the innermost loop.

As an example of using the continue operator, let us give a program for displaying all odd

digits on the screen:

```
#include <stdio.h> int main ()  
  
{  
  
    int i;  
  
    for (i = 0; i <= 9; i ++ ) / * variable i runs  
    through all digits from 0 to 9 * /  
  
    {  
  
        if (i% 2 == 0) / * if the digit is even, ignore  
        the remaining operators * /  
  
        continue; printf ("% d \ n", i);  
  
    }  
  
    return 0;  
  
}
```

Examples of

Example 1. It is required to compose a program for constructing a table of values of the function $y = ex - x^2$ on the segment $[A, B]$ with step H . In each line of the table, output the value of the argument and the corresponding value of the function in formatted form with 2 decimal places. Also find the minimum among all the values of the function on a given interval with accuracy H and the number of values of the function with an even integer part.

The following program uses the `modf ()` function,

having such a prototype:

```
double modf (double x, double * ip)
```

This function splits a real number x into integer and fractional parts, both parts having the same sign as the original value of x . The integer part is stored in `* ip`, and the fractional part is returned as the result of the function.

```
#include <stdio.h> #include <math.h>

#define A -2 / *          left segment
boundary * /          #define B 2 /
*          rights segment boundary *
/          #define H 0.1 / * step (argument
increment) * /
```

```
/ * the function F () calculates the value of the
function  $y = e^x - x^2$  at the point x * / double
F (double x)
```

```
{

return exp (x) - x * x;

}
```

```
int main ()
```

```
{

double x, y, / * argument and value of
```

```
function y = F (x) * / min,  
/ * minimum value of the function with  
precision H * / q; / * integer part of the  
valuefunctions * /
```

```
int n; / * the number of function values that  
have an even integer part * /
```

```
n = 0; x = A;
```

```
min = F (A); / * initial value of minimum *  
/ while (x <= B)
```

```
{
```

```
y = F (x); / * calculate the value of the  
function  $y = e^x - x^2$  at the point x * /printf  
("% 10.2f% 10.2f \ n", x, y); / * print the x  
and y values to the screen * / if (y <min)
```

```
min = y;
```

```
modf (y, & q); / * write the integer part of y  
to the variable q * / if ((long) q% 2 == 0) / *  
check for evenness * /
```

```
n ++;
```

```

x += H; / * increase          x to H * /

}

printf ("min =% .2f \ n", min); printf ("n =%
d \ n", n);

return 0;

}

```

Example 2. For a non-negative integer number n , find the sum and the number of its digits using the operations of integer division and taking the remainder of the division.

Note that the use of the operators $/$ and $\%$ for integers is based on the following statement: for any pair of integers a and b , where $b > 0$, there is a unique decomposition of the form $a = bq + r$, where q and r are integers and $0 \leq r < b$.

```

#include <stdio.h> int main ()

{

```

```
int n, / * original number
n * / sum, / * sum of digits of
number n * / k; / * amount digits of
the number n * /
```

```
sum = k = 0; / * initial value for sum and
number of digits * /
```

```
printf ("n =");
```

```
scanf ("% d", & n); / * enter the
number n * / do
```

```
{
```

```
k ++; / * take into
account the last digit of the number n * /
```

```
/ * add the last digit of the number n to the
current sum * / sum += n% 10;
```

```
n /= 10; /* discard the last
digit of the number n * /
```

```
} while (n != 0);
```

```
printf ("sum =% dk =% d \ n", sum, k); return
```



```
0;
```

```
}
```

Example 3. Determine if the integer `n` contains the digit `digit`.

```
#include <stdio.h>
```

```
int Search (int n, int digit)
```

```
{
```

```
int d; / * next digit of number n * /          do
```

```
{
```

```
d = n% 10;
```

```
n /= 10;
```

```
}
```

```

while (n! = 0 && d! = digit); return d ==
digit;

}

int main ()

{

int n, digit;

scanf ("% d% d", & n, & digit);

printf ("% s \ n", Search (n, digit)? "yes":
"no"); return 0;

}

```

Example 4. Write a program that prints the prime factorization of a given integer $n > 0$.

The following algorithm uses the following statement: let a be some integer, and $a > 1$, then there is a unique factorization into prime factors of the form

$a = p_1^{k_1} p_2^{k_2} \dots p_n^{k_n}$, where $k_1, k_2, \dots, k_n$ are positive integers,

12 n

$p_1, p_2, \dots, p_n$ are prime numbers and $p_1 < p_2 < \dots < p_n$.

```
#include <stdio.h> int main ()
{
    unsigned int n, / * original number
    n * / i; / * factor of
    the number n * /

    printf ("n =");

    scanf ("%u", & n); / * enter n * /

    i = 2; / * looking for factors, starting with 2 *
    / while (n != 1)
    {
        while (n % i != 0) i ++;
```

```
printf ("%u \n", i); /* we derive the next  
factor i */
```

```
n /= i; /* Divide n by the factor i so that it is  
no longer taken into account */
```

```
}
```

```
return 0;
```

```
}
```

We use the fact that if a natural number has its own divisor, that is, a divisor different from the number itself and unity,

then the value of this divisor does not exceed the root of the given number. Therefore, you can modify the program like this:

```
#include <stdio.h> int main ()
```

```
{
```

```
unsigned int n, i; printf ("n =");  
scanf ("%u", & n); i = 2;  
while (n != 1)  
{  
    if (n % i == 0)  
    {  
        printf ("%u \n", i); n /= i;  
    }  
    else  
        if (i * i > n) /* check if native divisors remain  
            * /  
            i = n; else i ++;  
    }  
    return 0;
```

}

Example 5. Check if a given non-negative integer n is prime.

Recall that an integer is called prime if it has exactly two divisors. Therefore, the number 1 is not prime.

$$\sqrt{n}$$

To write an algorithm, we use the following statement: if a non-negative integer n can be represented as a product of integers a and b ($n = ab$), with $a \geq 2$ and $b \geq 2$, then

or $a \leq [$

], or $b \leq [$

], where $[]$ is the integer part of the number.

That's why

$$\sqrt{n}$$

we will look for divisors of the number n in the range from 2 to $[]$.

```
#include <math.h>
```

```
/ * boolean function to test an integer for
```

simplicity * /

int Prime (unsigned int n)

{

if (n == 0 || n == 1) return 0; else

$\sqrt{n}$

{

int bound = sqrt ((double) n); /

* bound = [int i = 1;

while (++ i <= bound && n% i! = 0)

;

return (i> bound);

}

}

] * /

Example 6 (Euclid's algorithm). Given positive integers n and m . Find their greatest common divisor (GCD) using the following relations:

$$\text{GCD}(n, 0) = \text{GCD}(0, n) = n;$$

$$\text{GCD}(n, m) = \text{GCD}(n - m, m), \text{ if } n \geq m;$$

$$\text{GCD}(n, m) = \text{GCD}(n, m - n) \text{ if } m > n.$$

```
unsigned int Nod (unsigned int n, unsigned
int m)
```

```
{
```

```
while (n != 0 && m != 0) if (n >= m)
```

```
    n -= m; /* subtract m from n */ else
```

```
    m -= n; /* subtract the value n from m */
    /      if (m == 0) return n;
```

```
else return m;
```


}

This version of the Euclidean algorithm can be modified by taking into account the following relationships:

$\text{GCD}(n, m) = \text{GCD}(n, n \% m)$ if $m \neq 0$ and

$\text{GCD}(n, 0) = n$. unsigned int Nod (unsigned

int n, unsigned int m)

{

while (n != 0 && m != 0) if (n >= m)

n %= m; /* in n store the remainder of
dividing n by m */

else

m %= n; /* in m store the remainder of
dividing m by n */

```
if (m == 0) return n; else return m;  
}
```

Tasks

1. Find the sum of all odd (s1) and the sum of all even (s2) integers in the range from 1 to 100.
2. Print all two-digit positive multiples of three without using the conditional operator.
3. Given a natural n , calculate $n!$ ($0! = 1$, $n! = N(n - 1)!$).
4. An integer $n > 0$ is given. Determine if it is a power of 3.
5. Given an integer $n > 0$. Find the

smallest positive integer k whose square is greater than n : $k^2 > n$. Do not use the square root function.

6. An integer $n > 0$ is given. Determine whether the number n can be represented as $n = k^3$, where k is some integer.

7. For two positive integers a and b , find their least common multiple (LCM).

8. Count the number of three-digit natural numbers, in the record of which there are two identical digits.

9. Count the number of three-digit natural numbers in which all three digits are different.

10. Determine if an integer n contains odd digits.

11. Determine if integer $n > 0$ is

symmetric in decimal notation.

12. A natural number n is given. Find the value of the number obtained as follows: discard the digits 0 and 5 from the record of the number n , leaving the same order of the remaining digits.

13. Given a real number x and an integer $n > 0$. Using one cycle, find the sum $1 + x + x^2 + x^3 + \dots + x^n$.

14. You are given a real number x and an integer $n > 0$. Find the value of the expression $x - x^3 / (3!) + x^5 / (5!) - \dots + (-1)^n \cdot x^{2n+1} / ((2n+1)!)$.

15. The Fibonacci sequence is defined as follows: $x_0 = 0$, $x_1 = 1$, $x_n = x_{n-1} + x_{n-2}$ for $n \geq 2$. Let a natural number n be given. Evaluate x_n .

16. Given positive integers n and k . Using only addition and subtraction operations, find the integer part and the

remainder of n divided by k .

17. Determine whether a given natural number n can be represented as the sum of two squares of natural numbers.

18. Write a program that prints all the different prime factors of a given integer $n > 0$.

19. Let a natural number n be given. Print the squares of all numbers from 1 to n without using the multiplication operation (use the equality $(n + 1)^2 = n^2 + 2n + 1$).

SEQUENCE PROCESSING

The sequence of data does not always need to be stored in memory. The sequence can be processed as its elements arrive: when reading a file, when entering some data from the keyboard, and so on.

This section deals with tasks in which the

elements of a numerical sequence are entered one at a time from the keyboard. The sign of the completion of a numerical sequence will be considered the number 0, which is not its element.

Examples of

Example 1. Count the minimum elements of a sequence of real numbers.

```
#include <stdio.h> int main ()  
  
{  
  
    double a, / * value of the current sequence  
    element * / min; / *  
    minimum elementsequences * /  
  
    int n = 0; / * number of minimum  
    elements * / printf
```

```
("a =");
```

```
scanf ("%f", & a);
```

```
min = a; // * initial value of the  
minimum element *  
while (a != 0)
```

```
{
```

```
if (a == min) n ++;
```

```
else
```

```
if (a < min)
```

```
{
```

```
// * if the entered element is less than all  
previous ones * /
```

```
// * elements of the sequence, then override  
min and n * /
```

```
min = a; n = 1;
```

```
}
```

```
printf ("a =");  
  
scanf ("% lf", & a);  
  
}  
  
printf ("n =% d \ n", n); return 0;  
  
}
```

Example 2. Find the maximum number of consecutive positive elements in a sequence of integers.

In the program below, the value 0 is taken as the initial value of the variable max (the maximum number of consecutive positive elements), since the maximum is sought among non-negative elements. If you need to find the minimum or maximum among all the elements of some finite set, the elements of which are arbitrary numbers, then in no case should the number 0, beloved by novice programmers, be taken as the initial value for

min and max. For this case, there are a lot of approaches, for example, an arbitrary element of the set under consideration can be taken as the initial value.

```
#include <stdio.h> int main ()  
  
{  
  
    int a, / * value of the current element of the  
    sequence * / n, / * number of consecutive  
    positive elements * / max; / * max. the  
    number of consecutive positions. elements * /  
  
    max = 0; / * initial value of maximum * /  
    printf ("a =");  
  
    scanf ("%d", & a); while (a != 0)  
  
    {  
  
        n = 0;  
  
        / * count the amount consecutive positive  
        elements * /
```

```
while (a > 0)
```

```
{
```

```
    n ++;
```

```
    printf ("a =");
```

```
    scanf ("%d", & a);
```

```
}
```

```
if (n > max) max = n;
```

```
/* run through all consecutive negative  
elements */ while (a < 0)
```

```
{
```

```
    printf ("a =");
```

```
    scanf ("%d", & a);
```

```
}
```

```
}
```

```
printf ("max =% d \ n", max); return 0;  
  
}
```

The following examples show the clarity of the use of nested loops that run through whole series of elements that have this or that property, as in the previous example.

Example 3. Let there be a sequence of integers. It is required to deduce the lengths of all series of consecutive positive and consecutive negative elements.

```
/ * option with nested loops * /  
  
#include <stdio.h> int main ()  
  
{  
  
int a, n; n = 0;  
  
printf ("a =");
```

```
scanf ("%d", & a); while (a != 0)

{

while (a > 0)

{ /* option without nested loops */

#include <stdio.h> int main ()

{

int a, n_plus, n_minus; n_minus = n_plus = 0;
printf ("a =");

scanf ("%d", & a); while (a != 0)

{

if (a > 0)

{

n ++;

printf ("a =");
```

```
scanf ("%d", &a);

}

if (n > 0)

{

printf ("+:%d\n", n); n = 0;

}

while (a < 0)

{

n++;

printf ("a =");

scanf ("%d", &a);

}

if (n > 0)

n_plus++;
```

```
if (n_minus > 0)

{

printf ("-:%d \n", n_minus); n_minus = 0;

}

}

else

{

n_minus++;

if (n_plus > 0)

{

printf ("+:%d \n", n_plus); n_plus = 0;

}

}

}
```

```
{  
  
printf ("-:% d \ n", n); n = 0;  
  
printf ("a =");  
  
scanf ("% d", & a);  
  
}  
  
}/ * one more check is needed after the loop *  
/  
  
}if (n_minus> 0)  
  
return 0;  
  
}  
  
printf ("-:% d \ n", n_minus);  
  
if (n_plus> 0)  
  
printf ("+:% d \ n", n_plus); return 0;  
  
}
```

Example 4. Find the lengths of sections of strict monotonicity in a sequence of integers. For example, in the sequence 1, 2, 3, 2, 2 there are two sections of strict monotonicity, namely

1, 2, 3 and 3, 2.

```
#include <stdio.h> int main ()
```

```
{
```

```
int a, /* the value of the previous element in  
the sequence */ b, /* value of the current  
sequence element */
```



```

n;    /    *    length    of    the    strict
segmentmonotony
/ printf ("a =");

scanf ("% d", & a); if (a! = 0)

{

n = 1;

printf ("b =");

scanf ("% d", & b); while (b! = 0)

{

/ * section of monotonic increase * / while (b!
= 0 && b> a)

{

n ++;

a = b; printf ("b =");

scanf ("% d", & b);

```

```
}
```

```
if (n > 1)
```

```
{
```

```
printf ("n = %d \n", n); n = 1;
```

```
}
```

```
/* section of monotonic decrease */ while  
(b != 0 && b < a)
```

```
{
```

```
n ++;
```

```
a = b; printf ("b = ");
```

```
scanf ("%d", & b);
```

```
}
```

```
if (n > 1)
```

```
{
```

```
printf ("n = %d \n", n); n = 1;
```

```
}  
  
/ * run through all consecutive identical  
elements */  
  
while (b! = 0 && b == a)  
{  
    printf ("b =");  
    scanf ("%d", & b);  
}  
  
}  
  
}  
  
return 0;  
  
}
```

Tasks

1. Find the sum and the number of elements of the sequence.
2. A number k and a sequence of real numbers are given. Print the number of the first element of the sequence greater than k .
3. Check if the given sequence is ascending.
4. Print the numbers of those elements of the sequence that are less than their left neighbor, and the number of such elements.
5. Find the number of elements in the sequence before the first minimum element.
6. Find the maximum number of consecutive even elements in a sequence of

integers.

7. Let there be a sequence of integers. It is required to deduce the lengths of all series of consecutive even and consecutive odd elements.

8. Find the number of segments of strict ascending sequence in the sequence of integers and display the lengths of these segments.

9. Find the element number of the sequence, with which the longest subsequence of consecutive identical numbers begins, and the number of elements in this subsequence.

Self-study tasks

10. Find the smallest positive number in the sequence.

11. A number k and a sequence of real numbers are given. Print the number of the last element of the sequence greater than k .

12. Print the numbers of those elements of the sequence that are greater than their right-hand neighbor, and the number of such elements.

13. Let there be a sequence of real numbers containing at least two ones. Find the sum of the elements of the sequence located between the first and the last unit.

14. Find the two smallest elements of the sequence and list these elements in ascending order of their values.

15. Find the number of the first minimum and the number of the last maximum elements of the sequence.

16. Find the number of items after the last maximum item in the sequence.

17. Find the number of items located between the first and the last maximum item in the sequence. If the set has a single maximum element, then report it.

18. Find the maximum number of consecutive minimum elements of the sequence.

ONE-DIMENSIONAL ARRAYS

Getting Started with Arrays

Unlike ordinary variables, which store a single value, arrays are used to store a whole set of values of the same type. Array is a set of elements of the same type that are sequentially located in the computer's memory. To define an array, you must specify the type of elements that are included in it and the maximum number of elements that can be placed in the array. For example:

```

#include <stdio.h>    #include <stdlib.h>
#define N 10

int main ()

{

int a [N], / * array a of size N * /           i; /
*               counter * /

for (i = 0; i <N; i ++)

{

a [i] = rand ()% 100; printf ("% d \ n", a [i]);

}

return 0;

}

```

In our case, an array a of 10 integer elements is defined (that is, a block of 10 sequentially located elements of int type), which is filled with random numbers in the range from 0 to

99. Note that the numbering of the array elements starts from 0. Therefore, the elements of the array `a` must be accessed in the following way: `a [0]`, `a [1]`, ..., `a [N-1]`.

The item position number contained in square brackets is called the item index. The index must be an integer or an integer expression.

Once the array is defined, it can be initialized. Initializing an array means that when it is declared, the elements of this array will be assigned initial values. For example,

```
int a [5] = {10, 20, 30, 40, 50};
```

After this initialization, we will have an integer array of five elements with the following values:

```
a[0] = 10, a [1] = 20, a [2] = 30, a [3] = 40, a [4] = 50.
```

If the size of the array is not specified during initialization, then the size is determined by the number of initial values. For example, the above array of 5 integers can be constructed

as follows:

```
int a [] = {10, 20, 30, 40, 50};
```

If there are fewer initialized values of the array than all the elements of the array, then the rest of the elements are automatically initialized to zero. For example, if you initialize an array like this:

```
int a [10] = {0};
```

then all elements of this array will have zero value.

Note that arrays are not automatically initialized to zero. For this purpose, at least one element should be initialized.

To find out the amount of memory occupied by this or that array, you can apply the `sizeof ()` operator to the name of this array, for example:

```
#include <stdio.h> int main ()
```

```

{
int a [10];

printf ("% d \ n", sizeof (a)); return 0;

}

```

Examples of working with arrays

Example 1. Check if the elements of an integer array form an arithmetic progression.

```

#include <stdio.h>  #include <stdlib.h>
#define N 10

```

```

/ * const modifier indicates that element
values * /

```

```

/ * array a and value n cannot be

```

```

changed
/ int Progression (const int * a, const int n)
{
    int d, i;

    d = a [1] - a [0]; i = 1;
    while ((++ i <n) && (a [i] - a [i - 1] == d))

;

    return (i>= n);
}

```

```

int main ()
{
    int i, a [N];

    for (i = 0; i <N; i ++) a [i] = rand ()% 10;

    printf ("% s \ n", Progression (a, N)?

```

```
"progression": "no progression");  
  
return 0;  
  
}
```

Example 2. Determine if all elements of array *a* are pairwise distinct.

In the program below, the flag variable acts as a boolean variable. It is initialized with an initial value of 1 (true), that is, we will assume that by default all array elements are pairwise different. First, we will compare all elements with indices 1, 2, ..., *n*-1 with element *a* [0], then all

elements with indices 2, 3, ..., *n*-1 with element *a* [1], etc. As soon as a pair of identical elements is found, the process stops and the Different function returns a zero (false) value. Otherwise, the function returns 1 (true).

```

int Different (const double * a, const int n)
{
    int i, j, flag; i = 0;
    flag = 1;
    while (i <n && flag)
    {
        j = i;
        while (++ j <n && a [i]! = a [j])
            ;
        if (j <n) / * if j <n then a [i] == a [j] * / flag =
            0;
        else i ++;
    }
    return flag;
}

```

```
}
```

Example 3. It is required to determine the number of different elements of the array.

```
int CountDifferent (const double * a, const int  
n)
```

```
{
```

```
int k, / * number of distinct elements *  
/      i, j;
```

```
k = 0;
```

```
for (i = 0; i <n; i ++)
```

```
{
```

```
j = i;
```

```
while (++ j <n && a [i]! = a [j])
```

```

;

/ * if element a [i] is not among the elements
a [i + 1], a [i + 2],..., a [n-1], * /

/ * then increment counter k * /

if (j >= n) k ++;

}

return k;

}

```

Example 4. Let there be some array of real numbers a and some number x . It is required to rearrange the elements of the array so that at first there follow (in an arbitrary order) elements that are less than the number x , and then the elements that are not less than the number x . Use no more than one pass through the array.

We will view the elements of the array from

left to right until we meet the element $a[i] \geq x$. Then we will go through the elements of the array from right to left until we come across an element $a[j] < x$. After that, we swap the elements $a[i]$ and $a[j]$ and continue this process.

```
void Exchange (double * a, const int n, const  
double x)
```

```
{
```

```
int i = 0, j = n - 1, buf; while (i < j)
```

```
{
```

```
while (i < j && a[i] < x) i ++;
```

```
while (i < j && a[j] >= x) j--;
```

```
if (i < j)
```

```
{
```

```
/* swap places a[i] and a[j] */ buf = a[i];
```

```
a [i] = a [j]; a [j] = buf; i ++;
```

```
j--;
```

```
}
```

```
}
```

```
}
```

Tasks

1. Let there be an array of size n . First, output its elements with even indices, and then - with odd ones.

2. Let there be an integer array a of size n . Print the number of the first of those elements $a[i]$ that satisfy the double inequality $a[1] < a[i] < a[n]$.

3. Swap the minimum and maximum

elements of an array.

4. Cyclic shift of array elements to the right by one position.

5. Let there be an array of integers. Check if it alternates between even and odd numbers.

6. Determine whether the given array a is symmetric, that is, whether the first and last elements are equal, the second and the penultimate, and so on.

7. Determine the number of distinct elements of the array, if it is known that the elements of the given array form an ascending sequence.

8. Given a real number r and an array of size n . Find the element of the array that is closest to the given number.

9. Find the numbers of the two nearest numbers in the array.

10. Determine the maximum number of identical array elements.

11. You are given an integer array of size n . Let's call a group of consecutive identical elements a series, and the length of a series is the number of these elements. Display the lengths of all series in the given array.

12. Remove all zero elements from the array.

13. Find the lengths of sections of strict monotony in the array of integers.

14. You are given two arrays a and b , the elements of which are ordered in ascending order. Combine these arrays so that

the resulting array remains ordered.

15. Order array of size n in ascending order.

16. Let there be an array of real numbers. Print the indices of the array in the order in which their corresponding elements form an ascending sequence.

17. Divide the array into two parts, placing negative elements in the first, and non-negative ones in the second (the order of the elements in each of the groups must remain the same).

18. Let there be an array a , consisting of zeros and ones. It is required to order the array a using at most one pass over the given array.

Self-study tasks

19. Find the smallest element of all array elements of size $n > 2$, whose index is a multiple of 3.

20. You are given an array of size n , where n is an even number. Swap its first element with the second, the third with the fourth, and so on.

21. Determine if the array is in ascending order.

22. You are given two arrays a and b of the same size n . Form a new array c of the same size, each element of which is equal to the maximum of the elements of arrays a and b with the same index.

23. You are given an array a of size n . Form a new array b of the same size according to the following rule: element b_k is equal to the sum of elements of array a with numbers from 1 to k .

24. Increment all even numbers in the array by the original value of the first even number. If there are no even numbers in the array, then leave the array unchanged.

25. You are given an array of size n , all of whose elements, except for the last, are ordered in ascending order. Make the array ordered by moving the last element to the desired position.

26. You are given an array of size n , all but one of which are in ascending order. Make the array ordered by moving the out-of-order item to the desired position.

27. You are given an integer array of size n . Duplicate all even numbers in it.

28. Remove all minimum elements from the array.

29. An integer array of size $n > 2$ is given. Remove from the array all elements

with even numbers (2, 4, ...). Do not use the conditional operator.

30. Remove all identical elements from the array, leaving their first occurrences.

31. You are given an integer array of size n . Determine if it is a permutation.

32. Find the largest number of identical elements of an array that are in a row.

33. Calculate the sum of the elements of the array between the first minimum and last maximum elements.

34. You are given two ascending arrays of integers a and b . Find the number of distinct numbers that are present in both array a and array b .

35. Let there be an array a of real numbers of size n . Write to array b all elements of array a in ascending order and

without repetitions.

36. Form an array of prime factors of a given number.

37. Find all primes between 2 and n . Implement the solution of this problem in the form of the "sieve of Eratosphenes" algorithm, which is based on the following principle. First, in the set of numbers 2, 3, ..., n , cross out all numbers that are divisible by 2, except for 2. Then we consider the number 3 and cross out from this set all numbers that are divisible by 3, except for 3. Next, consider the next uncrossed number after 3, that is, 5, and cross out all subsequent numbers divisible by it. Etc.

MULTI-DIMENSIONAL ARRAYS

Defining and initializing two-dimensional arrays

The C language has the ability to create n-dimensional arrays. The ANSI standard stipulates that the value of n can be at least any number between 1 and 12.

This section will look at two-dimensional arrays.

Such arrays are described as follows: `int a [10] [20];`

That is, this defines a two-dimensional array of integers a, consisting of 10 rows and 20 columns. Each element of a two-dimensional array is uniquely identified by a pair of indices i and j: `a [i] [j]`, where i is the row number and j is the column number. As in the case of one-dimensional arrays, the numbering by the variables i and j starts from zero: `a [0] [0]` - the element located at the intersection of the zero row and zero column ... `a [9] [19]` - the element located at the intersection the ninth line and the nineteenth column.

A two-dimensional array can be initialized when it is declared:

```
int a [3] [2] = {{1, 2}, {3, 4}, {5, 6}};
```

In this case, the values in curly braces are grouped by line. If additional curly braces are not present, then the numbers are distributed along the lines: first the zero line is filled, then the first, and so on. For example, the array a can be initialized like this:

```
int a [3] [2] = {1, 2, 3, 4, 5, 6};
```

If not enough values are specified for any string, then the remaining values of this string are initialized to zeros. For example, if the matrix a is initialized like this:

```
int a [3] [2] = {{1}, {3}, {5}};
```

then the second column of this matrix will contain zero values: If it is necessary to initialize all elements of the matrix with zeros, then for this initialization is enough simply to write the following way:

```
int a [10] [20] = {0};
```

Tasks

1. An $m \times n$ matrix is given. Find the sum of the elements of all its even strings.

2. An $m \times n$ matrix is given. Find the minimum value among the sums of elements of all its columns and the column number with this minimum value.

3. Find the minimum element among the maximum elements of each row of the matrix.

4. Print the number of the first row of a matrix containing an equal number of positive and negative elements, and zero elements are not taken into account.

5. Find the number of rows of

the matrix, all of which are different.

6. You are given a square matrix of order n . Find the sum of the elements of its main and reverse diagonals.

7. You are given a square matrix of order n . It is required to transpose this matrix.

8. You are given a square matrix of order n . Find the sums of the elements of its diagonals parallel to the main one, starting from the one-element diagonal.

9. Given a square matrix of order n . Replace elements that are simultaneously above the main and back diagonals with zeros.

10. Delete all matrix columns containing only positive elements.

11. Find all the distinct elements of an integer square matrix.

12. A matrix is given. Write a program that orders the rows of this matrix in ascending order of the sums of the elements of its rows.

13. You are given an $m \times n$ matrix, where m and n are even numbers. Swap the upper left and lower right quarters of the matrix.

14. Print all elements of the matrix with their indices that are maximum in their row and minimum in their column.

15. Two rows of a matrix will be called equivalent if the sets of elements occurring in these rows coincide. Find the number of strings that are equivalent to a null string.

Self-study tasks

16. Find the minimum value in each column of the matrix.

17. Transform the matrix by swapping the minimum and maximum elements in each row.

18. Print the number of the first row of the matrix, which contains the maximum number of identical elements.

19. You are given a real square matrix of order n . Create a program for calculating the sum of elements located above the main and below the reverse diagonals.

20. Find the maximum element of the matrix. By rearranging its rows and columns, ensure that the maximum element is in the upper left corner of the matrix.

21. Create a program for finding

the number of matrix rows, the minimum element of which is negative.

22. Create a program for finding the minimum of all positive elements in each column of the matrix.

23. Replace the smallest element of each row of the matrix, starting at the second, with the largest element of the previous row.

24. You are given a square matrix of order n . Exclude from it the row and column at the intersection of which the minimum element of the main diagonal is located.

25. Determine if the $m \times n$ array contains identical strings.

26. Determine if the array is a magic square, that is, if the sum of each row, each column, and two diagonals in it coincides.

27. An integer matrix is given. Find the number of the first of its columns containing only odd numbers.

28. Write a program that orders the rows of a matrix in ascending order of the minimum elements of its rows.

POINTS AND ARRAYS

Pointers and Addresses

Pointers are variables whose values are memory addresses. If a “ordinary” variable contains some value, then the pointer contains the address of this or that variable, which contains some value.

A "regular" variable refers directly to a value, while a pointer refers to a value indirectly.

The syntax for creating a pointer is as

follows: `type * variable_name`,

where `type` is the type of the variable whose address will be contained in the pointer, `variable_name` is the name of the variable of the pointer type, symbol

An asterisk means that the variable being declared is a pointer.

Let `x` be some variable, for example, of an integer type (`int`), and `px` - a pointer to a variable of an integer type. The unary operation `&` defines the address of an object:

```
int x, * px;
```

```
px = & x; / *now the variable px contains the  
address of the variable x * /
```

After this assignment, the `px` variable will contain the address of the `x` variable. The only limitation of using the `&` operator is that it cannot be used on an object with a register memory class. Note that the address of the location of the variable is selected by the computer and can be changed at the next

program start.

It is very useful to initialize a pointer with a null value (NULL) when declaring it, if it is not known in advance which address of which variable it will have as its value:

```
double * pd = NULL;
```

Since the pointer contains the address of the object, this makes it possible to indirectly access this object through the pointer. The unary operation `*` (indirect addressing operation) refers to this

address to retrieve the contents of the object referenced by the pointer. For example, the operator

```
printf ("%d", * px);
```

will print the contents of the variable x.

You can use the void keyword as the type of a pointer variable. In this case, the variable-pointer can contain the address of any object, only the unary operation `*` and arithmetic

operations cannot be applied to such a pointer.

Pointers can be compared using comparison operations (<, <=, >, > =, ==, !=). Only pointers to data of the same type should participate in the comparison.

Function pointers and arguments

There are two ways to pass parameters to a function: by value and by reference. In C, arguments are passed to functions by value; the called function has no immediate ability to change the variable from the calling program. For example, in the following program, after calling the Swap (a, b) function, the values of the variables a and b in the main () function remain unchanged.

```
void Swap (int a, int b)
```

```
{  
  
int buf; buf = a; a = b;  
  
b = buf;  
  
}  
  
int main ()  
  
{  
  
int a = 1, b = 2; Swap (a, b); return 0;  
  
}
```

To actually change the values of these variables declared in the main () function, the program must look like this:

```
void Swap (int * pa, int * pb)  
  
{  
  
int buf; buf = * pa;
```

```
* pa = * pb;  
  
* pb = buf;  
  
}  
  
int main ()  
  
{  
  
int a = 1, b = 2; Swap (& a, & b); return 0;  
  
}
```

In this program, the arguments of the Swap () function are passed by reference, that is, the function is passed the addresses of the variables a and b, after which the indirect addressing operation (*) is used to change the values of these variables.

As another example of using pointers that are passed as parameters to a function, we present an algorithm for finding the sum and number of digits of a non-negative integer n:

```
#include <stdio.h>
```

```
void Digits (int n, int * psum, int * pk)
```

```
{
```

```
    * psum = 0;
```

```
    * pk = 0; do
```

```
{
```

```
    (* pk) ++;
```

```
    (* psum) += n% 10; n /= 10;
```

```
    } while (n != 0);
```

```
}
```

```
int main ()
```

```
{
```

```
    int n, / * original number n * / sum, / * sum  
    of digits of a number n * / k; /
```

```

*          number of digits of the number n *
/

printf ("n =");

scanf ("% d", & n); / * enter the number n * /
Digits (n, & sum, & k);

printf ("sum =% d \ n", sum); printf ("k =% d
\ n", k); return 0;

}

```

Pointers and Arrays

First, note that, like any variable, an array variable must have some value. The value of a variable of type array is the address of the zero element of the array:



Therefore, the following assignment is valid:

```
int a [10], * pa;
```

```
pa = a;
```

In this case, the pointer variable `pa` will have the address of the zero element of the array `a` as its value. Note that the assignment `pa = a` can also be written in this equivalent form: `pa = &a [0]`.

Writing or reading data to the array is as follows. To do this, the compiler calculates the address of the corresponding element, taking into account the size of the elements of which the array consists, and the amount of shift relative to the zero element. For

example, to write or read the i -th element of the double `a` [10] array, the compiler will multiply the shift value i by the size of the double element (that is, by 8) and receive $8 * i$ bytes. Then the compiler will add $8 * i$ bytes of shift to the address of the zero element of the array and thereby calculate the address of the i -th element of this array. After that, at this address, the i -th element can be read, or some value at this address can be written.

The compiler has no control over array bounds. If, for example, when writing or reading data, the address `a` [20] is specified, the compiler will write or read into the cell at the address shifted by 160 bytes relative to the address of the zero array element, some value. In this case, the selected memory area may belong to another variable, and the result of the program may be unpredictable.

There are two forms of entry for accessing an array item. The first method (index) is associated with the use of the `[]` operator, for example `a` [0], `a` [5].

The second way is related to the use of address expressions. Note that adding one to an integer variable will increase its value by 1. But if you add one to a pointer variable, then its value will increase by the number of bytes of the type it points to. For example, if you add one to a pointer to float, then the value of this pointer will increase by 4. Therefore, the value of `a[i]` can be written using the address expression: `*(a + i)`. That is, the value `a + i` is the address of the *i*-th element of the array `a`, and `*(a + i)` is the value of the *i*-th element of this array. Note also that `&a[i]` and `a + i` are equivalent. Any expression of the form `a[i]` is always reduced to the form `*(a + i)`, that is, the index expression is converted to address.

Below are two equivalent programs, with the first program always being converted by the compiler to the second.

```
#include <stdio.h>    #include <stdlib.h>
#define N 10

int main ()
```

```

{

int a [N], / * array a of size N * / i; / *
counter*/

for (i = 0; i <N; i ++)

{

a [i] = rand ()% 10; printf ("% d", a [i]);

}

return 0;

}

#include <stdio.h>    #include <stdlib.h>
#define N 10

int main ()

{

int a [N], / * array a of size N * / i; / *
counter*/

```

```
for (i = 0; i <N; i ++)  
  
{  
  
* (a + i) = rand ()% 10; printf ("% d", * (a +  
i));  
  
}  
  
return 0;  
  
}
```

We emphasize once again that the name of an array is not a variable, so you cannot, for example, write `a++`, although you can write `pa++`.

There are three options for passing an array as a parameter to a function:

1. The parameter is specified as an array indicating its size, for example

```
function_name (int a [100])
```

2. The parameter is specified as an array without specifying its size, for example

```
function_name (int a [])
```

3. The parameter is specified as a pointer, for example `function_name (int * a)`

When the compiler meets as a parameter a one-dimensional array written in the form `int`

a [100] or int a [], it converts this parameter to the form int * a.

Const modifier

If the value of the variable should not change, then a const modifier should be placed before its declaration, for example:

```
const int i = 10;
```

This modifier is especially useful when passing arguments to a function. Despite the fact that all calls in the C language are calls by value, in which the functions are passed copies of the arguments and, changing these same copies of the variables, the variables themselves in the calling function remain unchanged, sometimes it is desirable to show that the passed value is not should change. For example, if an array a (or rather, a pointer to the zero element of the array) and its dimension n are passed to the function as arguments, and the elements of the array must not change their value and the dimension n must remain constant, then the function can

be declared as follows:

```
function_name (const int * a, const int n)
```

If an attempt is made to change the value of an argument that was declared with the `const` modifier, then the compiler will issue a warning or an error message.

Variable size arrays

One-dimensional arrays are most common in programming. If we do not know in advance what size the array we need, then it makes sense to use dynamic arrays. At that moment in time when an array of one size or another is needed to store data, it is necessary to allocate dynamic memory of the required size, and when it becomes unnecessary, then release it. This is very convenient, because when declaring a static array, it is necessary to determine its size in advance during the creation of the program, and such an array will occupy memory throughout the entire operation of the given program.

To create a dynamic array, you need to specify to the `calloc ()` operator the type of array elements and their number. For example

measures, if you need a dynamic array `a` of 100 elements of type

`double`, then you need to do the following:

```
double * a; /* pointer to    dynamic array */  
  
/* block of 100 elements of type double: */ a  
= (double *) calloc (100, sizeof (double));
```

The `calloc ()` operator will return the address of the zero element of the allocated memory block on success, otherwise it will return the value `NULL`. All elements of the allocated memory area are reset to zero.

You can also create a dynamic array using the `malloc ()` operator:

```
double * a;  
  
/* block of 100 elements of double type: */  
  
a = (double *) malloc (100 * sizeof (double));
```

Unlike the `calloc ()` operator, the elements of the allocated memory area are not cleared.

If the dynamic array `a` after working with it is no longer needed, then the memory allocated for it can be freed by the `free ()` operator:

```
free (a);
```

Following is an example of creating a dynamic array.

```
#include <stdio.h> #include <stdlib.h>
```

```
/* displaying array elements on the screen */
```

```

void Print (const int * a, const int n)
{
    int i;

    for (i = 0; i <n; i ++) printf ("% d", a [i]);

    printf ("\ n");
}

int main ()
{
    int * a, / *          address of the zero
    element of the dynamic array * / size, / * size
    of dynamic array a * /

    i; / *          counter * /

    scanf ("% d", & size); / * enter the size of the
    array from the keyboard * / a = (int *) calloc
    (size, sizeof (int)); / * allocate memory for
    the array * / if (a == NULL)

```

```

printf ("not enough memory"); else

{

/ * fill the array elements with arbitrary
numbers */ for (i = 0; i <size; i ++)

a [i] = rand ()% 10; Print (a, size);

free (a); / * free the memory
allocated for the array */

}

return 0;

}

```

If, while working with a dynamic array, you need to change its size, then the `realloc ()` function will help in this case:

```
void * realloc (void * p, size_t size)
```

The `realloc ()` function replaces size with the

size of the object pointed to by p. For the part that is the smallest of the old and new sizes, the content will not change. If the new size is larger than the old one, then the additional space is not initialized, in this case the realloc () function returns a pointer to the new memory location. If there is no memory of the requested size, then the realloc () function returns NULL, and * p does not change.

```
#include <stdio.h> #include <stdlib.h>
```

```
void Print (const int * a, const int n)
```

```
{
```

```
int i;
```

```
for (i = 0; i <n; i ++) printf ("%d", a [i]);
```

```
printf ("\n");
```

```
}
```

```
int main ()
```

```

{

int * a, / *          address of the zero
element of the dynamic array * / size, /
*          size of dynamic array a * /

new_size , / *          new size of array a * /
i, / *          counter * /

* tmp;

scanf ("% d", & size); / * enter the size of the
array from the keyboard * / a = (int *) calloc
(size, sizeof (int)); / * allocate memory for
the array * / if (a == NULL)

printf ("not enough memory"); else

{

for (i = 0; i <size; i ++) a [i] = rand ()% 10;

Print (a, size);

new_size = 2 * size; / * new size of array a *
/ tmp = (int *) realloc (a, new_size * sizeof
(int));

```

```
if (tmp! = NULL)

{

for (i = size; i <new_size; i ++) a [i] = rand
()% 10;

Print (a, new_size);

}

free (a);

}

return 0;

}
```

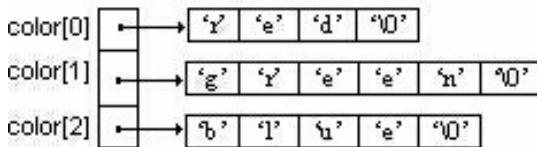
Arrays of Pointers

Arrays can also be formed from pointers. Very often arrays of pointers are used to work with arrays of strings. Each element of such an array is a pointer to some string, or, more

precisely, each element contains the address of the null character of the corresponding string.

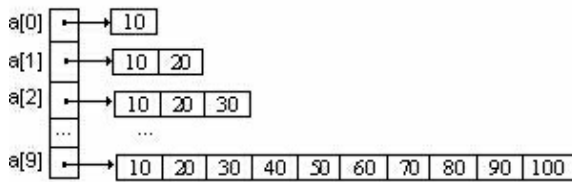
For example, let's declare and initialize an array of pointers color, consisting of the names of three colors (which make up colors in computer graphics):

```
char * color [3] = {"red", "green", "blue"};
```



An array of color pointers can be schematically represented as follows:

Each pointer refers to the null character of the corresponding string. Therefore, the array of pointers color can refer to strings of arbitrary length.



Let's

take another example. Let's create an array of pointers of 10 elements, which will store the addresses of arrays of an integer type, and the arrays will have different lengths, for example:

```
#include <stdio.h> #include <stdlib.h> int
main ()
```

```
{
```

```
int * a [10] = {NULL}; / *initialize all
elements of the array with zeros * /
```

```
int i, j;
```

```
for (i = 0; i <10; i ++)
```

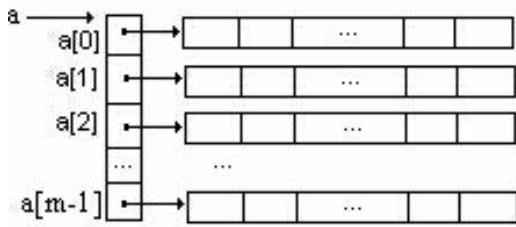
```
{
```

```
a [i] = (int *) calloc ((i + 1), sizeof (int)); for
```

```
(j = 0; j <= i; j ++)  
a [i] [j] = j * 10 + 10;  
  
}  
  
for (i = 0; i <10; i ++)  
  
{  
  
for (j = 0; j <= i; j ++ ) printf ("% d", a [i] [j]);  
printf ("\ n");  
  
}  
  
for (i = 0; i <10; i ++)  
  
{  
  
free (a [i]); a [i] = NULL;  
  
}  
  
return 0;  
  
}
```

Variable size 2D arrays

There are several ways to create a dynamic two-dimensional array. In this section, we will restrict ourselves to considering only one method, the most common and favorable for changing the size of a two-dimensional array. To create such an array, you first need to allocate memory for a one-dimensional array of pointers to one-dimensional arrays, and then allocate memory for one-dimensional arrays.



For

example, suppose you want to create a dynamic two-dimensional array $a[m][n]$ of type double:

In the program below, the `Dispose ()` function loops through the array of pointers and frees the memory occupied by each element, and then frees the memory occupied by the array of pointers itself.

The `Allocate ()` function first allocates memory for an array of pointers, and then loops through this array of pointers and allocates memory for one-dimensional arrays. If it is impossible to allocate memory at some iteration step, then the `Dispose ()` function is called to free the successfully allocated memory for some one-dimensional arrays, and the `Allocate ()` function itself returns `NULL`. Upon successful memory allocation for the entire two-dimensional array, the `Allocate ()` function returns a pointer to a one-dimensional array of pointers.

```
#include <stdio.h> #include <stdlib.h>
```

```
/ * free the memory occupied by the two-  
dimensional array a * / void Dispose (double  
** a, int m)
```

```
{
```

```
int i;
```

```
for (i = 0; i < m; i++) if (a [i] != NULL)
```

```
free (a [i]); free (a);
```

```
}
```

```
/ * memory allocation for a two-dimensional  
array a * / double ** Allocate (int m, int n)
```

```
{
```

```
double ** a; int i;
```

```
int flag; / * boolean variable * /
```

```
a = (double **) calloc (m, sizeof (double *));  
if (a != NULL)
```

```
{  
  
i = 0;  
  
flag = 1;  
  
while (i < m && flag)  
  
{  
  
a[i] = (double *) calloc (n, sizeof (double));  
if (a[i] == NULL)  
  
flag = 0; else i ++;  
  
}  
  
if (! flag)  
  
{  
  
Dispose (a, m); a = NULL;  
  
}  
  
}  
  
return a;
```

```
}
```

```
/* filling the elements of a two-dimensional  
array */ void InitArray (double ** a, int m,  
int n)
```

```
{
```

```
int i, j;
```

```
for (i = 0; i < m; i++) for (j = 0; j < n; j++)
```

```
a[i][j] = rand () % 10;
```

```
}
```

```
/* display the elements of a two-dimensional  
array */ void PrintArray (double ** a, int m,  
int n)
```

```
{
```

```
int i, j;
```

```
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < n; j++) printf ("%1f", a[i][j]);  
    printf ("\n");  
}  
}
```



```
int main ()  
  
{  
  
    double ** a; int m, n;  
  
    scanf ("%d %d", & m, & n); a = Allocate (m,  
n);  
  
    if (a != NULL)  
  
    {  
  
        InitArray (a, m, n); PrintArray (a, m, n);  
        Dispose (a, m);  
  
        a = NULL;  
  
    }  
  
    return 0;  
  
}
```

SYMBOLS AND STRINGS

Representation of symbolic information in a computer

Symbolic information is stored and processed in a computer in the form of a digital code, that is, a certain number-code is assigned to each symbol. Different sets of symbols are used for different types of computers and operating systems. The required character set provided in a particular computer usually includes:

- control characters corresponding to computer functions;
- numbers;

- letters of the alphabet;
- special characters (space, brackets, punctuation marks
- operation signs.

To represent symbolic information in a computer, an alphabet consisting of 256 characters is most often used. Each character of such an alphabet can be encoded with 8 bits (1 byte) of memory. All characters are numbered from 0 to 255, with each number corresponding to an 8-bit binary code from 00000000 to 11111111.

Among the character sets, the most widespread is the ASCII (American Standard Code for Information Interchange) encoding table. In this table, only the first 128 characters are standard (characters with

numbers from 0 to 127). First placed control (non-display) characters (characters with numbers from 0 to 31). Then there are letters of the Latin alphabet, numbers, punctuation marks, brackets and some other symbols, and the Latin letters are arranged in alphabetical order, the numbers are also ordered from 0 to 9. The remaining 128 codes of the ASCII character table are used to accommodate characters of national alphabets, pseudo-graphic characters and scientific symbols.

To represent symbolic variables in the C language, the char type is used. The value of a variable of type char is an 8-bit code corresponding to this or that character.

Symbol handling library

The symbol processing library (<ctype.h>) contains functions that perform checks and operations with symbol data. Each function receives an integer as an argument, which must be either unsigned char or EOF. Functions return nonzero value (true) if the argument c satisfies the condition, and 0 (false) otherwise.

For example, after the instruction

```
printf ("%s%s%s\n", "5", isdigit ('5')? "is":  
"not is", "a digit");
```

the following line will appear on the screen: 5

is a digit.

C strings

In the C language, there is no such separate data type as a string.

But you can define a string in two different ways.

String as an array of characters. First, a string can be defined as an array of characters that ends with the null character '\0'. The null character ('\0') is used to mark the end of a line. In the ASCII character table, this character is numbered 0. When a string is defined as an array of characters, it is given a name and the maximum number of characters that can be contained in it, taking into account the null character. For example `char s[10]`. This declaration of a string as an array

of characters requires a certain amount of memory space for ten elements. The value of string `s` is the address of its first character, through which all its elements are accessed.

For example, the following program declares an array of characters `s`, which is filled in by the user using the `fgets ()` function, and then printed to the screen using the `puts ()` function.

```
#include <stdio.h> int main ()  
  
{  
  
    char s [10]; / * array of characters *  
    /  
    printf ("enter no more than 9  
characters:"); fgets (s, 10, stdin);  
  
    puts (s); / * line output * /      return 0;  
  
}
```

When declared, a string can be initialized immediately, that is, it can be assigned to a character array:

```
char s [] = "moon";
```

This declaration creates an array `s`, consisting of 5 elements: `'m', 'o', 'o', 'n', '\0'`. Individual characters within the array can change, but the variable `s` itself will point to the same memory location.

Declaring a string as an array of characters with subsequent initialization can also be written as follows:

```
char s1 [5] = {'m', 'o', 'o', 'n', '\0'};
```

```
char s2 [7] = {'m', 'o', 'o', 'n', '\0'}.
```

The same can be done in a more convenient form, namely: `char s1 [5] = "moon";`

```
char s1 [7] = "moon";
```

We emphasize that the presence of a null character means that the number of elements

of the character array must be at least one more than the maximum number of characters planned to be allocated in memory.

Note that after declaring a line, for example

```
char s [10];
```

absolutely unacceptable record of the form

```
s = "Hello";
```

that is, the array name cannot be used on the left side of the assignment operator.

String as a pointer to the first character.

You can also define a string in another way - using a pointer to a character. If you declare `char * ps`, this sets the variable `ps`, which can contain the address of the first character in the string. In this case, memory is not reserved for storing characters, as is the case with an array of characters, and the variable `ps` itself is not initialized to a specific value.

If somewhere in the program after the declaration of `char * ps` an instruction of the

form `ps = "sun"` is encountered, then the compiler performs the following actions: characters 's', 'u', 'n' and '\0' are written to a certain memory area (most likely to a read only memory area); variable `ps` is assigned the address of the first character, that is, in our case, the address of the character 's'. After the initialization of the `ps` string, each of its characters can be accessed through the index: `ps[0]`, `ps[1]`, `ps[2]`.

The following is an example of using a pointer to a string:

```
#include <stdio.h> int main ()  
  
{  
  
    char s [17] = "this is a string";  
  
    /* the variable ps is assigned the address of  
    the first character of string s */ char * ps = s;  
  
    int i;  
  
    for (i = 0; i <16; i ++)
```

```
/ * access to the elements of the ps string by  
their indices: * / printf ("% c", ps [i]);
```

```
printf ("\ n");
```

```
ps = ps + eight; /*           the ps pointer is  
assigned the address of the letter 'a' * / printf  
("% s \ n", ps); / * output to screen  
newline * /
```

```
return 0;
```

```
}
```

As a result, two lines will appear on the screen: this is a string

a string

As with an array of characters, a string declared as a pointer to the first character can be initialized immediately, for example

```
char * ps = "sun";
```

In this case, a pointer variable ps is created that points to the string "sun".

Note that a string defined as a pointer to the first character of a string can be placed in an immutable memory area, that is, the values of the elements in this case cannot be changed, although the value of the ps pointer itself can be changed.

We emphasize again that when a string is declared as an array of characters (for example, `char s [10]`), space is reserved in memory, while the declaration of a pointer (`char * s`) requires space only for the pointer itself, which can point to any character or array of characters.

Below are some examples of working with characters and strings.

Example 1. Convert string s to integer. `int`

```
Atoi (const char * s)
```

```
{
```

```
int i, n;
```

```

i = n = 0;

while (s [i]>= '0' && s [i] <= '9')

{

n = n * 10 + (s [i] - '0'); i ++;

}

return n;

}

```

In this case, the expression `s [i] - '0'` gives the numerical value of the character stored in `s [i]`, since the digits '0', '1', ..., '9' are in a row in the character table. That is, '0' is converted to 0, '1' is converted to 1, and so on until the character '9' is converted to 9.

Example 2. Convert uppercase Latin character to lowercase. If the character is not

a capital Latin letter, then it remains unchanged.

```
int ToLower (const int c)
{
    if (c >= 'A' && c <= 'Z')
        return c + ('a' - 'A'); else return c;
}
```

Example 3. The function of removing from string *s* all characters that match the character *c*.

In the `DeleteSymbol ()` function, we will overwrite all the characters included in the string *s*, so that the character *c* does not appear among them.

```
void DeleteSymbol (char * s, const int c)
```

```

{
int i, j;

i = j = 0;

while (s [i]! = '\ 0')

{

if (s [i]! = c)

s [j ++] = s [i]; i ++;

}

s [j] = '\ 0';

}

```

Example 4. Determining the length of the string excluding the end-of-line character '\ 0'.

```

int Strlen (const char * s)

```

```
{  
  
int i = 0;  
  
while (s [i]! = '\ 0') i ++;  
  
return i;  
  
}
```

Example 5. Copying string t to string s.

```
void Strcpy (char * s, const char * t)  
  
{  
  
int i = 0;  
  
while ((s [i] = t [i])! = '\ 0') i ++;  
  
}
```

Example 6. Adding string t to string s.


```

void Strcat (char * s, const char * t)
{
    int i, j;

    i = j = 0;

    while (s [i]! = '\ 0') i ++;
    while ((s [i ++] = t [j ++])! = '\ 0')
;
}

```

String processing functions

Following are the main functions of string operations

(library <string.h>):

Let's give an example of using the strchr () function. After sewing the following lines of the program

```
char * s, c;
```

```
s = "check"; c = 'e';
```

```
printf ("% s% c% s% s% s \ n", "symbol", c,  
strchr (s, c)? "was": "was not", "found in", s);
```

the following result will appear on the screen:
symbol e was found in check.

Let's also consider an example of using the
strstr () function:

```
char * s, * t;
```

```
s = "this is a test !!!"; t = "a test";
```

```
printf ("% s \ n", strstr (s, t));
```

After executing these lines, the following

result will appear on the screen: a test !!!.

The strtok () function is used to split a string into lexemes delimited by separators (spaces, punctuation). The first call to strtok () returns the value of the pointer to the first character of the first token in string s and writes a null character ('\0') to s immediately after the returned token.

```
#include <stdio.h> #include <string.h>
```

```
#define DELIMITERS " .,:; \n \t " / *  
separator characters * / int main ()
```

```
{
```

```
char sentence [500]; / * tokenized string *  
/ char * word; / * address of the  
beginning of the next token in the line *  
/ fgets (sentence, 500, stdin);
```

```
word = strtok (sentence, DELIMITERS);
```

```
while (word! = NULL)

{

puts (word);

word = strtok (NULL, DELIMITERS);

}

return 0;

}
```

On subsequent calls with a zero value of the first argument, the `strtok ()` function continues to split the same string into tokens. The `NULL` argument indicates that when `strtok ()` is called, it should continue splitting from wherever the sentence was written the last time the function was called.

The above functions from the `<string.h>` library can be easily written independently. Below, as an example, there are three variants of the `Strlen ()` function using pointers that return the length of the string `s`.

We also present variants of the Strcpy () function for copying string t to string s.

String conversion functions

Following are the string conversion functions (library

<stdlib.h>).

For example, below are two programs that perform the same action using two equivalent instructions

```
x = atof (s); and x = strtod (s,
NULL);/ * example atof () function * /
#include <stdio.h> #include <stdlib.h>
```

```
int main ()
```

```
{
```

```
char * s = "1.5";/ * example strtod () function
```

```

* / #include <stdio.h> #include <stdlib.h>

int main ()

{

char * s = "1.5";

double x; x = atof (s);

printf ("x =% f \ n", x); return 0;

}double x;

= strtod (s, NULL); printf ("x =% f \ n", x);
return 0;

}

```

If you need to perform the reverse conversion of a number to a string, then the `sprintf ()` function will help with this. For example:

```
int a = 123; char s [10];
```

```
sprintf (s, "% d", a); puts (s);
```

Examples of working with strings

Example 1. An integer n is given, $1 \leq n \leq 26$.
Write to string

s first n uppercase Latin letters.

```
#include <stdio.h> #define LENGTH 27 int  
main ()  
{  
    char s [LENGTH]; int i;  
    unsigned int n; printf ("n =");  
    scanf ("% u", & n); for (i = 0; i <n; i ++)  
        s [i] = 'A' + i;
```

```
s [n] = '\ 0'; puts (s); return 0;
```

```
}
```

Example 2. You are given a string representing a positive integer. Find the sum of the digits of this number.

```
#include <ctype.h> int Sum (const char * s)
```

```
{
```

```
int i, sum;
```

```
i = sum = 0;
```

```
while (isdigit (s [i]))
```

```
{
```

```
sum += (s [i] - '0'); i ++;
```

```
}
```

```
return sum;
```

```
}
```


Example 3. Let there be an integer n in the decimal number system and some integer number $2 \leq p \leq 10$. You want to write a function that writes the representation of the number n in the p -th number system to the string s .

```
void Itoa (int a, int p, char * s)

{

    int i, j, buf; i = 0;

    /* generate numbers in reverse order */ do

    {

        s [i] = '0' + a% p; a /= p;

        i ++

    } while (a != 0);

    s [i] = '\0';

    /* flip the line */ j = i - 1;
```

```
i = 0;

while (i < j)

{

    buf = s [i]; s [i] = s [j]; s [j] = buf; i ++;

    j--;

}

}
```

Example 4. Given string s. Count the number of vowels and consonants of Latin letters contained in it.

```
#include <stdio.h>    #include <string.h>
#include <ctype.h>

#define N 500 / * size strings s * /
```

```
#define VOWEL "AEIOUYaeiouy" / * Latin  
vowels * /
```

```
void Count (char * s, int * ng, int * ns)
```

```
{
```

```
int i;
```

```
i = * ng = * ns = 0; while (s [i])
```

```
{
```

```
if (isalpha (s [i]))
```

```
if (strchr (VOWEL, s [i])) (* ng) ++;
```

```
else (* ns) ++; i ++;
```

```
}
```

```
}
```

```

int main ()

{

char s [N];

int ng, / *          number of vowels
letters * /          ns; / * number of consonants
* /

fgets (s, N, stdin); Count (s, & ng, & ns);

printf ("ng =% d ns =% d \ n", ng, ns); getch
();

return 0;

}

```

Comment. The Count () function uses the strchr () function from the <string.h> library, which checks if the character s [i] is contained in the string VOWEL, that is, the character s [i] is compared with all characters in the given string. And so with each

character of the string `s`. This is not entirely efficient.

Instead of comparing the character `s[i]` with all characters of the string `VOWEL`, we can organize the check for the belonging of the character `s[i]` to a set of vowels as follows.

Let `int flag[256]` be a logical array, each index of which corresponds to one or another character from the ASCII character table. All the elements of the flag array, the indices of which correspond to the vowel letters, are defined by ones, and all other elements - 0:

```
int flag[256] = {0};
```

```
for (i = 0; i < strlen(VOWEL); i++) flag  
[VOWEL[i]] = 1;
```

Now, to check whether a certain character with a vowel is a Latin letter, it is enough to write

```
if (flag [c])
```

Thus, the Count () function can be improved as follows:

```
void Count (char * s, int * ng, int * ns)

{

int i, size;

int flag [256] = {0}; size = strlen (VOWEL);
for (i = 0; i <size; i ++)

flag [VOWEL [i]] = 1;

i = * ng = * ns = 0; while (s [i])

{

if (isalpha (s [i]))

if (flag [s [i]]) (* ng) ++; else (* ns) ++;

i ++;
```

```
}
```

```
}
```

Example 5. Some string *s* is given. Select all non-negative integers in this line and find their sum.

To solve this problem, for a change, we will use the dynamic string `digit`, where we will write the next selected number from the string *s*.

```
#include <stdio.h> #include <stdlib.h>
#include <string.h> #define N 300
```

```
/* function returns the sum of integers from
```

```

string s * / int Sum (char * s)
{
    int i, j, sum;

    char * digit = NULL; sum = i = 0;

    while (s [i])
    {

        / * skip all non-digit characters: * / while (s
        [i] &&! isdigit (s [i]))

        i ++;

        j = i; / *          remember the position of
        the beginning of the next number * /

        / * run through all consecutive digits included
        in the next number:

        * /

        while (s [i] && isdigit (s [i])) i ++;
    }
}

```



```
 / * allocate memory for the dynamic string
digit: * / digit = (char *) malloc ((i - j + 1) *
sizeof (char));
```

```
strncpy (digit, & s [j], i - j); / * copy digits to
string digit * / digit [i - j] = '\ 0';
```

```
sum + = atoi (digit); / *add the next number
to the current amount* / free (digit); /
*      free memory * /
```

```
digit = NULL;
```

```
}
```

```
return sum;
```

```
}
```

```
int main ()
```

```
{
```

```
char s [N]; fgets (s, N, stdin);
```

```
printf ("sum =% d \ n", Sum (s)); return 0;  
}
```

Breaking a string into tokens

Consider the problem of highlighting all words in a sentence line. Let us denote by sentence the initial line-sentence in which you want to select all the words, and by word - the next selected word in the line sentence.

It is possible that the variable sentence is an array of characters, for example

```
char sentence [200];
```

It is also possible that the variable sentence is a pointer to a string:

```
char * sentence;
```

The same can be said for the word variable. These cases come from one or another task of working with strings.

Case 1. Consider this case: suppose the variable sentence is an array of characters and some of its elements, for example, separator characters, are allowed to be replaced with the character '\0', and the variable word is a pointer to a string.

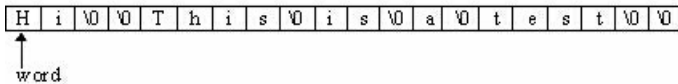
Let's compose a program that first replaces all word separators (spaces, punctuation marks) with the null character '\0', and the word pointer will each time contain the address of the first character of the next word in the sentence.

For example, suppose the character array sentence contains a string like this:

H	i	\0	\0	T	h	i	s	\0	i	s	\0	a	\0	t	e	s	t	\0	\0
---	---	----	----	---	---	---	---	----	---	---	----	---	----	---	---	---	---	----	----

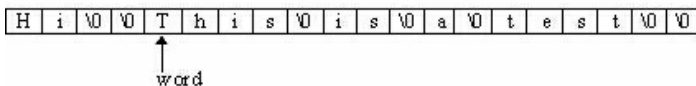
After replacing all the separator characters with the character '\0', we will have the string sentence in the following form:

Next, assign the address of the letter 'H' to the word variable:



After this assignment, the variable word will point to the word

“Hi”. Then we assign the address of the letter 'T' to the variable word:



Now the variable word points to the word "This" and so on.

Below is the given algorithm for extracting words from a string.

sentence.

```

#include <stdio.h> #include <string.h>

#define DELIMITERS " .,:; \ n \ t " / *
separator characters * / #define N 300

int main ()

{

char sentence [N]; / *                source string
* /

char * word; / * address of the beginning of
the next word in the sentence * / int i, j, n;

fgets (sentence, N, stdin); / * enter a string
from the keyboard * / n = strlen (sentence); /
* the length of the sentence * /

/ * in the loop, replace all separator characters
with the '\ 0' character * / for (i = 0; i < n; i
++)

{

```

```

if (strchr (DELIMITERS, sentence [i]))
sentence [i] = '\0';

}

i = 0;

while (i <n)

{

/ * skip all separators * / while (i <n &&
sentence [i] == '\0')

i ++;

word = & sentence [i]; / *          new start
position the words          * / puts
(word); / *          display the next word * /

/ * skip characters just printed the
words          * /
while (i <n && sentence [i] != '\0')

i ++;

```

```
}
```

```
return 0;
```

```
}
```

Comment. In the previous algorithm, for each character in the string sentence, the function `strchr ()` was called to determine whether the current character in the string was a separator character. This means that if the line sentence contains separator characters

Is "relatively" small, the complexity of the algorithm for checking whether each character of the string sentence belongs to separator characters will be something like this:

`strlen (sentence) * strlen (DELIMITERS).`

This is not very good, especially if we are dealing with a long enough line of sentence. The situation can be changed very easily. Let's introduce a logical array `int flag [256]`,

in which the ordinal numbers (indices) of elements correspond to the codes of characters from the ASCII character table. We initialize the elements of this array as follows: if the character with code i is a separator character, then we set $\text{flag}[i] = 1$, otherwise $\text{flag}[i] = 0$. And so for all characters from the ASCII character table ($i = 0, 1, \dots, 255$):

```
int flag [256] = {0};
```

```
for (i = 0; i <strlen (DELIMITERS); i ++)  
flag [DELIMITERS [i]] = 1;
```

After this loop, the flag array will look like this:

...	1	1	...	1	...	1	0	1	...	1	1	...	0	0	...	0	...
...	9	10	...	32	...	44	45	46	...	58	59	...	65	66	...	90	...
	'\t'	'\n'		'"		','	'.'	'.'		','	','		'A'	'B'		'Z'	

Now, checking if the i -th character of the string sentence is a separator character looks

like this:

```
if (flag [sentence [i]])
```

Therefore, replacing all separator characters with a null character would look like this:

```
for (i = 0; i <n; i ++)
```

```
if (flag [sentence [i]]) sentence [i] = '\0';
```

Case 2. Let's consider one more case. Let there be some line-sentence sentence, and the variable sentence is either a pointer to a line (that is, in general, the elements of this line cannot be changed, since the line can be in the read-only memory area), or an array of symbols, the values of the elements of which, according to the problem statement, cannot be changed. Also let the variable word be a dynamic array

characters, and all the words from the line

sentence must be copied one by one to the variable word.

In order for the algorithm to work very quickly, in this case, we take into account the remark indicated after case 1.

```
#include <stdio.h> #include <string.h>

#define DELIMITERS " .,:; \ n \ t " / *
separator characters * /           #define N
300

int main ()

{

char sentence [N]; / *           source
string * /

char * word; / * next word in the sentence *
/           int flag [256] = {0}, len, i, j;
```

```
len = strlen (DELIMITERS);
```

```
/* if the character with code i is a separator  
character, then we set flag [i] = 1:
```

```
*/
```

```
for (i = 0; i <len; i ++) flag [DELIMITERS  
[i]] = 1;
```

```
fgets (sentence, N, stdin); /* enter a string  
from the keyboard */ i = 0;
```

```
while (sentence [i])
```

```
{
```

```
/* skip all separators */ while (sentence [i]  
&& flag [sentence [i]])
```

```
i ++;
```

```
j = i; /* position start of a new word */
```

```
/* determine the position of the end of the  
next word in the line */
```

```
while (sentence [i] &&! flag [sentence [i]]) i  
++;
```

```
/* allocate memory for the next word: */  
word = (char *) calloc (i - j + 1, sizeof  
(char));
```

```
/* copy the characters of the next word into  
the word variable */ strncpy (word, &  
sentence [j], i - j);
```

```
word [i - j] = '\0';
```

```
puts (word);

free (word); / *           free dynamic
memory * / word = NULL;

}

return 0;

}
```

Tasks

1. Display all numbers, lowercase and uppercase Latin letters with their codes.
2. Given an even number $n > 0$ and symbols $c1$ and $c2$. Print a string of length n , which consists of alternating characters $c1$ and $c2$.
3. Some line is given. Print a string containing the same characters, but in reverse

order.

4. You are given a string s of size n . Write in string t of size $2n$ characters of string s , between which one space is inserted.

5. Given string s . Write a logical function that determines whether string s contains two adjacent identical characters.

6. Given string s . Count the frequency of occurrences of each character in string s .

7. A non-negative integer is given. Convert it to a string variable without using library functions.

8. You are given a string representing the binary notation of a positive integer. Print a string representing the decimal representation of this number.

9. Given strings s and s_0 . Remove

from string `s` the first substring that matches `s0`.

10. Given strings `s` and `s0`. Find the number of occurrences of string `s0` in string `s`.

11. Replace each series of consecutive spaces in a line with one space.

12. A sentence string is given. Find the number of words in a line.

13. Write the function `void WordN(char * s, char * t, int k)`, which

`k`The `k`-th word of string `s` writes to string `t`.

14. A sentence string is given. Print a line containing the same words, separated by one space and in reverse order.

15. A sentence string is given. Display all words of the given string in alphabetical order.

16. Some line is given. Select all non-negative integers in this line and display them on the screen.

Self-study tasks

17. An integer n is given, $1 \leq n \leq 26$. Print the last n lowercase (that is, small) letters of the Latin alphabet in reverse order, starting with the letter 'z'.

18. Determine how many different letters there are in the word s .

19. A non-negative integer is given. Write the binary representation of the given number to a string variable.

20. You are given a string representing the decimal notation of a positive integer.

Print a string representing the binary representation of the same number.

21. Replace each series of consecutive delimiters between words with one space in the string.

22. Given strings s and $s0$. Remove from string s the last substring that matches $s0$. If there are no matching substrings, then print the string s unchanged.

23. A sentence string is given. Find the number of words that contain at least one letter 'A'.

24. A sentence string is given. Find the number of symmetrical words.

25. A sentence string is given. Print a string containing the same words, separated by one space, and arranged in alphabetical order.

26. Write the function `void FillStr(char * s, char * t, int n)`, which fills string `t` with `n` repeated copies of the template string `s`.

27. Remove character groups from string `s` between parentheses '(' and ')'. The parentheses themselves should also be excluded. It is assumed that there are no other parentheses within each pair of parentheses.

STRUCTURES

Understanding Structures

Structure - the union of one or more objects, which can be variables, arrays, pointers, etc. Some objects that are used in a particular program, one way or another, can be logically related to each other, for example: the abscissa (double x) and the ordinate (double y) can be the coordinates of some point on the plane, or the surname (char name [20]), year of birth (int year) and group (char group [30]) can contain information about a particular student, etc. Therefore, for the convenience of working with such objects, they can be grouped under one name. If the array is a set of single-type data, then the structure is a more universal data type, since it can contain objects of different types.

A structure is declared using the `struct` keyword, followed by the name of the structure, followed by an enumeration of the list of objects enclosed in curly braces. For example, the description of a structure representing a point on a plane looks like this:

```
struct DOT  
  
{  
  
    double x; / * abscissa * /          double y; /  
    *          ordinate * /  
  
};
```

Variables declared within a structure are called its members. Elements of the same structure must have unique names, although the names of elements of different structures may contain the same name. Each structure declaration must end with a semicolon.

When a structure is declared within a

program, memory is not reserved for it; it simply describes a new data type. Having declared a structure, you can create variables of this type:

```
struct DOT A, B; / * variables A and B of  
type DOT * / struct STUDENT stud; / *  
variable stud of type STUDENT * /
```

Thus, we have declared two structure variables A and B, which can contain the coordinates of points on the plane, and one structure variable stud. Individual structure elements are accessed through the dot operator (.): Ax, Ay, stud.name, stud.year, stud.group. The following program enters the coordinates of a point on the plane A, and then displays the distance from that point to the origin.

```
#include <stdio.h> #include <math.h> struct  
DOT  
{
```

```

double x; double y;

};

int main ()

{

struct DOT A; scanf ("% lf", & A.x);

scanf ("% lf", & A.y);

printf ("d =% .2f \ n", sqrt (Ax * Ax + Ay *
Ay)); return 0;

}

```

An important point when describing a structure is where it is declared. If the declaration occurs outside of all functions (such a declaration is called external), then this declaration can be used by all functions following it. If the structure is declared inside some function, then this declaration can only be used by that function.

Like arrays, structured variables can be

initialized when they are defined, for example:

```
struct DOT A = {1.2, -2.5};
```

```
struct STUDENT stud = {"Ivanov", 1990,  
"KB-11"};
```

If a structure is passed as an argument to a function, then, unlike arrays, the transfer is by value. To transfer a structure by reference, you must pass its address.

Besides the fact that functions can be passed structures as arguments, functions can also return structures, for example:

```
#include <stdio.h> struct DOT  
  
{  
  
double x; double y;  
  
};
```

```

struct DOT Init (double x, double y)
{
    struct DOT A;

    Ax = x;

    Ay = y; return A;
}

int main ()
{
    struct DOT B = Init (2.4, 1.5); return 0;
}

```

If two structured variables are of the same type, then one variable can be assigned to the other, for example:

```

struct DOT A = {1.2, -2.5}, B; B = A;

```

In this case, the values of the elements of one structure will be the values of the

corresponding elements of the other structure, even if there are arrays among the elements of the structure.

You can also create arrays of structures. For example, you can define a sequence of 10 elements of plane points:

```
struct DOT a [10];
```

Then the reference to the coordinates of the points will be as follows:

```
a [0] .x, a [0] .y, ..., a [9] .x, a [9] .y
```

Associations

A union is a variable that at different points in time can store objects of different types and sizes. Only one of the union elements can be used at any given time. The main feature of the union is the fact that for each of the declared elements the same memory area is allocated, that is, they overlap. Therefore, if at a given moment of time some element of the union is used, then the values of the remaining elements become undefined.

Unions save space instead of wasting memory on variables that are not currently being used.

The keyword is used to describe the union

union, for example:

```
union NUMBER
```

```
{
```

```
long l; double d;
```

```
};
```

Here l is 4 bytes and d is 8 bytes. Therefore, a variable of type NUMBER will be allocated 8 bytes, that is, enough to store the largest of these two variables.

When declared, a union can be initialized only with a value of the type of its first element. Thus, in our case, the union can be initialized only with a value of the long type,

for example, you can write

```
union NUMBER value = {10}; / * that is,  
value.l == 10 * /
```

and it would be wrong to write like

```
union NUMBER value = {10.0};
```

Below is a small example using union:

```
#include <stdio.h> union NUMBER  
  
{  
  
long l; double d;  
  
};  
  
int main ()  
  
{  
  
union NUMBER value = {10}; printf ("l =%  
d \n", value.l); value.d = 5.0;
```

```
printf ("d =% f \ n", value.d); return 0;

}
```

Unions can be found in structures and arrays. The opposite is also possible. For example, the following are two equivalent STR declarations:union NUMBER

```
{

long l;struct STR

{

char s [10];

double d;

};

struct STR

{
```

```

char s [10];

union NUMBER value;

};union NUMBER

{

long l; double d;

} value;

};

```

After one published
announcement
STR structure goes like this:

```

int main ()

{

struct STR st = {"abc", 10}; / * st.s == “abc”,
st.value.l == 10 * / printf ("s =% s \ n", st.s);

```

```
printf ("l =% d \ n", st.value.l); st.value.d =  
1.0;
```

```
printf ("d =% f \ n", st.value.d); return 0;
```

```
}
```

DIRECTIVES PREPROCESSOR

A preprocessor is a program that performs some manipulations with the source code of the program before it undergoes translation. The preprocessor starts automatically when the program is compiled. All preprocessor directives start with a # sign. The preprocessor directives are not followed by a semicolon.

Directive #include

The #include directive includes the contents of the specified file into the program text. This directive has two forms of writing:

`#include "filename" #include <filename>`

If the file name is specified in quotes, then the preprocessor searches for the file in accordance with the specified route, and if it

is absent, in the current directory. If the file name is specified in angle brackets, which are used for standard library files, then the file is searched for in the standard directories of the operating system specified by the PATH command.

The `#include` directive is often used to include the following standard library headers: `stdio.h` (standard I / O library), `stdlib.h` (general purpose functions), `string.h` (library of functions that operate on strings), etc. etc.

#Define directive

This directive serves to replace constants, operators or expressions with some identifiers. Identifiers that replace string or numeric constants are called named constants. Identifiers that replace program fragments are called macro definitions, and

macro definitions can have arguments.

The `#define` directive has two forms of notation: `#define identifier text`

`#define identifier (parameter list) text`

This directive replaces all subsequent occurrences of the identifier with text. This process is called macro substitution. The text can be any fragment of a C program, or it can be completely absent. For example, upon encountering `#define E 2.711828` (named constant), the preprocessor will replace all occurrences of the `E` constant with the numeric constant `2.711828`.

If there is no text, then all instances of the identifier are removed from the program.

A macro definition without parameters is treated in the same way as a named constant. For example, you can write

`#define FOREVER for (;;)`

and then write infinite loops in the program as FOREVER.

The second form of the `#define` directive contains a list of parameters, which can contain several identifiers, separated by commas. Parameters in the text of a macro define the positions at which the arguments of the macro call should be substituted. A macro call must be a separate token and consist of an identifier and a list of arguments enclosed in parentheses, the number of which must match the number of parameters.

As the first example, let's give a macro definition with one parameter for finding the square of a number:

```
#include <stdio.h>
```

```
#define SQR (x) ((x) * (x))      int main ()  
{
```

```
printf ("% d \ n", SQR (5));
```

```
printf ("% f \ n", SQR (SQR (0.12))); return  
0;
```

```
}
```

Let us explain the meaning of parentheses in the macro definition $((x) * (x))$.

Surely one could write

```
#define SQR (x) x * x
```

But then only numbers, not expressions, will be squared correctly. For example, if the program contains the instruction

```
r = SQR (a + b);
```

then, after the preprocessor is running, this code will expand into the following code:

```
r = a + b * a + b;
```

Therefore, it will be more correct to write

```
#define SQR (x) ((x) * (x))
```

As a second example, we give a macro definition with two parameters to find the maximum value from these parameters:

```
#include <stdio.h>
```

```
#define MAX (x, y) ((x) > (y)? (x): (y))  
int  
main ()
```

```
{  
  
int x = 1, y = 2;  
  
int max = MAX (x, y); return 0;  
  
}
```

Directive #undef

The #undef directive is used to cancel the effect of the #define directive. The syntax for this directive is as follows:

```
#undef identifier
```

The #undef directive overrides the current #defined for the specified identifier. The scope of a named constant or macro extends from where it is defined to where it was canceled using #undef, or to the end of the file if it was not undone.

You can also cancel nonexistent constants and macrodefinitions. It is also allowed, after canceling some definition, to redefine it using `#define`.

FUNCTIONS

Function Basics

Function is a subroutine that organizes one or another algorithmic process and returns some value. A C program has at least one main () function that is automatically called when the program starts. The main () function itself can call other functions, and those, in turn, themselves can call certain functions.

The function consists of declarations and operators and is designed to solve a specific problem. Each function must have a name that is used to declare, define and call it.

When a function name is encountered in a program, control is transferred to the body of this function, that is, the function is called. When a function is called, some values (actual parameters) are passed to it using arguments (formal parameters), which are used by the function during its operation. Any

function argument can be a constant, variable, or expression.

The general format of a function definition is as follows:

```
return_type function_name (parameter_list)

{

announcement operators

}
```

The return type declares the type of the value returned by the function. If the function does not return a value, then the empty type is used as the return type - void. If the function receives no arguments, the parameter list is also declared void.

Functions can return values. After accessing the function, it performs some actions and, as

a result,

that of its work may return some value. The return statement returns the result from the called function to the caller. Any expression can follow the word return:

return expression;

The expression, if necessary, is cast to the return type of the function.

There are three ways to return control to the point in the program from which the function was called. The first two methods relate to the case when the function does not return a result. In the first case, control is transferred when the right brace is reached, which ends the function. In the second case, control is transferred when the operator is executed

return;

In the third case, when the function returns a result, the operator

return expression;

returns the value of the expression to the calling function.

There can be several return statements in a function. Then these operators will fix the corresponding exit points.

Function prototypes

The function prototype tells the compiler the type of the function's return value, the number of parameters the function receives, and the parameter types and ordering. The compiler checks the correspondence between the types of the passed actual parameters and the types of the formal parameters, that is, the compiler uses the function prototype to check the correctness of calls to this function. The function prototype has the same form as the function definition, with the only difference that the function body is absent, and the names of formal parameters can also be

omitted. If the function prototype is not set, but a function call is encountered, then an implicit prototype is built from the analysis of the function call form.

For example, in the program below, which asks for two numbers a and b and then displays the midpoint of the segment [a, b], the Middle () function is used. The prototype of this function looks like this:

```
double Middle (double, double);
```

This prototype indicates that the Middle () function has two double arguments and returns a double result.

```
#include <stdio.h>
```

```
double    Middle    (double,    double);    /  
*                function prototype * /
```

```
int main ()
```

```
{
```

```
double a, b; scanf ("% lf% lf", & a, & b);
```

```
printf ("middle =% f \ n", Middle (a, b));  
return 0;  
  
}
```

```
/ * description of the Middle () function * /  
double Middle (double a, double b)  
  
{  
  
return (a + b) / 2;  
  
}
```

An important feature of prototypes is the typecasting of arguments. For example, if the `Middle ()` function is passed two integer arguments 1 and 2, then these arguments will first be converted to double types (1.0 and 2.0), before calculating the value of the expression $(a + b) / 2$. Therefore, the result will be correct: 1.5. If these arguments were not cast to the double type, the function

would return the value 1.0.

Memory classes

Each variable, besides type, name and value, has one more characteristic - a memory class. A memory class characterizes the lifetime and storage location of an object during program execution. A variable can be auto, register, extern, and static, according to the memory class.

By default, if memory class is omitted, the variable is assumed to be automatic.

The scope of automatic, register and static objects is the block where they are defined, including nested blocks in this block, which do not contain overridden variables with the same names.

The lifetime of an object with memory classes auto and register coincides with the execution time of the block. Such objects are created at the moment of entering the block

and destroyed at the exit from it. In this case, variables with a register memory class, if possible, are stored in machine registers. This is especially important if the variable is used very often, for example, counter variables, sums, etc. Note that register variables can only be of integer type. The compiler can ignore the indication that it is desirable to place the variable in the machine register. In this case, such a variable is equivalent to an automatic variable. The description of the register variables looks like this:

```
register int i; register char c;
```

The register declaration can only be applied to automatic variables and formal parameters of a function. For example:

```
long Sum (register int n)
{
    register long sum; register int i;
```



```
for (i = 0; i <n; i ++)  
  
sum + = i; return sum;  
  
}
```

External variables are defined outside of functions, so they are available to almost any of them. Such variables exist throughout the execution of the entire program. The scope of an external variable extends from the point in the input file where it is declared to the end of the file. For example, after executing the following program:

```
#include <stdio.h>  
  
int i = 1;/* external variable */  
  
void Print ()  
  
{  
  
printf ("%d \n", i);
```

```
}  
  
int main ()  
  
{  
  
Print (); i ++;  
Print (); return 0;  
}
```

the following result will appear on the screen:

1

2

However, if this program is written in this form

```
#include <stdio.h> void Print ()  
  
{
```

```

printf ("%d \n", i); /* error */

}

int i = 1; /* external variable */

int main ()

{

Print (); i ++;

Print (); return 0;

}

```

then this will lead to an error, since the Print () function is outside the scope of the external variable i. If you need access to an external variable before it is declared, or if it is defined in another source file, use the extern description:

```

#include <stdio.h> void Print ()

{

extern int i; printf ("% d \ n", i);

}

int i = 1; / * external variable * /          int
main ()

{

Print (); i ++;

Print (); return 0;

}

```

The line `extern int i` declares, for the remainder of the file, that variable `i` is of type `int`, and no memory is allocated for it.

An external variable can be initialized only when it is defined. If an external variable is defined but not initialized, then, by default, it

is initialized to zero.

Note that the functions themselves are external objects, since in the C language it is forbidden to declare one function inside another function.

Static variables (static) can be local and global. Local static variables are visible only to the function where they are defined. Only, unlike automatic variables, static local variables retain their value even after exiting the function, and do not arise and are destroyed each time it is called. The next time this function is called, the static local variable will have the value it had when it was last exited. For example, after executing the following program, the numbers 10, 20,..., 100 will be displayed on the screen:

```
#include <stdio.h> void Print ()
```

```
{
```

```
static int i = 10; printf ("%d \n", i); i += 10;
```

```
}  
  
int main ()  
  
{  
  
    int j;  
  
    for (j = 1; j <= 10; j ++ ) Print ();  
  
    return 0;  
  
}
```

Global static variables are available to all functions of this file, located below their definition. But in other files, global static variables are unknown.

The previous example can be rewritten as follows:

```
#include <stdio.h> static int i = 10; void Print
```

```
()  
  
{  
    printf ("%d \n", i); i += 10;  
}  
  
int main ()  
  
{  
  
    int j = 1;  
  
    for (j = 1; j <= 10; j++) Print ();  
  
    return 0;  
  
}
```

Function pointers

Functions, like any data items, have addresses. As the name of the array is the address of the first element of the array, so the name of the function is the address of the beginning of the machine code of this function. Therefore, you can define a pointer to a function and work with it like a regular variable. Pointers to functions can be passed to other functions as arguments, they can be returned as the results of functions, they can be placed in arrays, and so on.

To get the address of a function, it is enough to write down its name (which contains its address) without trailing braces, that is, if `F ()` is some function, then `F` is the address of this function. For example:

```
#include <stdio.h>
```

```
int Compare (int a, int b)
```

```
{
```

```
return a >= b;
```



```
}
```

```
int main ()
```

```
{
```

```
printf ("%p \n", Compare); printf ("%p \n",  
main); getch ();
```

```
return 0;
```

```
}
```

Therefore, to pass the address of a particular function to a function, it is enough to pass the name of the passed function.

The following example illustrates the use of a function pointer.

```
#include <stdio.h>
```

```
int ShowMsg (char * msg)
```

```
{
```

```

return puts (msg);

}

int main ()

{

int (* pFunc) (char *); /* function pointer */
pFunc = ShowMsg;

(* pFunc) ("A message \n"); /*we call the
function through the pointer */ return 0;

}

```

Let's give another example. Let's write the Count () function that calculates the number of upper or lower Latin letters in string s. Let the functions int Big (int c) and int Small (int c), respectively, determine whether a character c is a capital or a small Latin letter. If the address of the Big () function is passed as a parameter to the Count () function, then

it will calculate the number of large Latin letters in the string `s`. If this function is passed the address of the `Small ()` function, then it will count the number of small Latin letters in string `s`.

In the header, the `Count ()` function has the following parameter: `int (* flag) (int)`

This means that the `Count ()` function can receive the address of absolutely any function as an argument, with the condition that it returns an integer type result, and takes the `int` type as a parameter. Note that parentheses are required around `* flag` to enforce appropriate operator precedence. The parentheses take precedence over the asterisk (`*`) operator. Therefore, if you write

```
int * flag (int)
```

then this declaration will mean a function that takes an integer as a parameter and returns a pointer to some integer.

The function that is passed to the `Count ()`

function as a parameter is called in the if condition:

```
if ((* flag) (s [i]))
```

In this case, dereferencing a function is necessary to call this very function.

```
#include <stdio.h> #define N 200
```

```
int Big (int); / * function prototype Big () * /
```

```
int Small (int); / * the prototype of the Small  
() function * / int Count (char * s, int (*)  
(int)); / * the prototype of the Count ()  
function * /
```

```
int Big (int c)
```

```
{
```

```
return (c> = 'A' && c <= 'Z');  
  
}
```

```
int Small (int c)  
  
{  
  
return (c> = 'a' && c <= 'z');  
  
}
```

```
int Count (char * s, int (* flag) (int))  
  
{  
  
int i, n;  
  
i = n = 0;  
  
while (s [i]! = '\ 0')  
  
{
```

```

    if ((* flag) (s [i])) n ++;

    i ++;

}

return n;

}

int main ()

{

    char s [N]; fgets (s, N, stdin);

    printf ("% d \ n", Count (s, Big)); printf ("% d
    \ n", Count (s, Small)); return 0;

}

```

Let's also give an example of using an array of pointers to functions. Let's define three functions: Sum, Subtraction, Multiplication, each of which has two integer parameters and returns some integer - the result of a binary

operation with given numbers (sum, difference, product). The addresses of these functions will be stored in an array f of size 3, which is initialized as follows:

```
int (* f [3]) (int, int) = {Sum, Subtraction, Multiplication};
```

After the user enters two integers (a and b) and the number of the arithmetic operation on these numbers (0 is the sum, 1 is the difference, 2 is the product), the program below will display the corresponding result (a + b , ab or a * b).

```
#include <stdio.h> int Sum (int, int);
```

```
int Subtraction (int, int); int Multiplication (int, int);
```

```
int Sum (int a, int b)
```

```
{
```

```
return a + b;
```

```
}
```

```
int Subtraction (int a, int b)
```

```
{
```

```
return a - b;
```

```
}
```

```
int Multiplication (int a, int b)
```

```
{
```

```
return a * b;
```

```
}
```

```
int main ()
```



```

{
int a, b, i;

int (* f [3]) (int, int) = {Sum, Subtraction,
Multiplication}; scanf ("% d% d", & a, & b);

scanf ("% d", & i);

if (i> = 0 && i <= 2) printf ("% d \ n", (* f
[i]) (a, b));

return 0;

}

```

Recursion

Recursion is a way of organizing an algorithmic process in which a function, in the process of executing its constituent

operators, refers to itself directly or through other functions.

Recursion can be direct or indirect. If a function calls itself, then it is a direct recursion. If the function calls another function, which then calls the original function, then indirect recursion takes place.

Any recursive function can be made non-recursive (iterative). Recursive algorithms are most useful when the data used is defined recursively. A recursive function usually looks shorter and clearer, but requires more CPU time and more memory.

Note that when a function calls itself, in this case a new copy of this function is executed, and therefore the local variables of the second version of the original function will be independent of the local variables of the original function.

A recursive function is called from the outside once. The conditions for complete termination of the work of a recursive function must be in it itself.

Examples using recursion

Example 1. The following program displays integers 10, 9, ..., 1 on the screen using the recursive function `Print ()`. This function takes as an argument the number to be displayed. After the number is printed, this function calls itself with an argument one less than the one just printed. The `Print ()` function ends when it receives a number less than one as a parameter.

```
#include <stdio.h> void Print (int);
```

```
void Print (int i)
```

```
{
```

```
if (i>= 1)
```

```
{
```

```

printf ("% d \ n", i); Print (i - 1);

}

}

int main ()

{

Print (10); return 0;

}

```

Example 2. As another simple example, consider a program that calculates the sum of all integers in the range from 1 to n (forget about the sum of an arithmetic progression for the moment), where the value of n is entered by the user from the keyboard. Let $\text{Sum}(n) =$

$= 1 + 2 + \dots + n$. Then the following equalities will take place: $\text{Sum}(n) = \text{Sum}(n - 1) + n$ for $n > 0$;

$\text{Sum}(0) = 0$.

Therefore, our program will look like this:

```
#include <stdio.h> int Sum (int);
```

```
int Sum (int i)
```

```
{
```

```
if (i>= 1)
```

```
return i + Sum (i - 1); else return 0;
```

```
}
```

```
int main ()
```

```
{
```

```
int n; printf ("n =");
```

```
scanf ("%d", &n);
```

```
printf ("sum =%d \n", Sum (n)); return 0;
```

}

Example 3. It is required to compose a recursive function

int Count (char * s, int n) of integer type, which calculates the number of lowercase Latin letters in string s.

Let us introduce the characteristic function F on the set of all characters from the ASCII character table: $F(c) = 1$ if the character c is a small Latin letter, and $F(c) = 0$ otherwise.

Let the length of the string s be n. Then the number (count) of small Latin letters in this line will be calculated by the following formula:

$$\text{count} = F(s[0]) + F(s[1]) + \dots + F(s[n-1]).$$

The recursive function int Count (char * s, int n) obeys the following recurrence relations:

$$\text{Count}(s, 0) = 0,$$

$\text{Count}(s, n) = \text{Count}(s, n - 1) + F(s[n-1])$
for $n > 0$.

```
#include <stdio.h>    #include <string.h>
#define N 300
```

```
int F(char);
```

```
int Count(char *, int);
```

```
int F(char c)
```

```
{
```

```
if (c >= 'a' && c <= 'z') return 1;
```

```
else return 0;
```

```
}
```

```
int Count(char * s, int n)
```

```
{  
  
if (n == 0) return 0;  
  
else return Count (s, n - 1) + F (s [n-1]);  
  
}
```

```
int main ()  
  
{  
  
char s [N]; fgets (s, N, stdin);  
  
printf ("count =% d \n", Count (s, strlen (s)));  
return 0;  
  
}
```

Example 4 (generating permutations). Let's give a more interesting example using the recursive method. It is required to generate all $n!$ permutations of an n -element set. Various

algorithms for solving this problem can be found in the books [13] and [15].

We will assume that the elements of the original set

$\{1, 2, \dots, n\}$ of order n are located in an n -element array a . The idea of the following simple algorithm for generating permutations is as follows. First, in the last place of array a we put the numbers $1, 2, \dots, n$ in turn, and the entire array a must always contain all the elements of the set $\{1, 2, \dots, n\}$. After that, as the element at the last place of the array a is fixed, we do the same with the remaining elements. For example, let element 1 be fixed in the last place. We will consider a subarray of array a , consisting of the first $n-1$ elements. This subarray contains numbers $2, 3, \dots, n$. Now in this subarray we will, in turn, put the numbers $2, 3, \dots, n$, etc. in the last place. After the elements are fixed in all places, we have another permutation before us.

```
#include <stdio.h> #define N 3
```

```
/* function Print displays the next  
permutation */ void Print (const int * a, const  
int n)
```

```
{
```

```
int i;
```

```
for (i = 0; i <n; i ++) printf ("%d", a [i]);
```

```
printf ("\n");
```

```
}
```

```
/* Swap function swaps variable values */  
void Swap (int * a, int * b)
```

```
{
```

```
int buf; buf = * a;
```

```
* a = * b;
```

```
* b = buf;
```

```
}
```

```
/  *   Perestanovka   function   generates  
permutations
```

```
*           a - the original array,
```

```
*           k - the number of the  
element where the next element is fixed
```

```
*           n - size of array a
```

```
* /
```

```
void Perestanovka (int * a, const int k, const  
int n)
```

```
{
```

```
if (k < 0)

{

/* display the next permutation */ / Print (a,
n);

}

else

{

int i;

for (i = 0; i <= k; i++)

{

Swap (& a [i], & a [k]); Perestанovka (a, k-1,
n); Swap (& a [i], & a [k]);

}

}

}
```

```

int main ()

{

int a [N]; int i;

/ * fill in the initial values of the array a * /
for (i = 0; i <N; i ++)

a [i] = i + 1;

Perestанovka (a, N-1, N); / * generate all
permutations * /      return 0;

}

```

Recursion problems

1. Write a recursive function double Fact (double n) of the real type that calculates the value of the factorial $n! = 1 \cdot 2 \cdot \dots \cdot n$.

2. Write a recursive function double Fact2 (double n) of real type that calculates the value of the double factorial $n!! = n \cdot (n - 2) \cdot (n - 4) \cdot \dots$, where the last factor in the

product is 2 if n is an even number, and 1 if n is odd.

3. Write a recursive function `double PowerN (double x, int n)` of real type, which finds the value of the n -th power of a number x by the formula: $x_0 = 1$, $x_n = x \cdot x_{n-1}$ for $n > 0$, $x_n = 1 / x_{-n}$ for $n < 0$.

4. Write a recursive function `long Fib (long n)` of integer type that calculates the n -th element of the sequence of Fibonacci numbers.

5. Write a recursive integer function `int DigitSum (int n)` that finds the sum of the digits of an integer n without using loop operators.

6. Write a recursive function that computes the maximum digit of an integer n .

7. Display the array using a recursive function.

8. Write a recursive function that calculates the sum of the array elements at odd positions.

9. Write a recursive function that calculates the number of odd elements in an array.

10. Write a recursive function that calculates the maximum value from all the elements in the array.

11. Write a recursive function `int DigitCount (char * s, int n)` of integer type that calculates the number of digits in string `s` without using loop operators.

12. Write a recursive function `int Symmetric (char * s, int n)` of boolean type that determines whether string `s` is a palindrome (that is, reads the same from left to right and from right to left). Do not use the loop operator in the body of the function.

13. Make up a recursive boolean function that determines whether the array a is in ascending order.

14. Write a recursive integer function that counts the number of occurrences of x in array a .

15. Construct a recursive boolean function that determines whether element x is contained in array a .

16. Build bubble sorting and direct selection sorting algorithms using recursion.

17. Write a recursive boolean function that determines whether all the elements of the array a are distinct.

18. Given two natural numbers $a > 1$ and $b > 1$. Write a recursive function for finding the GCD (a, b). Use ratios:

$$\text{Gcd}(a, 0) = \text{gcd}(0, a) = a;$$

$\text{GCD}(a, b) = \text{GCD}(ab, b)$ if $a \geq b$;

$\text{GCD}(a, b) = \text{GCD}(a, ba)$ if $b > a$.

19. Construct a recursive function C (n, m) of integer type that finds the number of combinations of n elements by m each using the formulas: $C(n, 0) = C(n, n) = 1$, $C(n, m) = C(n - 1, m) + C(n - 1, m - 1)$.

20. A piece can only move forward along a field of length n . The length of the move of a piece is no more than k . Display the various paths that the chip can take to pass the field from position 1 to position n .

21. The problem of sticking stamps. There are stamps in denominations $n_1, n_2, \dots, n_k$ rubles (the numbers $n_1, n_2, \dots, n_k$ are different), and the stock of stamps is considered unlimited. Calculate how many ways you can pay with these stamps the amount of N rubles, if two methods that differ in the order of the terms are considered different. Display all possible combinations

of these brands. When writing a recursive function, use the recursive relation

$$F(N) = F(N - n_1) + F(N - n_2) + \dots + F(N - n_k), F(N) = 0 \text{ for } N < 0,$$

$$F(0) = 1.$$

22. In array a of size n , remove the minimum number of elements so that the remaining ones form a strictly increasing sequence (or, which is the same, find the maximum strictly increasing subsequence of the sequence $a_1, a_2, \dots, a_n$ in length).

23. In array a of size n integers, find the longest subsequence of numbers such that each subsequent element of the subsequence would be divisible by the previous one.

WORK WITH MEMORY BITS

Bitwise operations

Before considering bitwise operations, let's clarify how integers are represented in binary form. As far as unsigned integers are concerned, everything is simple.

If the integer is signed, then for such numbers such a form of notation (encoding) as an additional code is usually used. This code is used so that when adding two numbers with different signs, the subtraction operation could be replaced with ordinary addition.

For example, to get -1's 8-bit's complement code, follow these steps. First, form the binary code of the modulus of the number -1, namely 1:

00000001.

Second, invert all the bits of the given number: 11111110.

Third, add one to the resulting number: 11111111.

This is the 8-bit's complement code of -1. If we now add the complementary code of the number -1 and the binary representation of the number 1, we get the following number:

100,000,000.

Since the resulting number is also considered 8-bit, then the most significant bit 1 is not taken into account, and thus the 8-bit result of the summation is 0.

The most significant bit of the complement code of binary numbers is responsible for the sign of the number: the most significant bit of non-negative numbers equals 0, negative ones - 1. Below is a table of 8-bit signed binary numbers.

Bitwise operations treat operands as ordered sets of bits, each bit can have one of two

values - 0 or 1. Such operations allow the programmer to manipulate the values of individual bits.

The shift operations $\ll$ and $\gg$ shift the bits of the first operand left or right by the number of bits specified by the second operand. In this case, the bits that go beyond the bit grid are lost, and the freed bits are filled with zeros. For example, for an 8-bit unsigned char 27, the following equalities hold:

$$00011011 \ll 1 = 00110110 \quad (27 \ll 1 = 54),$$

$$00011011 \ll 2 = 01101100 \quad (27 \ll 2 = 108),$$

$$\begin{array}{llll} 00011011 & \ll & 3 & = & 11011000 & (27 \\ \ll & & 3 = 216), \end{array}$$

$$\begin{array}{llll} 00011011 & \ll & 4 & = & 10110000 & (27 \\ \ll & & 4 = 176), \end{array}$$

$$00011011 \ll 5 = 01100000 \quad (27$$

<< 5 = 96),

00011011 >> 1 = 00001101 (27
>> 1 = 13),

00011011 >> 2 = 00000110 (27
>> 2 = 6).

If the shift to the right (>>) operation is applied to signed variables, then the vacated bits are filled with ones. For example, for an 8-bit signed number (char) -27, the following equalities take place:

11100101 << 1 = 11001010 (-27
<< 1 = -54),

11100101 << 2 = 10010100 (-27
<< 2 = -108),

11100101 << 3 = 00101000 (-27
<< 3 = 40),

11100101 >> 1 = 11110010 (-27

$\gg 1 = -14$),

$11100101 \gg 2 = 11111001$
(-27 $\gg 2 = -7$).

Note that shifting to the left (right) corresponds to multiplying (dividing) the left operand by a power of 2 equal to the value of the second operand.

The $\sim$ operation is unary, it reverses every bit of the integer. Operations $\&$, $|$ and $\wedge$ - double; the operands of these operations are integers of the same length. Operations are performed in pairs over all binary bits of the operands.

We also present some formulas using the operations $\&$, $|$ and $\wedge$, which we will need in the future:

$$x | 1 = 1, \quad x | 0 = x,$$

$$x \& 1 = x, \quad x \& 0 = 0,$$

$$x \wedge 1 = \sim x, \quad x \wedge 0 = x.$$

Examples using bitwise operations

Example 1. Write a function that prints

binary

integer representation a.

The DisplayBits function takes as a parameter the number a, which you want to represent in binary form, and the number of bits occupied by this number is n.

```
#include <stdio.h>
```

```
void DisplayBits (int a, char n)
```

```
{
```

```

char i, bit;

for (i = n - 1; i >= 0; i--)

{

/* select the i-th bit from the right */ bit = (a
>> i) & 1;

printf ("%d", bit);

}

}

```

```

int main ()

{

int a; scanf ("%d", &a);

DisplayBits (a, sizeof (int) * 8); return 0;

}

```

Example 2. Let there be an integer a. It is

required to set the i -th right bit in the number a to 1, without changing the values of other bits.

Let b denote an integer containing 1 in the i -th bit and 0 in the rest. It can be obtained like this: $b = 1 \ll i$. Then the required number is obtained in this way: $a \mid b$.

$$a \mid = 1 \ll i;$$

Example 3. Let there be an integer a . It is required to set the i -th right bit in the number a to 0, without changing the values of other bits.

Consider an integer b containing 0 in the i -th bit and 1 in the rest: $b = \sim (1 \ll i)$. Then the required number is obtained like this: $a \& b$.

$$a \& = \sim (1 \ll i);$$

Example 4. In the whole number a needs to invert the i -th bit from the right, without changing the values of other bits.

$$a \wedge = (1 \ll i);$$

Example 5. Two variables x and y of integer type are given. It is required to put the i -th right bit of the variable x on the j -th right position of the variable y , without changing the rest of the bits in y .

First, we zero out all the bits in the x variable, except for the i -th right bit x_i , which we put on the j -th right position in x :

$$((x \gg i) \& 1) \ll j.$$

Then we zero out the j -th from the right bit in the y variable, without changing the values of the other bits:

$$y \& \sim (1 \ll j).$$

To solve the problem, it remains to apply the bitwise OR operation to the previous two expressions:

$$y = ((x \gg i) \& 1) \ll j \mid y \& \sim (1 \ll j).$$

Example 6. Find the number of unit bits in a non-negative integer a .

The BitCount function checks the state of the least significant bit each time, gradually shifting all the bits one position to the right, and so on until all n bits of the integer have been checked.

```
short BitCount (unsigned int a, short n)
```

```
{
```

```
    short count; int i;
```

```
    count = 0;
```

```
for (i = 0; i <n; i ++)  
  
{  
  
if (a & 1) count ++;  
  
a >>= 1;  
  
}  
  
return count;  
  
}
```

The BitCount function can be slightly optimized, taking into account the fact that if after the next shift of all bits one position to the right the number *a* becomes zero, then there are no more one bits in it. Therefore, there is no need to always look at all *n* bits, but only until the most significant (left) one bit is taken into account.

```

short BitCount2 (unsigned int a)
{
    short count; count = 0; while (a != 0)
    {
        if (a & 1) count ++;
        a >>= 1;
    }

    return count;
}

```

Example 7. An integer $n > 0$ is given, which is some power of 2: $n = 2^k$. Find an integer k - an exponent of this degree.

```

int Degree (unsigned int n)
{

```

```

int k; k = -1;

while (n)

{

n >> = 1; k ++;

}

return k;

}

```

Example 8 (fast exponentiation algorithm).

Let it be required to calculate a^n , where a and n are some integers, and $n \geq 0$. Consider the following powers of a :

$$a, a^2, a^4, a^8, \dots, a^{2^t},$$

where $t = \lceil \log_2 n \rceil$. Note that each number from the specified sequence is obtained by multiplying the previous number by itself.

We represent the number n in the form of such an expansion:

$$n = n_t 2^t + n_{t-1} 2^{t-1} + \dots + n_1 2 + n_0,$$

where $n_0, n_1, \dots, n_t$ are numbers from the set $\{0, 1\}$. Then the number a^n can be calculated as follows:

$$a^n = a^{n_t 2^t + n_{t-1} 2^{t-1} + \dots + n_1 2 + n_0} = a^{n_t 2^t} a^{n_{t-1} 2^{t-1}} \dots a^{n_1 2} a^{n_0} =$$

...

Therefore, the number of multiplication operations in calculating a^n by this method does not exceed $2 \log_2 n$.

long Degree (int a, unsigned int n)

{

long deg, /* powers of a: a, a², a⁴, a⁸, ... */ rez; /*
* exponentiation result */

rez = 1; deg = a;

while (n != 0)

```

{
    if (n & 1) rez * = deg;
    deg * = deg; n >> = 1;
}

return rez;

}

```

Tasks

1. Find the number of zero and one bits in a non-negative integer a .
2. Let there be an array a of size $n \leq 32$, consisting of zeros and ones. It is required to write the values of the elements of the array a into a variable of the unsigned long type as a sequence of bits $a[n-1] \dots a[1] a[0]$.
3. Write a function for checking the parity of integers using only bitwise

operations.

4. Let an integer a be given. Print first the values of its bits with even indices, and then with odd ones.

5. Let an integer a be given. Check if it interleaves one and zero bits.

6. In the whole number a , shift all the bits to the left by k positions, replacing the freed bits with ones.

7. Cyclic shift bits in the integer number a by

k positions to the left.

8. Cyclic shift bits in the integer number a by

k positions to the right.

9. Find the number of the first (last)

one bit on the right in the integer a .

10. Find the number of the first (last) zero bit on the right in the integer a .

11. Determine if the bits in the integer a are arranged in ascending order, that is, is the given number representable as $2^n - 1$ for some n .

12. Rearrange the bits in the whole number in reverse order.

13. Let there be an integer $a > 0$. Write the octal representation of this number into a string variable using the following rule: the binary representation of the number a is split from right to left into triads (triples of digits), and each triad is replaced with an octal digit.

14. Let there be an integer $a > 0$. Write the hexadecimal representation of this number into the string variable, using the rule from the previous problem, breaking the binary representation of the number into

tetrads (four digits).

15. Write function

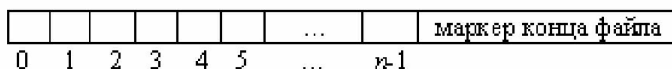
`unsigned long Perest (unsigned long x,
unsigned char * pi, char n)`, which permutes
the bits in number `x` in accordance with the
permutation `pi` from S_n ($n = 32$) and returns
the resulting value as the result.

WORKING WITH FILES

Files and streams

File - a named collection of arbitrary data located on an external storage device, which is stored, sent and processed as a whole. The file can contain a program, numeric data, text, encoded image, etc.

In the C language, any file is considered as a sequential stream of bytes, which ends with an end-of-file marker:



To begin with, note that the input data is not sent to the disk or printing device immediately. Since writing data to a disk and reading it from a disk take a lot of time, they first enter a memory area called a buffer,

which is intended for temporary storage of information. When the buffer is full, the data is sent at once to the disk or printer. The data read from the disk is also transferred in portions first to the buffer.

In order to write data to a file or read it from a file, you need to open the file using the `fopen` function from the `<stdio.h>` library. An open file returns a pointer (called a file pointer) to a `FILE` structure that contains information for working with the file. This structure contains: the address of the file buffer; the number of characters remaining in the buffer; the position of the current character in the buffer; a file is open for reading or writing; whether there were any errors when working with the file; whether an end-of-file marker has been encountered.

When writing to a file or reading from a file, the program obtains the necessary information from the `FILE` structure. A pointer to this structure is defined as follows:

```
FILE * f;
```

The `fopen ()` function returns a file pointer:
`FILE * fopen (char * name, char * mode)`

The call to `fopen ()` looks like this: `f = fopen (name, mode);`

An attempt to open a non-existent file for reading will immediately result in an error. Other errors may also occur, such as an attempt to read a read-protected file, and so on. If an error occurs while opening the file, the `fopen` function will return `NULL`. For example, the following instructions

```
FILE * f;
```

```
if ((f = fopen ("c: \\ a.data", "r")) == NULL)  
printf ("File could not be opened \n");
```

mean an attempt to open the file “c: \\ a.data” for reading. If an error occurred while opening the file (the file pointer is `NULL`), then the message “File could not be opened”

is displayed on the screen.

Let's consider the main library functions (library

<stdio.h>) for working with files.

`FILE * fopen (const char * filename, const char * mode)` The `fopen ()` function opens a file with the given name and returns a file pointer or `NULL` if the file could not be opened.

`int fflush (FILE * f)`

Records all unwritten data remaining in the buffer. This function is undefined for an input stream. The function returns `EOF` if an error occurs, otherwise `- 0`. The `fflush (NULL)` instruction performs the specified actions for all output streams.

`int fclose (FILE * f)`

Closes file `f`. Before that, there is a write of not yet written data from the buffer, or reading of data that has not yet been read from the buffer. When the stream is closed, the buffers captured by the system are freed. The function returns EOF on error and 0 otherwise.

`int feof (FILE * f)`

Determines if the end of the file has been reached. When it is reached, the function returns a nonzero value, otherwise it returns 0.

`int remove (const char * filename)`

Deletes the file with the specified name. Returns a nonzero value on error, otherwise 0 is returned.

int rename (const char * oldname, const char * newname) Rename () function replaces old file

name

new *newname*; if the attempt to change the name was unsuccessful, then it returns a nonzero value, otherwise - 0.

int fputc (int c, FILE * f)

Writes character c to file f. The function returns the written character on success, otherwise - EOF.

`int fgetc (FILE * f)`

Reads the next character from the file `f`, which is pointed to by the file pointer. The function returns the read character on success; if the end of the file is reached - EOF.

`int fputs (const char * s, FILE * f)`

Writes a character string `s` to file `f`. Returns the number of characters written; in case of error - EOF.

`char * fgets (char * s, int n, FILE * f)`

Reads a character string from a file into a character array `s`. In this case, no more than `n-1` characters are read, stopping reading if the `'\n'` character is encountered. The null character `'\0'` is also appended to the string `s`. The function returns `s`; if the file is exhausted,

then it is NULL.

`int fprintf (FILE * f, const char * format, argument_1,..., argument_n)` Writes characters to file `f` under format control. Returns the number of characters written; in case of error - EOF.

`int fscanf (FILE * f, const char * format, & argument_1,..., & argument_n)` Reads data from file `f` under format control. Returns the number of elements transformed; in case of error - EOF. This function works similarly to the `scanf ()` function (see section 2.2. Format I / O).

`int fseek (FILE * f, long count, int origin)`

Sets the file pointer `f` to a specific byte. Subsequent reading or writing will be carried out from this byte (from this position). In the

case of a binary file, the position is set at offset count from:

- the beginning of the file, if the value of the origin variable is

SEEK_SET;

- the current position if the value of the origin variable is

SEEK_CUR;

- end of file if origin is

SEEK_END.

int rewind (FILE * f)

Move the pointer to the beginning of the file.

This operator is equivalent to writing

```
fseek (f, 0, SEEK_SET);
```

```
long ftell (FILE * f)
```

Returns the current position of the file pointer f; on error, the function returns -1.

```
int ferror (FILE * f)
```

Returns a nonzero value if an error occurred during the next operation while working with file f, otherwise 0 is returned.

Text files

A text file is a sequence of lines, each line ending with a '\n' character and padded with

a '\ 0' character. A text file consists of ordinary printable characters (in ASCII codes), including spaces, newlines, and tabs. For example, in a text file, the integer 123 is represented by three digits '1', '2' and '3', which are located one after another in the file, forming the word “123”.

To specify that the file is open in text mode, you can add the 't' symbol to the value of the open mode mode: “rt”, “wt”, “at”, “r + t”, “w + t”, “ a + t ”.

Let's give examples of using library functions to work with text files.

Example 1 (function fputc ()). The following is a function to write character-by-character to a file named fileName, which continues until a simulated end-of-file character EOF is encountered in the standard stream.

Many operating systems allow you to simulate an end-of-file condition using the keyboard. On MS-DOS, this is accomplished by using the Ctrl + z key combination and

then pressing the Enter key. On Windows, Ctrl + z is used to simulate the end of a file, and Ctrl + d on UNIX.

```
int CreateTextFile (char * fileName)

{

FILE * f;

char c;

if ((f = fopen (fileName, "wt")) == NULL)
return 1;

while ((c = getchar ()) != EOF) fputc (c, f);

fclose (f); return 0;

}
```

Example 2 (function fgetc()). Display the contents of a text file on the screen. The function reads from the file one character at a time and displays the read characters on the

screen.

```
int ReadTextFile (char * fileName)
{
    FILE * f;

    char c;

    if ((f = fopen (fileName, "rt")) == NULL)
        return 1;

    while ((c = fgetc (f)) != EOF) putchar (c);

    fclose (f); return 0;
}
```

Example 3 (function fputs()). Line-by-line filling of the file. In the following program, 10 lines are entered from the keyboard and written line by line to a text file.

```
int CreateTextFile (char * fileName)

{

FILE * f;

char s [300]; int i;

if ((f = fopen (fileName, "wt")) == NULL)
return 1;

for (i = 0; i <10; i ++)

{

fgets (s, 300, stdin); / * enter string s from the
keyboard * / fputs (s, f); / * write string s to
file* /

}

fclose (f); return 0;

}
```

Example 4 (fgets () function). Reading the file line by line.

```
int ReadTextFile (char * fileName)
{
    FILE * f;

    char line [500];

    if ((f = fopen (fileName, "rt")) == NULL)
        return 1;

    while (fgets (line, 500, f) != NULL)
    {
        printf (line);
    }

    fclose (f); return 0;
}
```

Functions that work with characters and strings are intended only for reading and writing text. If you need to write or read data containing numeric values from a file, then you need to use the `fprintf ()` and `fscanf ()` functions.

Example 5 (function `fprintf ()`). The next function writes information about employees of a certain organization to a text file: employee surname, year of birth, salary. Data entry from the keyboard stops when the user enters the string “000” at the request to enter the next surname of the employee.

```
int CreateFile (char * fileName)

{

FILE * f;

char          name          [50];          /
*                                     surname*/

int           year;          /
*
```

```
year birth * /
```

```
float earnings; /  
* salary * /
```

```
if ((f = fopen (fileName, "wt")) == NULL)
```

```
return 1; printf ("name:");
```

```
scanf ("% s", name);
```

```
while (strcmp (name, "000"))
```

```
{
```

```
printf ("year:");
```

```
scanf ("% d", & year); printf ("earnings:");
```

```
scanf ("% f", & earnings);
```

```
fprintf (f, "% s% d% .2f \ n", name, year,  
earnings); printf ("name:");
```

```
scanf ("% s", name);
```

```
}
```

```
fclose (f); return 0;
```

```
}
```

Example 6 (function fscanf ()). Reading records from a file containing information about employees. The creation of this file is shown in the previous example.

```
int ReadFile (char * fileName)
```

```
{
```

```
FILE * f;
```

```
char          name          [50];          /
```

```
*
```

```
surname*/
```

```
int           year;          /
```

```
*
```

```
year birth * /
```

```

float          earnings;          /
*
* / if ((f = fopen (fileName, "rt")) == NULL)
    salary

return 1;

while (fscanf (f, "% s% d% f", name, & year,
& earnings) != EOF)

{

printf ("% s% d% .2f \ n", name, year,
earnings);

}

fclose (f); return 0;

}

```

Example 7 (function fscanf()). The following function extracts all words from a text file, which are separated by whitespace (" ", "\ t", "\ n"), and prints these words to the

screen.

```
int ReadWords (char * fileName)

{

FILE * f;

char s [500];

if ((f = fopen (fileName, "rt")) == NULL)
return 1;

while (fscanf (f, "% s", s) != EOF)

{

puts (s);

}

fclose (f); return 0;

}
```

Binary files

A binary file usually consists of a sequence of records of the same length and the same internal format, with the records continuously following each other.

Binary format means numbers are stored internally. For a character, the binary representation is the same as its textual representation (the ASCII binary representation of the character code). For numbers, their binary representation is very different from their textual representation. For example, the number 123 in binary has a representation 1111011 and places-

Xia in one byte of memory. In text format, this number would take 3 bytes of memory.

To write and read data in binary format, use the `fwrite ()` and `fread ()` functions, respectively. After writing data with `fwrite ()`, you can return that data with `fread ()`. The prototypes for these functions are as follows:

```
size_t fwrite (void * ptr, size_t size, size_t n,  
FILE * f) size_t fread (void * ptr, size_t size,  
size_t n, FILE * f)
```

The `fwrite ()` function writes to file `f` from the array specified in the `ptr` variable, `n` elements, the size of which is specified by the `size` parameter. This function returns the number of items successfully written.

The `fread ()` function reads from file `f` into the array specified in the `ptr` variable no more than `n` elements, the size of which is specified by the `size` parameter. This function returns the number of items successfully read.

If it is required, for example, to write the value of a variable `x` of type `double` to a binary file `f`, then this can be done as follows:

```
fwrite (& x, sizeof (x), 1, f);
```

If you want to write to the file an array of integers `a` containing `n` elements, then this can be done as follows:

```
fwrite (a, sizeof (int), n, f);
```

To specify that the file is open in binary mode, add the 'b' character to the open mode value: “rb”, “wb”, “ab”, “r + b”, “w + b”, “a + b”. Let's look at some examples of working with binary files.

Example 1. Let's fill the binary file with

```
integers. int CreateFile (char * fileName)
```

```
{
```

```
FILE * f;
```

```
int a;
```

```
if ((f = fopen (fileName, "wb")) == NULL)  
return 1;
```

```
printf ("a ="); scanf ("% d", & a); while (a! =  
0)
```

```
{  
    fwrite (& a, sizeof (a), 1, f);  
    printf ("a ="); scanf ("% d", & a);  
}  
  
fclose (f); return 0;  
}
```

Example 2. Reading the file of integers created in the previous example.

```
int ReadFile (char * fileName)  
{  
    FILE * f;  
  
    int a;
```

```
if ((f = fopen (fileName, "rb")) == NULL)
return 1;

while (fread (& a, sizeof (a), 1, f)) printf ("a
=%d \n", a);

fclose (f); return 0;

}
```

Example 3. The following Size () function returns the number of bytes occupied by the file named fileName. If no such file exists, then the function returns -1.

```
long Size (char * fileName)

{

FILE * f;

long n;
```

```

if ((f = fopen (fileName, "rb")) == NULL)
return -1;

fseek (f, 0, SEEK_END); /*move the pointer
to the end of the file */ n = ftell (f); /* read
the current pointer position */      fclose
(f);

return n;

}

```

Example 4. Reading a binary file of integers in reverse order.

```

int ReadConverseFile (char * fileName)

{

FILE * f;

int a; long i, n;

if ((f = fopen (fileName, "rb")) == NULL)
return 1;

```

```

n = Size (fileName) / sizeof (int); / *number
of components in the file * / for (i = n - 1; i>
= 0; i--)

{

fseek (f, i * sizeof (int), SEEK_SET); fread
(& a, sizeof (a), 1, f);

printf ("a =% d \ n", a);

}

fclose (f); return 0;

}

```

Example 5. It is required to replace all elements of the binary file of integers with their squares.

```

int SquareRechange (char * fileName)

```



```
{  
  
FILE * f;  
  
long i, n; int a;  
  
if ((f = fopen (fileName, "r + b")) == NULL)  
return 1;  
  
n = Size (fileName) / sizeof (int); for (i = 0; i  
<n; i ++)  
  
{  
  
fseek (f, i * sizeof (a), SEEK_SET); fread (&  
a, sizeof (a), 1, f);  
  
a * = a;  
  
fseek (f, i * sizeof (a), SEEK_SET); fwrite (&  
a, sizeof (a), 1, f);  
  
}  
  
fclose (f); return 0;  
  
}
```

The `fwrite ()` function allows entire structures to be written to a file, which can then be read with `fread ()`.

Example 6. Suppose there is information about the employees of a certain organization: the name of the employee, year of birth, salary. Let's combine the listed data items into the `WORKER` structure:

```
struct WORKER  
  
{  
  
    char name [20]; / *surname*/ int year; / *  
    year of birth * / float earnings; /* wage */  
  
};
```

To write the next information about the worker (the next structure) to disk, use the following instruction:

```
fwrite (& worker, sizeof (worker), 1, f);
```

where the worker variable is of type WORKER. Similarly, to read the next structure, we use the instruction

```
fread (& worker, sizeof (worker), 1, f);
```

In the program below, the CreateFile () function fills the worker information file, and the ReadFile () function displays the contents of the file to the screen.

```
#include <stdio.h> struct WORKER
```

```
{
```

```
char name [20]; int year;
```

```
float earnings;
```

```
};
```

```
/* fill in the file */
```

```

int CreateFile (char * fileName)

{

FILE * f;

struct WORKER worker;

if ((f = fopen (fileName, "wb")) == NULL)
return 1;

printf ("name:"); scanf ("% s", worker.name);

while (strcmp (worker.name, "000"))

{

printf ("year:");  scanf ("% d",  &
worker.year); printf ("earnings:");

scanf ("% f", & worker.earnings);

fwrite (& worker, sizeof (worker), 1, f); /
*writing structure to file */ printf ("name:");

scanf ("% s", worker.name);

```

```
}
```

```
fclose (f); return 0;
```

```
}
```

```
/* read file */
```

```
int ReadFile (char * fileName)
```

```
{
```

```
FILE * f;
```

```
struct WORKER worker;
```

```
if ((f = fopen (fileName, "rb")) == NULL)  
return 1;
```

```
while (fread (& worker, sizeof (worker), 1, f))
```

```
{
```

```
printf ("% s", worker.name); printf ("% d",  
worker.year);
```

```
printf ("%%. 2f \n", worker.earnings);
```

```
}
```

```
fclose (f); return 0;
```

```
}
```

```
int main ()
```

```
{
```

```
char * fileName = "c: \\ a.data"; CreateFile  
(fileName); ReadFile (fileName);
```

```
return 0;
```

```
}
```

Example 7. Sometimes you just need to add one entry to the end of the file. Therefore, we can write the function of filling the file from the previous example in the following form.

```
/* function to add an entry to the end of the  
file f */
```

```
int AddWorker (FILE * f, const struct  
WORKER * worker)
```

```
{
```

```
fseek (f, 0, SEEK_END); /*move the pointer  
to the end of the file f */ fwrite (worker,  
sizeof (* worker), 1, f);
```

```
if (ferror (f)) return 1;
```

```
else return 0;
```

```
}
```

```
/* fill in the file */
```

```
int CreateFile (char * fileName)
```

```
{
```

```
FILE * f;

struct WORKER worker;

if ((f = fopen (fileName, "wb")) == NULL)
return 1;

printf ("name:"); scanf ("% s", worker.name);

while (strcmp (worker.name, "000"))

{

printf ("year:");  scanf ("% d",  &
worker.year); printf ("earnings:");

scanf ("% f", & worker.earnings);

AddWorker (f, & worker); / * add an entry to
the file * / printf ("name:");

scanf ("% s", worker.name);

}

fclose (f); return 0;
```



```
}
```

Example 8. Very often you have to copy files. Let's consider a fast algorithm for copying files using the clipboard. The following is the CopyFile () function, which copies the contents of fileNameIn into file fileNameOut.

```
#define LENGTH 1024 /* buffer size */
```

```
int CopyFile (const char * fileNameIn, const  
char * fileNameOut)
```

```
{
```

```
FILE * in, * out;
```

```
char buffer [LENGTH]; long n;
```

```

in = fopen (fileNameIn, "rb");

out = fopen (fileNameOut, "wb"); if (in ==
NULL || out == NULL)

{

fcloseall (); / * close all streams *
/ return 1;

}

while (n = fread (buffer, sizeof (char),
LENGTH, in)) fwrite (buffer, sizeof (char), n,
out);

fcloseall (); return 0;

}

```

Tasks for text files

1. There is a text file. Calculate the

number of Latin letters contained in it.

2. Determine how many distinct letters appear in the text file.

3. Given string s and a text file. Write a function that appends the string s to the end of a text file.

4. You are given a file name and an integer n , $1 \leq n \leq 26$. Create a text file with the specified name and write n lines into it as follows: the first line must contain a lowercase Latin letter "a", the second - the letters "ab", third - letters

"Abc", etc .; the last line must contain n initial lowercase Latin letters in alphabetical order.

5. Find the first shortest line in a text file.

6. In the text file, remove all blank

lines.

7. Duplicate in a text file all those lines that contain a space character.

8. Replace all consecutive spaces in the text file with one space.

9. In a text file, count the number of occurrences of each lowercase Latin letter and create a new text file, each line of which has the form "<letter> - <number of its occurrences>". Do not include letters that are not in the text in the file. Letters must be in alphabetical order.

10. Given a text file. Find the number of paragraphs in the text if paragraphs are separated from each other by one or more blank lines.

11. You are given a text file containing left-justified text. Each line contains no more than 100 characters. Align the text to the right

by adding the required number of spaces to the beginning of each nonblank line.

12. Given a text file, each line contains no more than 100 characters. Write all lines of the file to a new file, changing the order of the letters in each line to the opposite.

13. Given a text file, each line of which represents an integer, padded on the left and right with several spaces. Find the number of these numbers and their sum.

14. You are given a text file that contains an $m \times n$ table of integers. Find the sum of the elements of the k -th column of the given table.

15. You are given a text file that contains an $m \times n$ table of integers. Find the sum of the elements of all its even strings.

16. You are given a text file that contains an $n \times n$ table of integers. Find the sum of the elements of the main diagonal of

the given table.

17. You are given a text file that contains an $n \times n$ table of integers. Write to another text file the same table, only in transposed form. Do this with and without using an additional two-dimensional array.

18. Print all words from a text file that do not contain any numbers.

19. Write all words from a source text file line by line to another text file.

20. Find the very first word starting with a capital letter in the text file.

21. Print all words of maximum length from a text file.

22. You are given a text file that contains a table of $m \times n$ integers, and some of the numbers contain extraneous characters. It is required to display the coordinates of all

such incorrectly written numbers.

23. Given a text file that contains a table of student details. The first column contains surnames, the second - the year of birth, the third - the average grade. a) Display only those lines of this table that contain the maximum average score. b) Display information only about those students who, among the youngest, have the highest GPA.

24. In a text file, count the number of occurrences of each word and create a new text file, each line of which looks like “<word> - <number of its occurrences>”. Words must be in the order of their first appearance in the source file.

Self-study tasks

25. Given a text file. Print the number of lines it contains.

26. You are given a file name and an integer n , $1 \leq n \leq 26$. Create a text file with the specified name and write n lines of length n to it; line with number k , $k = 1, \dots, n$, must contain k initial uppercase (that is, uppercase) Latin letters, padded on the right with "*" (asterisk). For example, for $n = 4$ the file must contain the lines "A ***", "AB **", "ABC *", "ABCD".

27. Given string s and a text file. Add line s to the beginning of the file.

28. Given a text file. Calculate the frequency of occurrences of each character in the given file.

29. Replace all uppercase Latin letters in the text file with lowercase, and all lowercase with uppercase.

30. Find the first longest line in a text file.

31. In the text file, replace all empty lines with the string s.

32. Two text files are given. Append the contents of the second file to the end of the first file.

33. Duplicate all non-blank lines in a text file.

34. Given a text file. Find the number of paragraphs in the text if the first line of each paragraph starts with 5 spaces ("red line"). Ignore empty lines between paragraphs.

35. You are given a text file that contains an $m \times n$ table of integers. Find the sum of the elements of the k-th row of the given table.

36. You are given a text file that contains an $n \times n$ table of integers. Find the sum of the elements of the opposite diagonal

of the given table.

37. Print the last word of the smallest length in a text file.

38. Count the number of symmetric words in a text file.

39. Print all words from a text file containing at least one digit.

40. Print all words of minimum length from a text file.

41. From the original text file, write line by line all words that do not contain numbers to another text file.

42. You are given a text file containing left-justified text. Each line contains no more than 100 characters. Align the text to the center by adding the required number of spaces at the beginning of each nonblank line.

43. You are given a text file, each line of which represents an integer or a real number, padded on the left and right with several spaces (real numbers have a nonzero fractional part). Print the number of integers and their sum.

44. The file contains information about books: author's name, book title, number of pages, price. a) Determine the cheapest book. b) Among the most expensive books, find the books with the fewest pages.

Binary Tasks

1. In the integers file, remove all negative numbers.

2. A file of real numbers is given. Create two new files, the first of which contains elements of the original file with odd

numbers, and the second - with even ones.

3. Given a file of integers. Duplicate all odd-numbered elements in it.

4. Given a file of integers. Create a new file containing the same items as the original file, but in reverse order.

5. It is required to replace all elements of the binary file of integers with their modules.

6. Given a file of integers. Check if it is symmetrical.

7. Two arbitrary files are given. It is required to add the contents of the second file to the first file, and the contents of the first to the second file.

8. Two files of real numbers are given, the elements of which are ordered in ascending order. Combine these files into a

new file so that its elements also appear in ascending order.

9. Fill in the file with real numbers, then find the number of its increments.

10. Given a file of integers. Double its size by writing all its original elements to the end of the file (in the same order).

11. Given a file of integers. Double its size by writing all its original elements to the end of the file (in reverse order).

12. Given a file of integers. Arrange its items in ascending order.

13. There is a file containing the coordinates of points in the xOy plane, each record contains the abscissa and ordinate of a point. a) Find the coordinates of the vertices of the rectangle that will contain all the specified points. b) Find the radius of a circle centered at the origin that will contain all of the point data.

14. There is a file of vehicle records. Each record contains the following fields:

```
char mark [30]; / * car make * / unsigned int  
year; /* year of issue */
```

```
double mas, speed; / * mass and maximum  
speed of the car * /
```

a) Find how many fastest cars are there. b) Print all brands of cars without repetitions. c) Find the fastest among the oldest cars. d) Print the brands of only the newest cars in order of increasing weight.

15. Some file (text or binary) is given. Write function

```
void Sdvig (char * fileName, char k),
```

which encrypts the file named fileName using a shift method using an integer key k. This encryption method is as follows. Consider the source file as a sequence of bytes (of type char) by opening it in binary format. Let k be some fixed integer of type char. Reading the value of the next byte from the file and writing it to a variable of type char c, we

convert it in this way: $c + = k$. In this case, addition occurs modulo

256. The received new value is written in place of the just read byte in the source file. And so with all bytes-

the file in question. To speed up the encryption algorithm, it is desirable to read the bytes from the file in whole blocks, convert them, and then write them to their previous positions. Also write a function to decrypt the file. Note that in order to decrypt a file, it is necessary to perform the $c - = k$ operation with all bytes of the encrypted file.

16. Given file f . Write function

`void Vizhener (char * fileName, char * key),`

which encrypts it using the Vigenere method. Namely: let $key = k_1k_2... k_s$ be some word of length s (encryption key). We read the first byte of file f and encrypt it using the key k_1 (by the method described in the previous task), then we read the second byte of the file

f and encrypt it using the key k_2 , and so on. The $(s + 1)$ th read byte is encrypted again with the key k_1 , that is, in general, the file f is split into blocks of length s, on which the keyword key is superimposed. Also write a function to decrypt the file.

17. Given file f. Write function

`void Zamena (char * fileName, char k [256]),`

which encrypts it using a simple replacement method. Let's describe this method. Let A be the set of all values of a variable of type char, $S(A)$ the symmetric permutation group of the set A. The encryption key will be an arbitrary fixed permutation $k \in S(A)$. If $x_1x_2 \dots x_s$ is the content of file f, then it is encrypted into ciphertext $y_1y_2 \dots y_s =$

$= k(x_1) k(x_2) \dots k(x_s)$. The decryption process looks as follows: $x_1x_2 \dots x_s = k^{-1}(y_1) k^{-1}(y_2) \dots k^{-1}(y_s)$, where k^{-1} is the substitution from $S(A)$, inverse to the substitution k.

Self-study tasks

18. Given a file of integers. Create two new files, the first of which contains even numbers from the original file, and the second contains odd numbers.

19. A file of real numbers is given. Find the arithmetic mean of its elements and the sum of its even-numbered elements.

20. In the file of integers, find the number of series, that is, sets of contiguous identical elements.

21. Given a file of integers. Create a new file of integers containing the lengths of all series of the original file.

22. Swap the minimum and maximum elements in the file of real numbers.

23. A file of real numbers is given. Replace each element in the file, except for the first and last, with its arithmetic mean with the previous and next element.

24. Given N files of real numbers (FILE * f [N]), the elements of which are ordered in ascending order. Combine all these files into a new file so that its elements are also in ascending order.

25. There is a file of student records. Each record contains the following fields:

```
char name [30]; / * last name student * /
```

```
unsigned int year; / * year of birth * / struct  
SUBJECT mark [10]; / * array of items * /
```

where SUBJECT is a structure defined as follows: struct SUBJECT

```
{
```

```
char subjectName [30]; / * subject name * /  
int z; / * score on subject * /
```

```
};
```

There is also a file containing information about the instructors: char subjectName [30];
/* item name */

```
char name [30]; /* last name teacher */
```

a) Print the ranking of students by academic performance, with students with the same rating ranked by age. b) For each teacher, print a list of the most successful students in his subject.

26. The file contains information about the results of a session of a certain group: name of the student, grades for five exams. Calculate the number of students with the highest score.

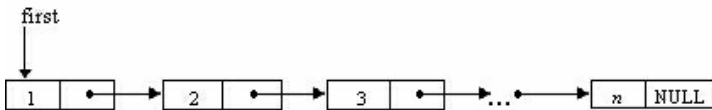
27. The file contains information about books: author's name, book title, number of pages, price. a) Determine the most appropriate

horny book. b) Among the cheapest books, find the books with the largest number of pages.

28. The file contains descriptions of goods available for sale in a trade organization. The product description includes the name, price and quantity of the product. a) Determine the most expensive product. b) Find the most expensive among the smallest goods.

DATA STRUCTURES

Singly linked lists



A list is a finite set of dynamic elements located in different memory areas, linearly connected to each other using special pointers. Each element of a singly linked list has one or more information fields and a link to another element:

Each element of the list is described as

follows: struct ELEMENT

{

```
int data; / *                information field
list * /

struct ELEMENT * next; / *pointer to the
next item in the list * /

};
```

The linked list is accessed through a pointer

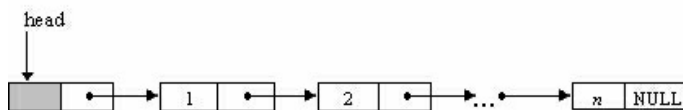
first to the first element of the list, which is defined as follows: struct ELEMENT * first;

Subsequent list items are accessible via reference fields. The last element's reference field is NULL to mark the end of the list. The data in a linked list is stored dynamically, that is, a new item is created or some existing list item is deleted as needed.

If you add an additional link at the beginning of a regular unidirectional list, which will not be considered as an element

cop of the list, and as an element containing the address of the first element of the list, we get a list with a capital link. The pointer to the main element is denoted by head:

```
struct ELEMENT * head;
```



For such lists, some processing algorithms are written easier, since the list always contains at least one link (even if it is empty).

In the C language, the `malloc ()` function is used to create dynamic objects, which has one parameter:

```
void * malloc (int size)
```

`Malloc ()` returns a pointer to a memory location for an object of size `size`. If there is no memory of the requested size, then the function returns `NULL`.

For example, to allocate memory for a list item, you need to use this function as follows:

```
struct ELEMENT * q; / * pointer to list  
element * / q = (struct ELEMENT *) malloc  
(sizeof (struct ELEMENT));
```

This function creates a dynamic object of size (in bytes) `sizeof (struct ELEMENT)` and returns the address of the allocated memory area. A dynamic object created using the `malloc ()` function will exist until the end of the main program, or until it is destroyed by the `free ()` function:

```
void free (void * p)
```

The `free ()` function frees the memory area it points to

p... If *p* is `NULL`, then this function does nothing. It is very important to note that

repeated use of the free () function on the same pointer will result in an error in the program. For example, the following record will be erroneous:

```
int * a;
```

```
a = (int *) malloc (sizeof (int)); free (a);
```

```
free (a); / *                               error */
```

Calling the free () function for a null pointer is absolutely painless. Therefore, it is recommended to assign a zero value to the pointer associated with it after freeing the dynamic memory area. Therefore, the previous error can be corrected in this way:

```
int * a;
```

```
a = (int *) malloc (sizeof (int)); free (a);
```

```
a = NULL;
```

```
free (a);
```

The following operations are usually applied to linear lists:

- 1) search for an element with a given property;
- 2) printing all the elements of the list, starting with some element;
- 3) determining the number of items in the list;
- 4) inserting an additional list item before or after the specified item;
- 5) deleting an element with a given property;

6) joining two lists into one list in one way or another (for example, merging two ordered lists while maintaining order);

7) splitting the list into several sublists;

8) deleting all elements of the list.

Access to the information and reference field of the list item q (if only memory for the item q was previously allocated by the malloc () function) is carried out, therefore, as follows:

`(* q) .data`

`(* q) .next`

There is another equivalent form of accessing the fields of an element q:

`q-> data q-> next`

It is the second form that we will use in the following examples. Compare, for example, accessing the fields of the next element after q:

`(* (* q) .next) .data`

`(* (* q) .next) .next`

`q-> next-> data q-> next-> next`

Also, instead of giving everyone

ELEMENT, we use the typedef facility:

```
typedef struct ELEMENT
{
    int data;

    struct ELEMENT * next;
} ELEMENT;
```

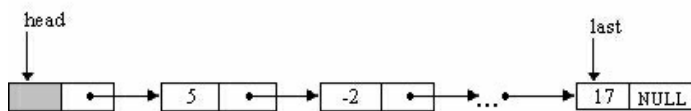
Now you can just write ELEMENT, such as ELEMENT q, rather than struct ELEMENT q.

**Examples of working with
singly linked lists**

The examples in this section use overriding lists. In the information field of the head element, if the situation permits, you can store service information, for example, the number of elements in the list, etc.

Example 1. You want to create a list of elements of a sequence of integers entered by the user. You also need to write a function to delete all elements of the list that have some value x .

In the implementation of the algorithm for solving this problem, we need a pointer variable *head*, which will contain the address of the main element of the list, and a pointer variable *last*, which will contain the address of the last element of the list as its value:



The operation of adding a new element to the end of the list is formatted as a separate function

AddElement (ELEMENT ** last, int x)

This function receives as arguments an integer x, which must be written to the information field of the created element, and a pointer to the last pointer variable (this is done because when adding a new element to the end of the list, the address of this element must be stored in the last variable , so the address of the pointer variable is passed).

```
#include <stdio.h> #include <malloc.h>
```

```
/* list item */ typedef struct ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * next;
```

```
} ELEMENT;
```

```

/*creation   header element list */

/ ** head   - pointer to heading element, */

/ ** last   - pointer to the last element*/

void CreateHead (ELEMENT ** head,
ELEMENT ** last)

{

* head = (ELEMENT *) malloc (sizeof
(ELEMENT)); (* head) -> next = NULL;

* last = * head;

}


/ * add a new element with the value x to the
end of the list, that is, after the element
pointed to by the last pointer

* /

void AddElement (ELEMENT ** last, int x)

```



```

{

ELEMENT * q; / *          new list item *
/

/ * allocate memory for the next element of
the list: * / q = (ELEMENT *) malloc (sizeof
(ELEMENT));

/ * fill in the information and reference fields
of the new element: * / q-> data = x;

q-> next = NULL;

/ * bind the new q element to the rest of the
list, that is, in the reference field of the * last
element, write the address of the new
element:

* /

(* last) -> next = q;

* last = q;

/ * now the added element is the last one list

```

item

* /

}

/* display all the elements of the list */ void
Print (ELEMENT * head)

{

ELEMENT * q;

q = head-> next; while (q! = NULL)

{

printf ("% d", q-> data); q = q-> next;

}

printf ("\ n");

}

```
/* remove all elements with value x */
```

/* to remove a list item with a value x, you need to view the contents of the next item after the item each time,

pointed to by the current pointer q

```
*/
```

```
void DeleteElement (ELEMENT * head,  
ELEMENT ** last, int x)
```

```
{
```

```
    ELEMENT * q, * t; q = head;
```

```
    while (q->next != NULL)
```

```
    {
```

```
        if (q->next->data == x)
```

```
        {
```

```
            t = q->next;
```

```
q-> next = q-> next-> next; free (t);
```

```
}
```

```
else q = q-> next;
```

```
}
```

```
* last = q;
```

```
}
```

```
/* remove the entire list, not including the  
head element */ void Destruct (ELEMENT *  
head, ELEMENT ** last)
```

```
{
```

```
ELEMENT * q, * t; q = head-> next; while  
(q != NULL)
```

```
{
```

```
t = q;
```

```
q = q-> next; free (t);
```

```
}
```

```
head-> next = NULL;
```

```
* last = head;
```

```
}
```

```
int main ()
```

```
{
```

```
ELEMENT * head, * last; / *the capital and  
last elements of the list * / int a;
```

```
CreateHead (& head, & last); / * create  
heading element* / printf ("a =");
```

```
scanf ("%d", & a);
```

```
/ * form the list * / while (a! = 0)
```

```
{
```

```
AddElement (& last, a); / *add another item  
to the list * / printf ("a =");
```

```
scanf ("% d", & a);
```

```
}
```

```
Print (head); / * print the list * /          printf  
("deleted x =");
```

```
scanf ("% d", & a);
```

```
/ * delete all elements of the list that have a  
value * / DeleteElement (head, & last, a);
```

```
Print (head); / * print a new list * /  Distruct  
(head, & last); / * delete the entire list * /  
return 0;
```

```
}
```

Example 2. You can limit yourself to one function for creating a list from the previous example, without writing a separate function AddElement (). Only this option is not

entirely convenient for a more general case - when, in the intervals between the operations of adding elements to the list, it is required to perform certain calculations or manipulations with the list.

Note that in this case the need for the variable last is dropped.

```
void CreateList (ELEMENT ** head)
{
    int a; ELEMENT * q;

    /* create heading element */

    * head = (ELEMENT *) malloc (sizeof
(ELEMENT)); (* head) -> next = NULL;

    q = * head; printf ("a =");

    scanf ("%d", & a);

    /* form the list */ while (a! = 0)
```

```

{

q-> next = (ELEMENT *) malloc (sizeof
(ELEMENT)); q = q-> next;

q-> data = a;

q-> next = NULL; printf ("a =");

scanf ("% d", & a);

}

}

```

Example 3. Generate a list of user-entered integers in ascending order and without repeating elements. Display the elements of the list on the screen and find the sum of all the elements of the list.

In this problem, it is not necessary to enter the last pointer variable to the last element of the list, since each time it is necessary to look through its elements, starting from the

beginning, in order to put a new element in such a way that all the conditions of the problem are satisfied. The Insert () function will be responsible for the operation of inserting a new element.

Also in this example, for a change, we will write a recursive function Print () to display all the elements of the list on the screen.

```
#include <stdio.h> #include <malloc.h>
```

```
typedef struct ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * next;
```

```
} ELEMENT;
```

```
void CreateHead (ELEMENT ** head)
```

```
{  
  
* head = (ELEMENT *) malloc (sizeof  
(ELEMENT)); (* head) -> next = NULL;  
  
}
```

```
/ * insert a new element with the value x * /  
void Insert (ELEMENT * head, int x)
```

```
{  
  
ELEMENT * q, * t; q = head;  
  
while (q-> next! = NULL && q-> next->  
data <x) q = q-> next;  
  
if (q-> next == NULL || x <q-> next-> data)  
  
{  
  
t = (ELEMENT *) malloc (sizeof  
(ELEMENT)); t-> data = x;  
  
t-> next = q-> next; q-> next = t;  
  
}
```

```
}
```

```
}
```

```
/* a recursive function that prints all the  
elements of the list to the screen
```

```
*/
```

```
void Print (ELEMENT * q)
```

```
{
```

```
if (q != NULL)
```

```
{
```

```
printf ("%d", q->data); Print (q->next);
```

```
}
```

```
}
```

```
/* sum of list items */ int Sum (ELEMENT  
* head)
```

```
{
```

```
    ELEMENT * q;
```

```
    int s = 0;
```

```
    q = head-> next; while (q! = NULL)
```

```
    {
```

```
        s += q-> data; q = q-> next;
```

```
    }
```

```
    return s;
```

```
}
```

```
/* remove the list from memory */ void  
Distruct (ELEMENT * head)
```

```
{
```

```
ELEMENT * q, * t; q = head-> next; while  
(q! = NULL)
```

```
{
```

```
t = q;
```

```
q = q-> next; free (t);
```

```
}
```

```
head-> next = NULL;
```

```
}
```

```
int main ()
```

```
{
```

```
ELEMENT * head;
```

```
int a; CreateHead (& head); printf ("a =");
```

```
scanf ("% d", & a); while (a! = 0)
```

```
{
```

```

Insert (head, a); printf ("a =");

scanf ("%d", & a);

}

Print (head-> next); printf ("\n");

printf ("sum =%d\n", Sum (head)); Destruct
(head);

return 0;

}

```

Example 4. Below is an algorithm for sequential search for an element with a given value x . The Find function returns the address of the first element with a value x . If the element with the x value is not in the list, then the function will return NULL.

```

ELEMENT * Find (ELEMENT * head, int x)

```

```
{  
ELEMENT * q;  
  
q = head-> next;  
  
while (q! = NULL && q-> data! = x) q = q->  
next;  
  
return q;  
  
}
```

Example 5. Let there be a list of integers, the elements of which are ordered in ascending order. Write a function that adds an element with the value x to the given list so that the list remains ordered.

```
void Insert (ELEMENT * head, int x)
```

```

{
ELEMENT * q, * t; q = head;

while (q-> next! = NULL && q-> next->
data <x) q = q-> next;

t = (ELEMENT *) malloc (sizeof
(ELEMENT)); t-> data = x;

t-> next = q-> next; q-> next = t;

}

```

Example 6. Count the maximum number of identical items in a list.

```

long MaxPovt (ELEMENT * head)

{

ELEMENT * q, * t; long max, n;

```



```
q = head-> next; max = 0;
while (q! = NULL)
{
n = 1;
t = q-> next;
while (t! = NULL)
{
if (q-> data == t-> data) n ++;
t = t-> next;
}
if (n> max) max = n;
q = q-> next;
}
return max;
```

```
}
```

Example 7. Write a function that determines whether the list with a heading is empty.

```
int IsEmpty (ELEMENT * head)
```

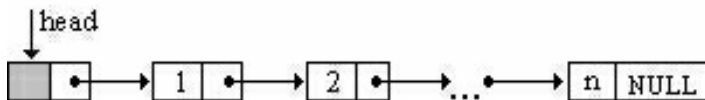
```
{
```

```
return (head-> next == NULL);
```

```
}
```

Example 8. Write a function that turns the list over, that is, the first element (excluding the capitalized one) becomes the last, the second - the penultimate, and so on.

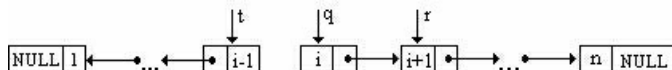
Let the original list look like this:



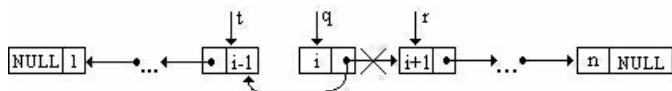
The algorithm consists in writing the address of the previous element in the reference field of the next element q , which stores the address of the next element after q .

At the first step, write the NULL value into the reference field of the first element (excluding the capitalization). The i -th step can be schematically depicted as follows.

a) The variable t contains the address of the last element of the new list, and the variable q contains the address of the next element of the original list, in the reference field of which you want to write the address of the element t :

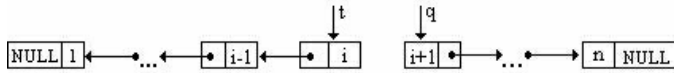


b) Change the value of the reference field of the element q :



c) Redefine the values of the t and q

elements:



Below is the algorithm for flipping the list.

```
void Reverse (ELEMENT * head)
```

```
{
```

```
    ELEMENT * q, * r, * t; q = head-> next;
```

```
    t = NULL;
```

```
    while (q != NULL)
```

```
    {
```

```
        r = q-> next; q-> next = t; t = q;
```

```
        q = r;
```

```
    }
```

```
    head-> next = t;
```

}

Singly linked list tasks

1. Find the number of maximum elements of the list of real numbers.
2. Write a function that, from the list L, builds two new lists: L1 - from positive elements and L2 - from negative elements of L.
3. Determine if the list is in ascending order.
4. Generate a list of user-entered integers in the order in which these numbers are entered, but without repetition of elements.

5. Write a function that leaves only the first occurrences of identical elements in the list L.

6. Determine the number of distinct elements of a list of real numbers if it is known that its elements form an ascending sequence.

7. There is a list of integers. Duplicate all even numbers in it.

8. Write a function that, based on two given linear lists, forms a new list consisting of elements that are simultaneously included in both data in the list.

9. Write a function that, using two linear lists L1 and L2, forms a new list L, consisting of elements included in L1, but not included in L2.

10. Write a function that leaves only one from each group of consecutive identical elements in a linear list.

11. Write a function that removes elements from a list that are included in it only once.

12. Let there be a list L1 of real numbers. Write to list L2 all elements of list L1 in ascending order of their values.

13. Let there be a list of real numbers $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_p$. Generate a new list $b_1 \rightarrow b_2 \rightarrow \dots \rightarrow b_n$ of the same dimension according to the following rule: element b_k is equal to the sum of elements of the original list with numbers from 1 to k .

Self-study tasks

14. Write a function that, having received as a parameter a pointer q to one of the elements of the list and some number x , adds a new element with the value x after (before) the element pointed to by the reference q .

15. Write a function that, having received as a parameter a pointer q to one of the elements of the list, deletes the element located after the element (the element itself) pointed to by the reference q .

16. Generate a list of real numbers. Then transform it by adding the maximum element to the positive numbers.

17. Remove from the list all items that appear more than once.

18. A list of integers is given.

Duplicate all prime numbers in it.

19. Determine if the list of real numbers contains elements greater than the sum of all the elements in the list.

20. Determine if the elements of the list of real numbers form a geometric progression.

21. Remove all maximum elements from the list of real numbers.

22. Let there be two lists, the elements of which are ordered in ascending order. Form a new list from the elements of the first and second list, the elements of which will be ordered.

23. Generate a list of real numbers $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n$, entered by the user. Then form a new list $b_1 \rightarrow b_2 \rightarrow \dots \rightarrow b_n$ of the same dimension according to the following rule: the element b_k is the maximum of the elements of the original list with numbers

from 1 to k.

24. Let there be a text file. Using a linear list, count the number of occurrences of each word and create a new text file, each line of which looks like "<word> -

<number of his appearances> ". Words must be in lexicographic order.

25. A looped list is a list whose last item points to the first. This list allows the program to cycle through it over and over again for as long as needed. Write a function for adding an item to a cyclic list and a function for printing all items in the list, starting with some item.

26. Write n user-entered integers to a cyclic list. Also write a function that receives as parameters a pointer to one of the elements of the list q and an integer $k > 0$. This function runs through the elements of the list, starting from the element q, and stops at the kth element, which is removed, and the count

starts with the next element, and so on until there is one element left in the list. The function returns the value of the given element.

Stacks, queues

A stack is a finite ordered set of elements located in heap, in which the addition of new elements and removal of existing ones is performed only at one end of it, which is called the top of the stack.

The stack is a special case of a linear, simply connected list. Due to the importance of stacks and queues, they were given their own names. The following operations are performed on the stack:

- 1) adding a new element to the stack;

2) access to the last enabled
item
stack);

3) the last included item was
popped from the stack. Stacks are very often
used by compilers to track

living functions and their parameters.
Whenever a function is called, the function
must know how to return to the calling
function, so the return address is pushed onto
the stack. If there are a number of
consecutive calls, then return addresses are
pushed on the stack in this last-in-first-out
order, so that after each function finishes
executing, it jumps to the function that called
it.

Example 1. In the following example, a stack
is created, and its contents are displayed on
the screen, and after the information is
retrieved from the next element of the stack,

this element is removed.

```
#include <stdio.h> #include <malloc.h>
```

```
typedef struct ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * next;
```

```
} ELEMENT;
```

```
/* add an item to the top of the stack */ void
```

```
Push (ELEMENT ** first, int x)
```

```
{
```

```
ELEMENT * q;
```

```
q = (ELEMENT *) malloc (sizeof  
(ELEMENT)); q-> data = x;
```

```
q-> next = * first;
```

```
* first = q;
```

```
}
```

/ * Extract information from the top of the stack and then remove this top. The information from the top of the stack, first, is pushed into element x, and the top is removed.

```
* /
```

```
int Extract (ELEMENT ** first, int * x)
```

```
{
```

```
    ELEMENT * q;
```

```
    if (* first == NULL) return 0;
```

```
    * x = (* first) -> data; q = * first;
```

```
    * first = (* first) -> next; free (q);
```

```

return 1;

}

int main ()

{

ELEMENT * first = NULL; / * top of the
stack * / int i, x;

for (i = 0; i <10; i ++) Push (& first, i);

while (Extract (& first, & x)) printf ("x =% d
\n", x);

return 0;

}

```

Another special kind of linear singly linked list is a queue. The following operations are performed on the queue:

- 1) adding a new element to the

end of the queue;

2) access to the first element
of the queue;

3) dequeuing the first item.

Example 2. Let's create a queue of integers and display its contents on the screen; moreover, after extracting information from the next element of the queue, we will delete this element.

```
#include <stdio.h> #include <malloc.h>
```

```
typedef struct ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * next;
```

```
} ELEMENT;
```



```

/* add an item to the end of the queue */

void AddElement (ELEMENT ** first,
ELEMENT ** last, int x)

{
ELEMENT * q;

q = (ELEMENT *) malloc (sizeof
(ELEMENT)); q-> data = x;

q-> next = NULL; if (* last == NULL)
* first = q;

else (* last) -> next = q;

* last = q;

}

/* retrieve information from the first element

```

of the queue with the subsequent removal of this vertex

```
* /
```

```
int Extract (ELEMENT ** first, ELEMENT  
** last, int * x)
```

```
{
```

```
    ELEMENT * q;
```

```
    if (* first == NULL) return 0;
```

```
    * x = (* first) -> data; q = * first;
```

```
    * first = (* first) -> next; if (* first == NULL)
```

```
    * last = NULL; free (q);
```

```
    return 1;
```

```
}
```

```
int main ()
```

```

{
int i, x;

ELEMENT * first = NULL, * last = NULL;
for (i = 0; i <10; i ++)

AddElement (& first, & last, i); while
(Extract (& first, & last, & x))

printf ("x =% d \ n", x); return 0;
}

```

Tasks per stacks and queues

1. Form a stack of integers entered by the user, and then display its contents on the screen without removing the elements from which information is retrieved. Also write a function to remove the stack.

2. Reverse the elements of an integer array of size n using the stack.

3. Write a function that prints out words in a text file in reverse order. The file can only be traversed once.

4. Write the elements of the integer array to the queue. Write a function to extract items from the queue until the first item in the queue becomes even.

5. You are given two non-empty queues that contain the same number of elements. Combine queues into one in which the elements of the original queues alternate.

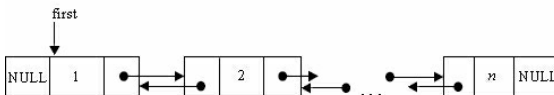
6. Two non-empty queues are given. The items in each of the queues are sorted in ascending order. Combine queues into one while maintaining the ordering of elements.

7. Let there be a file of real numbers and some number

C... Using a queue, print first all elements less than C, and then all other elements.

Doubly linked lists

For more flexible work with linear lists, each element can be provided with an additional reference field, so that each element contains links to two adjacent elements:



In this case, the operation of adding and removing a list item is simplified, and you can also explore the list in any direction.

Doubly linked list problems

1. Let there be a singly linked list of

real numbers, each element of which contains an additional (unrealized) prev reference field. Convert the original singly linked list into a doubly linked list, in which each element is associated not only with the next element (using the next field), but also with the previous one (using the prev field).

2. Write a function that, using an arbitrary pointer to one of the elements of a doubly linked list, counts the number of elements in this list.

3. Write a function that, having received as a parameter a pointer to one of the elements of a doubly linked list of real numbers and two numbers, adds the first number to the beginning of the list, and the second to its end.

4. A number x and a pointer q to one of the elements of a non-empty doubly linked list are given. Insert a new item with the value x after this list item.

5. Write a recursive function that displays all the elements of a doubly linked list in the opposite direction, starting from the last element.

6. There is a doubly linked list of real numbers. Duplicate all positive numbers in it.

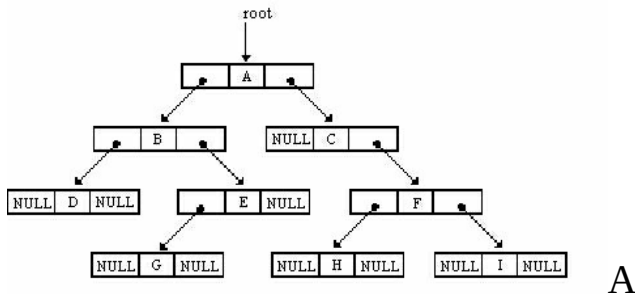
7. Some number x is given. Remove all elements with the x value from the doubly linked list.

8. Let there be some doubly linked list of real numbers and some number C . It is required to rearrange the values of the elements in such a way that first there follow (in an arbitrary order) elements that are less than C , and then elements that are not less than C .

9. Determine if the items in a doubly linked list are arranged symmetrically.

10. Cyclic shift of the elements of the doubly linked list by k positions to the right.

Binary trees



binary tree is a finite set of elements that is either empty or contains one element called the root of the tree, and the rest of the elements of the set are divided into two disjoint subsets, each of which is itself a binary tree. These subsets are called the right and left subtrees of the original tree.

This figure shows a tree with nine vertices. Vertex A is the root of the tree. The left subtree is rooted B, and the right subtree is

the second subtree is the root C. No branch means an empty subtree. For example, a subtree rooted with C does not have a left subtree, it is empty. The right subtree rooted in E is also empty. Binary subtrees rooted in D, G, H, and I have empty left and right subtrees.

A vertex that has empty left and right subtrees is called a terminal vertex or leaf. A nonterminal vertex is called an internal vertex.

A binary tree is called strictly binary if each of its non-leaf vertices has non-empty right and left subtrees.

The vertex v immediately below the vertex u is called the immediate descendant of u . If the vertex u is at the i -th level, then the child vertex v is at the $(i + 1)$ -th level. Levels are numbered starting from 0, that is, the root of the tree is at level zero, its immediate

descendants are at the first level, and so on. The maximum level of any of the vertices of the tree is called its height.

The length of the path to the vertex v is the number of branches (edges) that must be traversed from the root to the vertex v . That is, if the vertex is at the i -th level, then the length of the path to this vertex is equal to i .

A tree is said to be perfectly balanced if the number of vertices in its right and left subtrees differs by no more than one.

A binary tree is called a search tree if any vertex v of a given tree has the property that all keys in the left subtree of vertex v are less than the key of vertex v , and all keys in the right subtree of vertex v are greater than the key of vertex v . Note that the type of search tree corresponding to a dataset depends on the sequence in which the elements of this data are placed in the tree.

Each vertex of a binary tree in C is described as follows:

```

struct ELEMENT
{
    int data; / *           informational
    field*/

    struct ELEMENT * left; / *    pointer to the
    root of the left subtree * / struct ELEMENT *
    right; / *pointer to the root of the right
    subtree * /

};

```

Examples using binary trees

Example 1. Let an n-element array of integers be given. Form a perfectly balanced tree from the elements of the array a. Then remove the tree.

The Node function receives the number n of tree elements as input, creates a root vertex and builds recursively in the same way a left subtree with $nl = n / 2$ vertices and a right subtree with $nr = n - nl - 1$ vertices.

```
#include <stdio.h> #include <malloc.h>
```

```
typedef struct ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * left, * right;
```

```
} ELEMENT;
```

```
ELEMENT * Node (long n, int * a, long * pi)
```

```
{
```

```

if (n <= 0) return NULL;

else

{

long nl, nr; ELEMENT * q;

nl = n / 2; / * number of vertices in the left
subtree      * / nr = n - nl - 1; / * number
of vertices in the right subtree * / q =
(ELEMENT *) malloc (sizeof (ELEMENT));

q-> data = a [* pi]; (* pi) ++;

q-> left = Node (nl, a, pi); q-> right = Node
(nr, a, pi); return q;

}

}

/ * recursive      deletion wood * /

/ ** q - pointer to the root of the tree *
/      void Destruct (ELEMENT ** q)

```

```

{
if (* q! = NULL)

{
Distruct (& ((* q) -> left));

Distruct (& ((* q) -> right)); free (* q);
}
}

```

```

/ * print binary tree * / void Print
(ELEMENT * q, long n)

```

```

{
if (q! = NULL)

{
long i;

```

```

Print (q-> right, n + 5); for (i = 0; i <n; i ++)
printf ("");

printf ("% d \ n", q-> data); Print (q-> left, n
+ 5);

}

}

```

```

int main ()

{

ELEMENT * root = NULL;

long i;

int a [10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

i = 0;

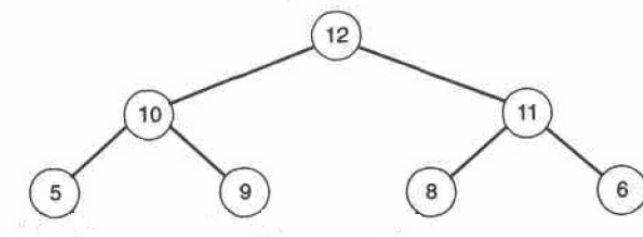
root = Node (10, a, & i); Print (root, 0);
Distruct (& root);

```

```
return 0;
```

```
}
```

In this program, the Print () function displays a binary tree on the screen. Moreover, the elements of the tree are displayed on the screen in layers, starting from the root vertex. Each level will be displayed vertically on the screen. For example, such a binary tree



will have the following presentation on the screen:

6

eleven

eight

12

nine

ten

5

Example 2. Let A be some property that an arbitrary element of the tree q either has or does not have (for example, the property to be a prime number for integral elements of the tree, the property to be a positive number, etc.). And let it be required to find the number of elements of a binary tree possessing the given property A . We define: $A(q) = 1$ if the element possesses property A , and $A(q) = 0$ otherwise. Then the function of counting the number of elements of a binary tree with property A will look like this:

long Count (ELEMENT * q)

```

{
if (q == NULL)

return 0;

else return A (q) + Count (q-> left) + Count
(q-> right);

}

```

For example, if it is required to find the number of elements of a binary tree, then we define property A for the top of the tree as the property of being a non-empty top. And then the function of counting the vertices of the binary tree will look like this:

```

long NodeCount (ELEMENT * q)

{

if (q == NULL)

return 0;

```

```
else return 1 + NodeCount (q-> left) +  
NodeCount (q-> right);  
  
}
```

The following function counts the number of leaves in a binary tree:

```
long LeafCount (ELEMENT * q)  
{  
    if (q == NULL)  
        return 0; else  
    {  
        if ((q-> left == NULL) && (q-> right ==  
NULL)) return 1;  
        else return LeafCount (q-> left) + LeafCount  
(q-> right);  
    }  
}
```

```
}
```

```
}
```

Example 3. The following function checks if a binary tree is perfectly balanced:

```
int Balance (ELEMENT * q)
```

```
{
```

```
if (q == NULL)
```

```
return 1; else
```

```
{
```

```
if (abs (NodeCount (q-> left) - NodeCount  
(q-> right))> 1) return 0;
```

```
else return (Balance (q-> left) && Balance  
(q-> right));
```

```
}
```

```
}
```

Example 4. Write a function for adding an element x to a perfectly balanced tree so that the resulting tree remains perfectly balanced.

```
void AddElement (ELEMENT ** q, int x)
{
    if (* q == NULL)
    {
        * q = (ELEMENT *) malloc (sizeof
        (ELEMENT)); (* q) -> data = x;
        (* q) -> left = (* q) -> right = NULL;
    }
    else
```

```

{

/ * if the number of elements in the left
subtree of the vertex q does not exceed the
number of vertices in the right subtree of this
vertex, then we go down the left branch,
otherwise - along the right

* /

if (NodeCount ((* q) -> left) <= NodeCount
((* q) -> right)) AddElement (& ((* q) ->
left), x);

else AddElement (& ((* q) -> right), x);

}

}

```

Example 5. Building a search tree for user-entered numbers.

The Insert function below adds a new element x to the search tree as follows. Compare the value of the current vertex q

with x. If the values match, then increase the count field of vertex q. If the value of the vertex q is less than x, then go down the right branch. If the value of the vertex q is greater than the value of x, then we go down the left branch. If the vertex q is empty, then we insert a new element with the value x at this position.

The Display function displays the values of all keys of the search tree in ascending order of their values with the number of repetitions.

```
#include <stdio.h> #include <malloc.h>
typedef struct ELEMENT
{
    int key; long count;
    struct ELEMENT * left, * right;
} ELEMENT;
```

```

void Insert (ELEMENT ** q, int x)

{

if (* q == NULL)

{

*   q   = (ELEMENT *) malloc  (sizeof
(ELEMENT)); (* q) -> key = x;

(* q) -> count = 1;

(* q) -> left = (* q) -> right = NULL;

}

else

{

if ((* q) -> key == x)

(((* q) -> count) ++; else

if ((* q) -> key > x) Insert (& ((* q) -> left),

```


x);

```
else Insert (& ((* q) -> right), x);
```

```
}
```

```
}
```

```
void Display (ELEMENT * q)
```

```
{
```

```
if (q! = NULL)
```

```
{
```

```
Display (q-> left);
```

```
printf ("% d:% d \ n", q-> key, q-> count);
```

```
Display (q-> right);
```

```
}
```

```
}
```

```

int main ()

{

ELEMENT * root = NULL;

int a; printf ("a =");

scanf ("% d", & a); while (a! = 0)

{

Insert (& root, a); printf ("a =");

scanf ("% d", & a);

}

Display (root); return 0;

}

```

Example 6. Following is a recursive and

iterative function that determines if element x is present in the search tree:

```
/* recursive element search function */
```

```
int Search (ELEMENT * q, int x)
```

```
{
```

```
/* iterative  
function search element */  
/
```

```
int Search2 (ELEMENT * root, int x)
```

```
{
```

```
if (q == NULL)
```

```
return 0; else
```

```
if (q-> key == x) return 1;
```

```
else
```

```
if (q-> key > x)
```

```
return Search (q-> left, x); else
```

```
return Search (q-> right, x);
```

```
}
```

```
}
```

```
ELEMENT * q = root; for (;;) 
```

```
{
```

```
if (q == NULL)
```

```
return 0; else
```

```
if (q->key == x) return 1;
```

```
else
```

```
if (q->key > x) q = q->left;
```

```
else q = q->right;
```

```
}
```

Example 7. Write a function for removing a vertex with a given key x from the search

tree.

When deleting a vertex, one of three cases can occur.

cases:

vertices with a key equal to x , no;

- the vertex with key x has at most one descendant;
- the vertex with key x has two children.

If the removed vertex is a leaf, then it is removed, and

the corresponding pointer at its parent is set to zero.

If the deleted vertex has only one child, then it is deleted, and the corresponding pointer in its parent vertex is not set to this child. In this

case, the child vertex takes the place of the deleted vertex in the tree.

In the third case, we proceed as follows. Let q be the removed vertex. Find the vertex Max of the left subtree of vertex q with the maximum key value. Max is the rightmost vertex in the left subtree of vertex q . Then we replace the key and counter of the vertex q with the corresponding values of the vertex Max. After that, delete the Max.


```
ELEMENT * Max (ELEMENT * q)
```

```
{
```

```
if (q == NULL)
```

```
return NULL; else
```

```
if (q-> right == NULL) return q;
```

```
else return Max (q-> right);
```

```
}
```

```
void Delete (ELEMENT ** q, int x)
```

```
{
```

```
ELEMENT * t;
```

```
if (* q != NULL)
```

```
{
```

```
if ((* q) -> key > x) Delete (& ((* q) -> left),
```

x);

else

if ((* q) -> key < x) Delete (& ((* q) -> right),
x);

else

{

if ((* q) -> left == NULL)

{

t = * q;

* q = (* q) -> right; free (t);

}

else

if ((* q) -> right == NULL)

{

t = * q;

```

* q = (* q) -> left; free (t);

}

else

{

t = Max ((* q) -> left); (* q) -> key = t-> key;
(* q) -> count = t-> count;

Delete (& ((* q) -> left), t-> key);

}

}

}

}

```

Example 8. Finding the height of an arbitrary binary tree.

```
long Height (ELEMENT * q)
```

```
{
```

```
if (q == NULL)
```

```
return -1; else
```

```
{
```

```
long i, j;
```

```
i = Height (q-> left);
```

```
j = Height (q-> right); if (i > j)
```

```
return i + 1; return j + 1;
```

```
}
```

```
}
```

Example 9. Determination of the number of elements at the k-th level of the tree.

```

/ * the variable level is responsible for the
number of the level of the top q * / long
Count (ELEMENT * q, long level, long k)

{

if (q == NULL || level > k) return 0;

else

if (level == k)

return 1;

else return Count (q-> left, level + 1, k) +
Count (q-> right, level + 1, k);

}

```

Example 10. Display all possible paths leading from the root to the leaves of a binary tree.

To solve this problem, we use an additional

structure - a doubly linked list, which will contain the elements of the next path of the binary tree. The ObhodTree () function works as follows. If the next vertex of the tree q is not zero, then

- 1) add the address of the vertex q to the list;

2) if vertex q is a leaf, then print the list, otherwise, if vertex q is not a leaf, then recursively

call the `ObhodTree()` function for the elements of the left subtree of the vertex `q`, and then for the elements of the right subtree of this vertex;

3) remove vertex q from the list.

```
#include <stdio.h> #include <malloc.h>
```

```
/* binary tree element */ typedef struct  
TREE_ELEMENT
```

```
{
```

```
int data;
```

```
struct TREE_ELEMENT * left, * right;
```

```
} TREE_ELEMENT;
```

```
/* doubly linked list item */ typedef struct  
LIST_ELEMENT
```

```
{
```

```
TREE_ELEMENT * node;
```

```
struct LIST_ELEMENT * next, * prev;
```

```
} LIST_ELEMENT;
```

```
/* create heading list item */
```

```
void CreateHead (LIST_ELEMENT ** head,  
LIST_ELEMENT ** last)
```

```
{
```

```
* head = (LIST_ELEMENT *) malloc (sizeof  
(LIST_ELEMENT)); (* head) -> prev = (*  
head) -> next = NULL;
```

```
* last = * head;
```

```
}
```

```
/* add the top of the tree t to the list */
```

```
void AddToList (LIST_ELEMENT ** last,  
TREE_ELEMENT * t)
```

```
{
```



```

LIST_ELEMENT * q;

q = (LIST_ELEMENT *) malloc (sizeof
(LIST_ELEMENT)); q-> node = t;

q-> next = NULL; q-> prev = * last; (* last) -
> next = q;

* last = q;

}

```

```

/ * print the list (display the next path) * /
void PrintList (LIST_ELEMENT * head)

{

LIST_ELEMENT * q;

q = head-> next; while (q!= NULL)

{

printf ("% d", q-> node-> data); q = q-> next;

```

```
}
```

```
printf ("\n");
```

```
}
```

```
/* deleting the last element from the list */  
void DeleteFromList (LIST_ELEMENT **  
last)
```

```
{
```

```
LIST_ELEMENT * q;
```

```
q = * last;
```

```
(* last) -> prev-> next = NULL;
```

```
(* last) = q-> prev; free (q);
```

```
}
```

```
/* traversing the elements of the binary tree  
and forming paths */ void ObhodTree
```

```

(TREE_ELEMENT * q, LIST_ELEMENT *
head,

LIST_ELEMENT ** last)

{

if (q! = NULL)

{

AddToList (last, q);

if (q-> left == NULL && q-> right ==
NULL) PrintList (head);

else

{

ObhodTree (q-> left, head, last); ObhodTree
(q-> right, head, last);

}

DeleteFromList (last);

}

```

```
}
```

```
int main ()
```

```
{
```

```
LIST_ELEMENT * head = NULL, * last =  
NULL; TREE_ELEMENT * root = NULL;
```

```
... /* create a binary tree */ CreateHead (&  
head, & last); ObhodTree (root, head, & last);
```

```
return 0;
```

```
}
```

Consider here one algorithm for solving this problem.

Only in this case, to store the tree vertices that are part of the next path, we use a dynamic one-dimensional array. Since the maximum length of all available paths from the root to the leaves is exactly equal to the height of the tree plus one, the dimension of the array will be equal to this number.

```
/ * binary tree element * / typedef struct  
ELEMENT
```

```
{
```

```
int data;
```

```
struct ELEMENT * left, * right;
```

```
} ELEMENT;
```

```
/ * display the next path * / void PrintRoute  
(ELEMENT ** a, long n)
```

```
{
```

```
long i;
```

```
for (i = 0; i <n; i ++) printf ("%d", a [i] ->  
data);
```

```
printf ("\n");
```

```
}
```

```

/ * traversal of elements of a binary tree and
formation of paths * / void ObhodTree
(ELEMENT * q, ELEMENT ** a, long * pn)

{

if (q! = NULL)

{

a [* pn] = q; / * write the
next vertex to array a * / (* pn) ++; / * in the
array one vertex became more * /

/ * if the top of the tree is a leaf, then display
the next path on the screen

* /

if (q-> left == NULL && q-> right ==
NULL) PrintRoute (a, * pn);

else

{

ObhodTree (q-> left, a, pn); ObhodTree (q->
right, a, pn);

```

}

(* pn) -; / *

remove q vertex

from array a * /

}

}

```

int main ()

{

ELEMENT * root = NULL; / * root of the
tree * / ELEMENT** a; / * array for storing
paths * / long n, /
* length of the next paths * /

h; / * height
wood * /

... / * create a binary tree * /

h = Height (root); / * calculate the height of
the tree * /

/ * allocate memory to store the tree nodes
that are part of the next path

* /

a = (ELEMENT **) malloc ((h + 1) * sizeof
(ELEMENT *)); n = 0;

ObhodTree (root, a, & n); free (a);

```



```
return 0;
```

```
}
```

Binary tree problems

1. Find the sum of the elements of a binary tree.
2. Find vertices whose number of children in the left subtree is not equal to the number of children in the right subtree.
3. Find vertices for which the height of the left subtree is not equal to the height of the right subtree.
4. Write a function that determines the number of occurrences of an element x in a binary tree.

5. Find the maximum element of a binary tree and the number of repetitions of the maximum element in the given tree.
6. Write a function that determines whether there are at least two identical elements in a binary tree.
7. Write a function that determines the maximum number of identical elements in a binary tree.
8. Write a function that determines whether a binary tree is symmetric.
9. Write a function that determines if a binary tree is a search tree.
10. Let there be a binary tree T . Form two ideally balanced trees from negative and non-negative elements of T .
11. Display all paths leading from the root to the leaves of a binary tree, for which

the total weight of elements is minimal.

12. Find the last number from all levels of the binary tree that have positive elements.

13. Find the maximum element at each level of the binary tree.

14. Find the number of inner vertices and the number of leaves at each level of the tree.

15. Find the sums of the elements of all odd levels.

16. Find the minimum and maximum paths between the leaves of a binary tree.

17. Remove the smallest number of vertices from a binary tree so that the resulting tree is strictly binary.

18. Let there be a text file. Using the search tree, create another text file - a

frequency dictionary containing words and the number of occurrences of each word in the source file.

19. Write a function that performs a layer-by-layer traversal of a binary tree, in which the values of the vertices are printed from level to level, starting from the root vertex. The values of the tree nodes at each level are printed from left to right. To implement the algorithm, use the queue structure as follows. At the first step, insert the root vertex into the queue. Then write a cycle that works according to this principle.

Until the queue is empty:

- 1) take the first element q from the queue;
- 2) display the value of the element q on the screen;
- 3) if vertex q has a left child vertex in the tree, then add this child to

the queue;

4) if vertex q has a right child vertex in the tree, then add this child to the queue;

5) remove element q from the queue.